Project-3

Embedded C Code Implementation Guide

Document Type	Per-Component Implementation Guide
Standard	IEEE 1016 SDD -> Embedded C
Target MCU Family	STM32
Target Files	main.c and main.h ONLY
Global Rules	20 rules - applied to ALL components
Total Components	134
Date	March 30, 2026
Prepared By	Prompt Builder Agent (AI-Generated)

Table of Contents

Table of Contents	2
1. Global Coding Rules	8
2. Hard File Rule	10
3. DD-COMP-001 - ControlLoopTask	11
4. DD-COMP-002 - PIDControlAlgorithm	15
5. DD-COMP-003 - ADCSampler	19
6. DD-COMP-004 - UARTCommandHandler	23
7. DD-COMP-005 - CommandParser	27
8. DD-COMP-006 - UARTDiagnosticsService	31
9. DD-COMP-007 - UARTDriverConfig	35
10. DD-COMP-008 - ConfigService	39
11. DD-COMP-009 - ConfigStore	43
12. DD-COMP-010 - ADCValidationService	47
13. DD-COMP-011 - UARTFaultHandler	51
14. DD-COMP-012 - FaultManager	55
15. DD-COMP-013 - SafeStateController	59
16. DD-COMP-014 - CalibrationStore	63
17. DD-COMP-015 - MISRA Compliance Process	67
18. DD-COMP-016 - Supply Voltage Assumption	71
19. DD-COMP-017 - ADC Channel Configuration	75
20. DD-COMP-018 - ADC Read Service	79
21. DD-COMP-019 - Position Scaling Algorithm	83

22. DD-COMP-020 - Control Output Calculator 87
23. DD-COMP-021 - Control Loop Scheduler 91
24. DD-COMP-022 - Feedback Fault Monitor 95
25. DD-COMP-023 - Fault Reporter 99
26. DD-COMP-024 - Actuator Drive Output 103
27. DD-COMP-025 - ThresholdConfigRepository 107
28. DD-COMP-026 - ThresholdConfigService 111
29. DD-COMP-027 - ShippingPositionStopLogic 115
30. DD-COMP-028 - UartCommandParser 119
31. DD-COMP-029 - UartCommandParser_OFF 123
32. DD-COMP-030 - ActuatorStateMachine 127
33. DD-COMP-031 - ActuatorStateMachine_OFF 131
34. DD-COMP-032 - UartAckService 135
35. DD-COMP-033 - AdcDriverConfig 139
36. DD-COMP-034 - AdcSamplingScheduler 143
37. DD-COMP-035 - AdcContinuousAcquisition 147
38. DD-COMP-036 - ADC Sample Buffer 151
39. DD-COMP-037 - Flap Control Computation 155
40. DD-COMP-038 - Actuation Range Guard 159
41. DD-COMP-039 - Real-Time ADC Freshness Check 163
42. DD-COMP-040 - Open-Loop Inhibit Gate 167
43. DD-COMP-041 - ADC Validity Monitor 171
44. DD-COMP-042 - Drive Permission Check 175

45. DD-COMP-043 - Target ADC Validator 179
46. DD-COMP-044 - ActuatorSafetyMonitor 183
47. DD-COMP-045 - ActuatorTargetStopController 187
48. DD-COMP-046 - SupplyVoltageADCChannelConfig 191
49. DD-COMP-047 - SupplyVoltageScaler 195
50. DD-COMP-048 - SupplyVoltageSampler 199
51. DD-COMP-049 - OvervoltageThresholdStore 203
52. DD-COMP-050 - OvervoltageDetector 207
53. DD-COMP-051 - OvervoltageDriveDisabler 211
54. DD-COMP-052 - ActuatorCommandGate 215
55. DD-COMP-053 - FaultTransitionHandler 219
56. DD-COMP-054 - OvervoltageUartNotifier 223
57. DD-COMP-055 - FaultStateRetentionLogic 227
58. DD-COMP-056 - FaultExitVoltageChecker 231
59. DD-COMP-057 - OvervoltageMonitor 235
60. DD-COMP-058 - UartActuatorCommandInterface 239
61. DD-COMP-059 - ActuatorOnCommandParser 243
62. DD-COMP-060 - UART Command Parser 247
63. DD-COMP-061 - Command Dispatcher 251
64. DD-COMP-062 - Actuator Controller 255
65. DD-COMP-063 - Invalid Command Handler 259
66. DD-COMP-064 - Invalid Command Logger 263
67. DD-COMP-065 - Voltage Configuration Store 267

68. DD-COMP-066 - Voltage Range Configuration 271
69. DD-COMP-067 - Startup Safety Manager 275
70. DD-COMP-068 - StartupDelayManager 279
71. DD-COMP-069 - ADCResolutionConfigurator 283
72. DD-COMP-070 - ADCReferenceConfigurator 287
73. DD-COMP-071 - ADCSamplingScheduler 291
74. DD-COMP-072 - ADCSaturationDetector 295
75. DD-COMP-073 - ADCPinInitializer 299
76. DD-COMP-074 - ActuatorGPIOInitializer 303
77. DD-COMP-075 - PWMTimerInitializer 307
78. DD-COMP-076 - UART Peripheral Initializer 311
79. DD-COMP-077 - ADC DMA Configuration Service 315
80. DD-COMP-078 - Independent Watchdog Service 319
81. DD-COMP-079 - Clock and Timer Configuration 323
82. DD-COMP-080 - Layered Architecture Enforcement Interface 327
83. DD-COMP-081 - System State Machine 331
84. DD-COMP-082 - Reset Initialization Handler 335
85. DD-COMP-083 - Nominal Transition Controller 339
86. DD-COMP-084 - FaultStateManager 343
87. DD-COMP-085 - HardwareWatchdogInitializer 347
88. DD-COMP-086 - WatchdogServicer 351
89. DD-COMP-087 - FatalMalfunctionResetHandler 355
90. DD-COMP-088 - ADCOnlyControlAlgorithm 359

91. DD-COMP-089 - ControlLoopScheduler 363
92. DD-COMP-090 - ThresholdStopEvaluator 367
93. DD-COMP-091 - HysteresisBandManager 371
94. DD-COMP-092 - RawAdcDataPolicy 375
95. DD-COMP-093 - UartStatusScheduler 379
96. DD-COMP-094 - UartStatusMessageBuilder 383
97. DD-COMP-095 - UartConfigService 387
98. DD-COMP-096 - UartBaudRateConfig 391
99. DD-COMP-097 - UartFrameFormatConfig 395
100. DD-COMP-098 - UartTransmitIntervalValidator 399
101. DD-COMP-099 - OvercurrentMonitor 403
102. DD-COMP-100 - ThermalMonitor 407
103. DD-COMP-101 - StallDetectionService 411
104. DD-COMP-102 - ShortCircuitDetector 415
105. DD-COMP-103 - UndervoltageMonitor 419
106. DD-COMP-104 - WatchdogService 423
107. DD-COMP-105 - FaultStateTransitionHandler 427
108. DD-COMP-106 - TemperatureProcessing 431
109. DD-COMP-107 - NvmConfigStorage 435
110. DD-COMP-108 - HumidityIndependencePolicy 439
111. DD-COMP-109 - AdcAcquisitionService 443
112. DD-COMP-110 - ControlLoopScheduler 447
113. DD-COMP-111 - ADCReader 451

114. DD-COMP-112 - CommandCalculator 455
115. DD-COMP-113 - ShippingLimitGuard 459
116. DD-COMP-114 - UARTCommandHandler 463
117. DD-COMP-115 - ActuatorEnableController 467
118. DD-COMP-116 - FaultMonitor 471
119. DD-COMP-117 - FaultInputProvider 475
120. DD-COMP-118 - FaultLogger 479
121. DD-COMP-119 - FaultLogStore 483
122. DD-COMP-120 - SystemStateMachine 487
123. DD-COMP-121 - FaultToStateTransitionService 491
124. DD-COMP-122 - RecoveryCommandHandler 495
125. DD-COMP-123 - FaultStatusTracker 499
126. DD-COMP-124 - StateStatusProvider 503
127. DD-COMP-125 - StateStatusAPI 507
128. DD-COMP-126 - ArchitectureDiagramExporter 511
129. DD-COMP-127 - DocumentationRepositoryAdapter 515
130. DD-COMP-128 - SchematicPdfExporter 519
131. DD-COMP-129 - PcbGerberExporter 523
132. DD-COMP-130 - FirmwareReleaseTagger 527
133. DD-COMP-131 - CalibrationDocIncluder 531
134. DD-COMP-132 - TestReportGenerator 535
135. DD-COMP-133 - DemonstrationVideoIncluder 539
136. DD-COMP-134 - ReleasePackageBuilder 543

1. Global Coding Rules

These rules apply to **every component** in this project without exception. Violations must be treated as build errors.

#	Rule
1	MISRA-C 2012 compliance required.
2	No dynamic memory allocation - malloc, calloc, realloc, free are forbidden.
3	No recursion of any kind.
4	Single entry point, single exit point per function.
5	Maximum 50 lines of executable code per function.
6	All functions must have a Doxygen comment: @brief, @param, @return, @trace.
7	All global variables must be static in main.c, extern-declared in main.h.
8	Use fixed-width types only: uint8_t, uint16_t, uint32_t, int8_t, int16_t, int32_t, bool.
9	All #define macros must be UPPER_SNAKE_CASE.
10	All function names must be lower_snake_case.
11	Struct and typedef names must be PascalCase with _t suffix (e.g. SensorData_t).
12	Error codes must use a typedef enum, not magic numbers.
13	Hardware register access must use volatile pointers or CMSIS/HAL macros.
14	ISR functions must be prefixed ISR_ and must only set a flag then return.
15	No floating-point unless hardware FPU is confirmed available.
16	All arrays must be fixed-size; size defined by a #define constant.
17	All pointer parameters must be checked for NULL before dereferencing.
18	No implicit type conversions - cast explicitly.
19	Code files are main.c and main.h ONLY - no new files.

20	Every function declaration in main.h, definition in main.c.

2. Hard File Rule

ABSOLUTE CONSTRAINT - CODE CHANGES ONLY IN main.c AND main.h:

```
- All new functions, declarations, includes, macros, typedefs, and global
variables derived from this design MUST be placed exclusively in main.c
or main.h.
- Do NOT create any new .c or .h files.
- Do NOT modify any existing file other than main.c or main.h.
- Do NOT move or refactor code that already exists in other files.
- If a component logically belongs in a separate module, implement it
as a static function inside main.c and declare it in main.h.
- This rule overrides all other code organisation preferences.
```

3. DD-COMP-001 - ControlLoopTask

DD-COMP-001	**ControlLoopTask** | Service | Module: ControlModule | Pattern: Observer | SW-REQ-001

`@trace DD-COMP-001 SW-REQ-001`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void ctl_init(void)`	void	Initialise all state for ControlLoopTask
`main.c`	`static uint8_t ctl_process(void *sensor_data)`	uint8_t - 0 success, non-zero error code	Execute core logic for ControlLoopTask
`main.c`	`static uint8_t read_inputs(void)`	uint8_t	Helper function: read inputs
`main.c`	`static uint8_t compute_control(void)`	uint8_t	Helper function: compute control
`main.c`	`static uint8_t apply_outputs(void)`	uint8_t	Helper function: apply outputs

Doxygen Comments

```
/**
* @brief Initialise ControlLoopTask.
* @trace DD-COMP-001 SW-REQ-001
*/
/**
* @brief Run ControlLoopTask process cycle.
* @param sensor_data Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-001 SW-REQ-001
*/
/**
* @brief Read inputs.
* @trace DD-COMP-001 SW-REQ-001
*/
/**
* @brief Compute control.
```

```
 * @trace DD-COMP-001 SW-REQ-001
 */
 /**
 * @brief Apply outputs.
 * @trace DD-COMP-001 SW-REQ-001
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	read sensor inputs
3	compute control command
4	update state
5	apply actuator outputs
6	log status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking calls in loop
2	unvalidated inputs
3	global state mutation
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ctl' (e.g. ctlresult).`

2	State struct: static ControlLoopTask_t ctlstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_001_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ControlLoopTask_t;`
	State struct for ControlLoopTask

`void ctl_init(void);`	Prototype: initialise ControlLoopTask
`uint8_t ctl_process(void *p_data);`	Prototype: run ControlLoopTask process cycle
`uint8_t read_inputs(void);`	Prototype: read inputs
`uint8_t compute_control(void);`	Prototype: compute control
`uint8_t apply_outputs(void);`	Prototype: apply outputs

`typedef enum {`
`DD_COMP_001_ERR_NONE = 0x00U,`
`DD_COMP_001_ERR_NULL = 0x01U,`
`DD_COMP_001_ERR_INVALID = 0x02U,`
`DD_COMP_001_ERR_TIMEOUT = 0x03U`
`} DD_COMP_001_ERR_t;`
	Error code enum for ControlLoopTask

main.c - What to Add

Item	Purpose

`static ControlLoopTask_t ctlstate = {0};`	Module-level zero-initialised state for ControlLoopTask

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: read sensor inputs`
`* Step 3: compute control command`
`* Step 4: update state`
`* Step 5: apply actuator outputs`
`* Step 6: log status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call ctl_init() once in main() init sequence. Call ctl_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

4. DD-COMP-002 - PIDControlAlgorithm

DD-COMP-002 **PIDControlAlgorithm** | Algorithm | Module: ControlModule | Pattern: N/A | SW-REQ-001

`@trace DD-COMP-002 SW-REQ-001`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void pid_init(void)`	void	Initialise all state for PIDControlAlgorithm
main.c	`static uint8_t pid_update(void *setpoint)`	uint8_t - 0 success, non-zero error code	Execute core logic for PIDControlAlgorithm
main.c	`static uint8_t calc_error(void)`	uint8_t	Helper function: calc error
main.c	`static uint8_t integrate_error(void)`	uint8_t	Helper function: integrate error
main.c	`static uint8_t deriv_error(void)`	uint8_t	Helper function: deriv error

Doxygen Comments

```
/**
* @brief Initialise PIDControlAlgorithm.
* @trace DD-COMP-002 SW-REQ-001
*/
/**
* @brief Run PIDControlAlgorithm process cycle.
* @param setpoint Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-002 SW-REQ-001
*/
/**
* @brief Calc error.
* @trace DD-COMP-002 SW-REQ-001
*/
/**
* @brief Integrate error.
```

```
 * @trace DD-COMP-002 SW-REQ-001
 */
 /**
 * @brief Deriv error.
 * @trace DD-COMP-002 SW-REQ-001
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read setpoint and measurement
2	compute control error
3	update integral term with limits
4	compute derivative term
5	calculate control output
6	apply output limits

Forbidden - Must NOT Do

#	Forbidden Rule
1	integral windup
2	derivative kick
3	output saturation without handling
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'pid' (e.g. pidresult).`

2	State struct: static PIDControl_t pidstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_002_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} PIDControl_t;`
	State struct for PIDControlAlgorithm

`void pid_init(void);`	Prototype: initialise PIDControlAlgorithm
`uint8_t pid_update(void *p_data);`	Prototype: run PIDControlAlgorithm process cycle
`uint8_t calc_error(void);`	Prototype: calc error
`uint8_t integrate_error(void);`	Prototype: integrate error
`uint8_t deriv_error(void);`	Prototype: deriv error

`typedef enum {`
`DD_COMP_002_ERR_NONE = 0x00U,`
`DD_COMP_002_ERR_NULL = 0x01U,`
`DD_COMP_002_ERR_INVALID = 0x02U,`
`DD_COMP_002_ERR_TIMEOUT = 0x03U`
`} DD_COMP_002_ERR_t;`
	Error code enum for PIDControlAlgorithm

main.c - What to Add

Item	Purpose

`static PIDControl_t pidstate = {0};`	Module-level zero-initialised state for PIDControlAlgorithm

`/* Process function logic order:`
`* Step 1: read setpoint and measurement`
`* Step 2: compute control error`
`* Step 3: update integral term with limits`
`* Step 4: compute derivative term`
`* Step 5: calculate control output`
`* Step 6: apply output limits`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call pid_init() once in main() init sequence. Call pid_update() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

5. DD-COMP-003 - ADCSampler

DD-COMP-003	**ADCSampler** | Service | Module: SensorModule | Pattern: Factory | SW-REQ-002

`@trace DD-COMP-003 SW-REQ-002`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void adcSampInit(void)`	void	Initialise all state for ADCSampler
`main.c`	`static uint8_t adcSampRun(void *adc_channels)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCSampler
`main.c`	`static uint8_t adcReadRaw(void)`	uint8_t	Helper function: adcReadRaw
`main.c`	`static uint8_t adcFilter(void)`	uint8_t	Helper function: adcFilter
`main.c`	`static uint8_t adcStore(void)`	uint8_t	Helper function: adcStore

Doxygen Comments

```
/**
* @brief Initialise ADCSampler.
* @trace DD-COMP-003 SW-REQ-002
*/
/**
* @brief Run ADCSampler process cycle.
* @param adc_channels Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-003 SW-REQ-002
*/
/**
* @brief Adcreadraw.
* @trace DD-COMP-003 SW-REQ-002
*/
/**
* @brief Adcfilter.
```

```
 * @trace DD-COMP-003 SW-REQ-002
 */
 /**
 * @brief Adcstore.
 * @trace DD-COMP-003 SW-REQ-002
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	check configuration
2	trigger ADC conversion
3	read raw samples
4	apply filtering or averaging
5	store or publish results
6	handle errors or timeouts

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking the main loop
2	unbounded buffer growth
3	ignoring ADC errors
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adc' (e.g. adcresult).`

2	State struct: static AdcSampler_t adcstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_003_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcSampler_t;`
	State struct for ADCSampler

`void adcSampInit(void);`	Prototype: initialise ADCSampler
`uint8_t adcSampRun(void *p_data);`	Prototype: run ADCSampler process cycle
`uint8_t adcReadRaw(void);`	Prototype: adcReadRaw
`uint8_t adcFilter(void);`	Prototype: adcFilter
`uint8_t adcStore(void);`	Prototype: adcStore

`typedef enum {`
`DD_COMP_003_ERR_NONE = 0x00U,`
`DD_COMP_003_ERR_NULL = 0x01U,`
`DD_COMP_003_ERR_INVALID = 0x02U,`
`DD_COMP_003_ERR_TIMEOUT = 0x03U`
`} DD_COMP_003_ERR_t;`
	Error code enum for ADCSampler

main.c - What to Add

Item	Purpose

`static AdcSampler_t adcstate = {0};`	Module-level zero-initialised state for ADCSampler

`/* Process function logic order:`
`* Step 1: check configuration`
`* Step 2: trigger ADC conversion`
`* Step 3: read raw samples`
`* Step 4: apply filtering or averaging`
`* Step 5: store or publish results`
`* Step 6: handle errors or timeouts`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call adcSampInit() once in main() init sequence. Call adcSampRun() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

6. DD-COMP-004 - UARTCommandHandler

DD-COMP-004	**UARTCommandHandler** | Service | Module: CommModule | Pattern: Command | SW-REQ-003

`@trace DD-COMP-004 SW-REQ-003`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void uartcmd_init(void)`	void	Initialise all state for UARTCommandHandler
`main.c`	`static uint8_t uartcmd_run(void *uart_rx)`	uint8_t - 0 success, non-zero error code	Execute core logic for UARTCommandHandler
`main.c`	`static uint8_t uartcmd_parse(void)`	uint8_t	Helper function: uartcmd parse
`main.c`	`static uint8_t uartcmd_exec(void)`	uint8_t	Helper function: uartcmd exec
`main.c`	`static uint8_t uartcmd_reply(void)`	uint8_t	Helper function: uartcmd reply

Doxygen Comments

```
/**
* @brief Initialise UARTCommandHandler.
* @trace DD-COMP-004 SW-REQ-003
*/
/**
* @brief Run UARTCommandHandler process cycle.
* @param uart_rx Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-004 SW-REQ-003
*/
/**
* @brief Uartcmd parse.
* @trace DD-COMP-004 SW-REQ-003
*/
/**
* @brief Uartcmd exec.
```

```
 * @trace DD-COMP-004 SW-REQ-003
 */
 /**
 * @brief Uartcmd reply.
 * @trace DD-COMP-004 SW-REQ-003
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read incoming UART bytes
2	assemble a complete command frame
3	validate command format and checksum
4	parse command and arguments
5	execute matching handler
6	send response or error
7	log or update status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UART reads
2	silent parse failures
3	tight coupling to hardware
4	unbounded command length
5	No malloc/calloc/free anywhere in this component
6	No recursive function calls - use iterative logic only
7	No processing inside ISR - set flag and return only
8	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	All local variables prefixed 'uartcmd' (e.g. uartcmdresult).
2	State struct: static UartCommandHandler_t uartcmdstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_004_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartCommandHandler_t;`
	State struct for UARTCommandHandler

`void uartcmd_init(void);`	Prototype: initialise UARTCommandHandler
`uint8_t uartcmd_run(void *p_data);`	Prototype: run UARTCommandHandler process cycle
`uint8_t uartcmd_parse(void);`	Prototype: uartcmd parse
`uint8_t uartcmd_exec(void);`	Prototype: uartcmd exec
`uint8_t uartcmd_reply(void);`	Prototype: uartcmd reply

`typedef enum {`
`DD_COMP_004_ERR_NONE = 0x00U,`
`DD_COMP_004_ERR_NULL = 0x01U,`
`DD_COMP_004_ERR_INVALID = 0x02U,`
`DD_COMP_004_ERR_TIMEOUT = 0x03U`
`} DD_COMP_004_ERR_t;`
	Error code enum for UARTCommandHandler

main.c - What to Add

Item	Purpose
`static UartCommandHandler_t uartcmdstate = {0};`	Module-level zero-initialised state for UARTCommandHandler

`/* Process function logic order:`
`* Step 1: read incoming UART bytes`
`* Step 2: assemble a complete command frame`
`* Step 3: validate command format and checksum`
`* Step 4: parse command and arguments`
`* Step 5: execute matching handler`
`* Step 6: send response or error`
`* Step 7: log or update status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Command. Call uartcmd_init() once in main() init sequence. Call uartcmd_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

7. DD-COMP-005 - CommandParser

DD-COMP-005	**CommandParser** | Class | Module: CommModule | Pattern: Builder | SW-REQ-003

`@trace DD-COMP-005 SW-REQ-003`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initParser(void)`	void	Initialise all state for CommandParser
main.c	`static uint8_t parseCommand(void *rawCommand)`	uint8_t - 0 success, non-zero error code	Execute core logic for CommandParser
main.c	`static uint8_t tokenize(void)`	uint8_t	Helper function: tokenize
main.c	`static uint8_t validateSyntax(void)`	uint8_t	Helper function: validateSyntax
main.c	`static uint8_t buildCommand(void)`	uint8_t	Helper function: buildCommand

Doxygen Comments

```
/**
* @brief Initialise CommandParser.
* @trace DD-COMP-005 SW-REQ-003
*/
/**
* @brief Run CommandParser process cycle.
* @param rawCommand Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-005 SW-REQ-003
*/
/**
* @brief Tokenize.
* @trace DD-COMP-005 SW-REQ-003
*/
/**
* @brief Validatesyntax.
```

```
 * @trace DD-COMP-005 SW-REQ-003
 */
 /**
 * @brief Buildcommand.
 * @trace DD-COMP-005 SW-REQ-003
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load parsing rules
2	split input into tokens
3	normalize tokens
4	validate command structure
5	map tokens to action
6	build command object
7	report errors if any

Forbidden - Must NOT Do

#	Forbidden Rule
1	global state mutations
2	silent error handling
3	ambiguous token rules

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'cmd' (e.g. cmdresult).`
2	`State struct: static CommandParser_t cmdstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`

5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_005_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} CommandParser_t;`
	State struct for CommandParser

`void initParser(void);`	Prototype: initialise CommandParser
`uint8_t parseCommand(void *p_data);`	Prototype: run CommandParser process cycle
`uint8_t tokenize(void);`	Prototype: tokenize
`uint8_t validateSyntax(void);`	Prototype: validateSyntax
`uint8_t buildCommand(void);`	Prototype: buildCommand

`typedef enum {`
`DD_COMP_005_ERR_NONE = 0x00U,`
`DD_COMP_005_ERR_NULL = 0x01U,`
`DD_COMP_005_ERR_INVALID = 0x02U,`
`DD_COMP_005_ERR_TIMEOUT = 0x03U`
`} DD_COMP_005_ERR_t;`
	Error code enum for CommandParser

main.c - What to Add

Item	Purpose
`static CommandParser_t cmdstate = {0};`	Module-level zero-initialised state for CommandParser

`/* Process function logic order:`
`* Step 1: load parsing rules`
`* Step 2: split input into tokens`
`* Step 3: normalize tokens`
`* Step 4: validate command structure`
`* Step 5: map tokens to action`
`* Step 6: build command object`
`* Step 7: report errors if any`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Builder. Call initParser() once in main() init sequence. Call parseCommand() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

8. DD-COMP-006 - UARTDiagnosticsService

DD-COMP-006	**UARTDiagnosticsService** | Service | Module: CommModule | Pattern: Facade | SW-REQ-004

`@trace DD-COMP-006 SW-REQ-004`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void uartDiagInit(void)`	void	Initialise all state for UARTDiagnosticsService
`main.c`	`static uint8_t uartDiagProcess(void *uart_rx_buf)`	uint8_t - 0 success, non-zero error code	Execute core logic for UARTDiagnosticsService
`main.c`	`static uint8_t readUartFrame(void)`	uint8_t	Helper function: readUartFrame
`main.c`	`static uint8_t parseDiagCmd(void)`	uint8_t	Helper function: parseDiagCmd
`main.c`	`static uint8_t sendDiagResp(void)`	uint8_t	Helper function: sendDiagResp

Doxygen Comments

```
/**
* @brief Initialise UARTDiagnosticsService.
* @trace DD-COMP-006 SW-REQ-004
*/
/**
* @brief Run UARTDiagnosticsService process cycle.
* @param uart_rx_buf Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-006 SW-REQ-004
*/
/**
* @brief Readuartframe.
* @trace DD-COMP-006 SW-REQ-004
*/
/**
* @brief Parsediagcmd.
```

```
 * @trace DD-COMP-006 SW-REQ-004
 */
 /**
 * @brief Senddiagresp.
 * @trace DD-COMP-006 SW-REQ-004
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize UART diagnostics state
2	read incoming UART bytes
3	assemble and validate diagnostic frame
4	parse command and update status
5	prepare response payload
6	send response over UART
7	log or flag errors

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UART reads
2	dynamic memory allocation
3	silent parse failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'diag' (e.g. diagresult).
2	State struct: static UartDiagnosticsService_t diagstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_006_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartDiagnosticsService_t;`
	State struct for UARTDiagnosticsService

`void uartDiagInit(void);`	Prototype: initialise UARTDiagnosticsService
`uint8_t uartDiagProcess(void *p_data);`	Prototype: run UARTDiagnosticsService process cycle
`uint8_t readUartFrame(void);`	Prototype: readUartFrame
`uint8_t parseDiagCmd(void);`	Prototype: parseDiagCmd
`uint8_t sendDiagResp(void);`	Prototype: sendDiagResp

`typedef enum {`
`DD_COMP_006_ERR_NONE = 0x00U,`
`DD_COMP_006_ERR_NULL = 0x01U,`
`DD_COMP_006_ERR_INVALID = 0x02U,`
`DD_COMP_006_ERR_TIMEOUT = 0x03U`
`} DD_COMP_006_ERR_t;`
	Error code enum for UARTDiagnosticsService

main.c - What to Add

Item	Purpose
`static UartDiagnosticsService_t diagstate = {0};`	Module-level zero-initialised state for UARTDiagnosticsService

`/* Process function logic order:`
`* Step 1: initialize UART diagnostics state`
`* Step 2: read incoming UART bytes`
`* Step 3: assemble and validate diagnostic frame`
`* Step 4: parse command and update status`
`* Step 5: prepare response payload`
`* Step 6: send response over UART`
`* Step 7: log or flag errors`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call uartDiagInit() once in main() init sequence. Call uartDiagProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

9. DD-COMP-007 - UARTDriverConfig

DD-COMP-007	**UARTDriverConfig** | Class | Module: CommModule | Pattern: Singleton | SW-REQ-005

`@trace DD-COMP-007 SW-REQ-005`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void uartCfgInit(void)`	void	Initialise all state for UARTDriverConfig
main.c	`static uint8_t uartCfgProcess(void *configData)`	uint8_t - 0 success, non-zero error code	Execute core logic for UARTDriverConfig
main.c	`static uint8_t loadDefaults(void)`	uint8_t	Helper function: loadDefaults
main.c	`static uint8_t applyBaud(void)`	uint8_t	Helper function: applyBaud
main.c	`static uint8_t validatePins(void)`	uint8_t	Helper function: validatePins

Doxygen Comments

```
/**
* @brief Initialise UARTDriverConfig.
* @trace DD-COMP-007 SW-REQ-005
*/
/**
* @brief Run UARTDriverConfig process cycle.
* @param configData Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-007 SW-REQ-005
*/
/**
* @brief Loaddefaults.
* @trace DD-COMP-007 SW-REQ-005
*/
/**
* @brief Applybaud.
```

```
 * @trace DD-COMP-007 SW-REQ-005
 */
 /**
 * @brief Validatepins.
 * @trace DD-COMP-007 SW-REQ-005
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load default UART settings
2	read user configuration
3	validate pin assignments
4	set baud rate and format
5	apply hardware registers
6	verify configuration status

Forbidden - Must NOT Do

#	Forbidden Rule
1	using global mutable state
2	hardcoding pin numbers
3	ignoring invalid baud rates

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uartCfg' (e.g. uartCfgresult).`
2	`State struct: static UartDriverConfig_t uartCfgstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_007_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartDriverConfig_t;`
	State struct for UARTDriverConfig

`void uartCfgInit(void);`	Prototype: initialise UARTDriverConfig
`uint8_t uartCfgProcess(void *p_data);`	Prototype: run UARTDriverConfig process cycle
`uint8_t loadDefaults(void);`	Prototype: loadDefaults
`uint8_t applyBaud(void);`	Prototype: applyBaud
`uint8_t validatePins(void);`	Prototype: validatePins

`typedef enum {`
`DD_COMP_007_ERR_NONE = 0x00U,`
`DD_COMP_007_ERR_NULL = 0x01U,`
`DD_COMP_007_ERR_INVALID = 0x02U,`
`DD_COMP_007_ERR_TIMEOUT = 0x03U`
`} DD_COMP_007_ERR_t;`
	Error code enum for UARTDriverConfig

main.c - What to Add

Item	Purpose
`static UartDriverConfig_t uartCfgstate = {0};`	Module-level zero-initialised state for UARTDriverConfig

`/* Process function logic order:`
`* Step 1: load default UART settings`
`* Step 2: read user configuration`
`* Step 3: validate pin assignments`
`* Step 4: set baud rate and format`
`* Step 5: apply hardware registers`
`* Step 6: verify configuration status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call uartCfgInit() once in main() init sequence. Call uartCfgProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

10. DD-COMP-008 - ConfigService

DD-COMP-008	**ConfigService** | Service | Module: ConfigModule | Pattern: Repository | SW-REQ-006

`@trace DD-COMP-008 SW-REQ-006`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void cfg_init(void)`	void	Initialise all state for ConfigService
main.c	`static uint8_t cfg_process(void *config_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for ConfigService
main.c	`static uint8_t cfg_load(void)`	uint8_t	Helper function: cfg load
main.c	`static uint8_t cfg_validate(void)`	uint8_t	Helper function: cfg validate
main.c	`static uint8_t cfg_save(void)`	uint8_t	Helper function: cfg save

Doxygen Comments

```
/**
* @brief Initialise ConfigService.
* @trace DD-COMP-008 SW-REQ-006
*/
/**
* @brief Run ConfigService process cycle.
* @param config_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-008 SW-REQ-006
*/
/**
* @brief Cfg load.
* @trace DD-COMP-008 SW-REQ-006
*/
/**
* @brief Cfg validate.
```

```
 * @trace DD-COMP-008 SW-REQ-006
 */
 /**
 * @brief Cfg save.
 * @trace DD-COMP-008 SW-REQ-006
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load existing configuration
2	merge incoming settings
3	validate values against schema
4	apply defaults for missing fields
5	persist updated configuration
6	return status and data

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	silent validation failures
3	blocking I/O in hot path
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'cfg' (e.g. cfgresult).`

2	State struct: static ConfigService_t cfgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_008_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ConfigService_t;`
	State struct for ConfigService

`void cfg_init(void);`	Prototype: initialise ConfigService
`uint8_t cfg_process(void *p_data);`	Prototype: run ConfigService process cycle
`uint8_t cfg_load(void);`	Prototype: cfg load
`uint8_t cfg_validate(void);`	Prototype: cfg validate
`uint8_t cfg_save(void);`	Prototype: cfg save

`typedef enum {`
`DD_COMP_008_ERR_NONE = 0x00U,`
`DD_COMP_008_ERR_NULL = 0x01U,`
`DD_COMP_008_ERR_INVALID = 0x02U,`
`DD_COMP_008_ERR_TIMEOUT = 0x03U`
`} DD_COMP_008_ERR_t;`
	Error code enum for ConfigService

main.c - What to Add

Item	Purpose

`static ConfigService_t cfgstate = {0};`	Module-level zero-initialised state for ConfigService

`/* Process function logic order:`
`* Step 1: load existing configuration`
`* Step 2: merge incoming settings`
`* Step 3: validate values against schema`
`* Step 4: apply defaults for missing fields`
`* Step 5: persist updated configuration`
`* Step 6: return status and data`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call cfg_init() once in main() init sequence. Call cfg_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

11. DD-COMP-009 - ConfigStore

DD-COMP-009	**ConfigStore** | Database | Module: ConfigModule | Pattern: Repository | SW-REQ-006

`@trace DD-COMP-009 SW-REQ-006`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init_cfg(void)`	void	Initialise all state for ConfigStore
`main.c`	`static uint8_t process_cfg(void *cfg_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for ConfigStore
`main.c`	`static uint8_t load_cfg(void)`	uint8_t	Helper function: load cfg
`main.c`	`static uint8_t save_cfg(void)`	uint8_t	Helper function: save cfg
`main.c`	`static uint8_t validate_cfg(void)`	uint8_t	Helper function: validate cfg

Doxygen Comments

```
/**
* @brief Initialise ConfigStore.
* @trace DD-COMP-009 SW-REQ-006
*/
/**
* @brief Run ConfigStore process cycle.
* @param cfg_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-009 SW-REQ-006
*/
/**
* @brief Load cfg.
* @trace DD-COMP-009 SW-REQ-006
*/
/**
* @brief Save cfg.
* @trace DD-COMP-009 SW-REQ-006
```

```
*/
/**
* @brief Validate cfg.
* @trace DD-COMP-009 SW-REQ-006
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration from storage
2	parse configuration data
3	validate configuration fields
4	apply defaults for missing values
5	persist updated configuration
6	return status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	silent validation failures
3	blocking I/O on main thread
4	No dynamic struct allocation
5	Array sizes must be compile-time #define constants
6	No pointer arithmetic on struct members

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'cfg' (e.g. cfgresult).`
2	`State struct: static ConfigStore_t cfgstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`

4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_009_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ConfigStore_t;`
	State struct for ConfigStore

`void init_cfg(void);`	Prototype: initialise ConfigStore
`uint8_t process_cfg(void *p_data);`	Prototype: run ConfigStore process cycle
`uint8_t load_cfg(void);`	Prototype: load cfg
`uint8_t save_cfg(void);`	Prototype: save cfg
`uint8_t validate_cfg(void);`	Prototype: validate cfg

`typedef enum {`
`DD_COMP_009_ERR_NONE = 0x00U,`
`DD_COMP_009_ERR_NULL = 0x01U,`
`DD_COMP_009_ERR_INVALID = 0x02U,`
`DD_COMP_009_ERR_TIMEOUT = 0x03U`
`} DD_COMP_009_ERR_t;`
	Error code enum for ConfigStore

main.c - What to Add

Item	Purpose
`static ConfigStore_t cfgstate = {0};`	Module-level zero-initialised state for ConfigStore

`/* Process function logic order:`
`* Step 1: load configuration from storage`
`* Step 2: parse configuration data`
`* Step 3: validate configuration fields`
`* Step 4: apply defaults for missing values`
`* Step 5: persist updated configuration`
`* Step 6: return status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call init_cfg() once in main() init sequence. Call process_cfg() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

12. DD-COMP-010 - ADCValidationService

DD-COMP-010	**ADCValidationService** | Service | Module: SafetyModule | Pattern: Chain of Responsibility | SW-REQ-007

`@trace DD-COMP-010 SW-REQ-007`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void initAdcVal(void)`	void	Initialise all state for ADCValidationService
`main.c`	`static uint8_t processAdcVal(void *adcSamples)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCValidationService
`main.c`	`static uint8_t checkRange(void)`	uint8_t	Helper function: checkRange
`main.c`	`static uint8_t filterNoise(void)`	uint8_t	Helper function: filterNoise
`main.c`	`static uint8_t logStatus(void)`	uint8_t	Helper function: logStatus

Doxygen Comments

```
/**
* @brief Initialise ADCValidationService.
* @trace DD-COMP-010 SW-REQ-007
*/
/**
* @brief Run ADCValidationService process cycle.
* @param adcSamples Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-010 SW-REQ-007
*/
/**
* @brief Checkrange.
* @trace DD-COMP-010 SW-REQ-007
*/
/**
```

```
 * @brief Filternoise.
 * @trace DD-COMP-010 SW-REQ-007
 */
 /**
 * @brief Logstatus.
 * @trace DD-COMP-010 SW-REQ-007
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration and thresholds
2	read raw ADC samples
3	apply filtering to samples
4	validate values against limits
5	flag anomalies and update status
6	emit validation report

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking on I/O
2	hardcoded thresholds
3	silent error handling
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcVal' (e.g. adcValresult).`

2	State struct: static AdcValidationService_t adcValstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_010_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcValidationService_t;`
	State struct for ADCValidationService

`void initAdcVal(void);`	Prototype: initialise ADCValidationService
`uint8_t processAdcVal(void *p_data);`	Prototype: run ADCValidationService process cycle
`uint8_t checkRange(void);`	Prototype: checkRange
`uint8_t filterNoise(void);`	Prototype: filterNoise
`uint8_t logStatus(void);`	Prototype: logStatus

`typedef enum {`
`DD_COMP_010_ERR_NONE = 0x00U,`
`DD_COMP_010_ERR_NULL = 0x01U,`
`DD_COMP_010_ERR_INVALID = 0x02U,`
`DD_COMP_010_ERR_TIMEOUT = 0x03U`
`} DD_COMP_010_ERR_t;`
	Error code enum for ADCValidationService

main.c - What to Add

Item	Purpose

`static AdcValidationService_t adcValstate = {0};`	Module-level zero-initialised state for ADCValidationService

`/* Process function logic order:`
`* Step 1: load configuration and thresholds`
`* Step 2: read raw ADC samples`
`* Step 3: apply filtering to samples`
`* Step 4: validate values against limits`
`* Step 5: flag anomalies and update status`
`* Step 6: emit validation report`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Chain of Responsibility. Call initAdcVal() once in main() init sequence. Call processAdcVal() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

13. DD-COMP-011 - UARTFaultHandler

DD-COMP-011	**UARTFaultHandler** | Service | Module: SafetyModule | Pattern: Strategy | SW-REQ-007

`@trace DD-COMP-011 SW-REQ-007`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void uartFaultInit(void)`	void	Initialise all state for UARTFaultHandler
main.c	`static uint8_t uartFaultProcess(void *uartStatusReg)`	uint8_t - 0 success, non-zero error code	Execute core logic for UARTFaultHandler
main.c	`static uint8_t checkUartStatus(void)`	uint8_t	Helper function: checkUartStatus
main.c	`static uint8_t clearUartErrors(void)`	uint8_t	Helper function: clearUartErrors
main.c	`static uint8_t logUartFault(void)`	uint8_t	Helper function: logUartFault

Doxygen Comments

```
/**
* @brief Initialise UARTFaultHandler.
* @trace DD-COMP-011 SW-REQ-007
*/
/**
* @brief Run UARTFaultHandler process cycle.
* @param uartStatusReg Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-011 SW-REQ-007
*/
/**
* @brief Checkuartstatus.
* @trace DD-COMP-011 SW-REQ-007
*/
/**
* @brief Clearuarterrors.
```

```
 * @trace DD-COMP-011 SW-REQ-007
 */
 /**
 * @brief Loguartfault.
 * @trace DD-COMP-011 SW-REQ-007
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize fault state and counters
2	read UART status and error flags
3	detect and classify fault conditions
4	clear or reset faulted UART flags
5	record fault event and update metrics
6	notify system or higher layer if needed

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking in ISR context
2	clearing flags without logging
3	using dynamic memory
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ufh' (e.g. ufhresult).`

2	State struct: static UartFaultHandler_t ufhstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_011_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartFaultHandler_t;`
	State struct for UARTFaultHandler

`void uartFaultInit(void);`	Prototype: initialise UARTFaultHandler
`uint8_t uartFaultProcess(void *p_data);`	Prototype: run UARTFaultHandler process cycle
`uint8_t checkUartStatus(void);`	Prototype: checkUartStatus
`uint8_t clearUartErrors(void);`	Prototype: clearUartErrors
`uint8_t logUartFault(void);`	Prototype: logUartFault

`typedef enum {`
`DD_COMP_011_ERR_NONE = 0x00U,`
`DD_COMP_011_ERR_NULL = 0x01U,`
`DD_COMP_011_ERR_INVALID = 0x02U,`
`DD_COMP_011_ERR_TIMEOUT = 0x03U`
`} DD_COMP_011_ERR_t;`
	Error code enum for UARTFaultHandler

main.c - What to Add

Item	Purpose

`static UartFaultHandler_t ufhstate = {0};`	Module-level zero-initialised state for UARTFaultHandler

`/* Process function logic order:`
`* Step 1: initialize fault state and counters`
`* Step 2: read UART status and error flags`
`* Step 3: detect and classify fault conditions`
`* Step 4: clear or reset faulted UART flags`
`* Step 5: record fault event and update metrics`
`* Step 6: notify system or higher layer if needed`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Strategy. Call uartFaultInit() once in main() init sequence. Call uartFaultProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

14. DD-COMP-012 - FaultManager

DD-COMP-012	**FaultManager** | Service | Module: SafetyModule | Pattern: Observer | SW-REQ-007

`@trace DD-COMP-012 SW-REQ-007`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initFaults(void)`	void	Initialise all state for FaultManager
main.c	`static uint8_t processFaults(void *faultSignals)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultManager
main.c	`static uint8_t logFault(void)`	uint8_t	Helper function: logFault
main.c	`static uint8_t updateStatus(void)`	uint8_t	Helper function: updateStatus
main.c	`static uint8_t clearFault(void)`	uint8_t	Helper function: clearFault

Doxygen Comments

```
/**
* @brief Initialise FaultManager.
* @trace DD-COMP-012 SW-REQ-007
*/
/**
* @brief Run FaultManager process cycle.
* @param faultSignals Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-012 SW-REQ-007
*/
/**
* @brief Logfault.
* @trace DD-COMP-012 SW-REQ-007
*/
/**
* @brief Updatestatus.
```

```
 * @trace DD-COMP-012 SW-REQ-007
 */
 /**
 * @brief Clearfault.
 * @trace DD-COMP-012 SW-REQ-007
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	collect fault signals
3	evaluate fault rules
4	update fault state
5	record events
6	notify listeners

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking operations
2	global mutable state
3	silent error drops
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fault' (e.g. faultresult).`

2	State struct: static FaultManager_t faultstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_012_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultManager_t;`
	State struct for FaultManager

`void initFaults(void);`	Prototype: initialise FaultManager
`uint8_t processFaults(void *p_data);`	Prototype: run FaultManager process cycle
`uint8_t logFault(void);`	Prototype: logFault
`uint8_t updateStatus(void);`	Prototype: updateStatus
`uint8_t clearFault(void);`	Prototype: clearFault

`typedef enum {`
`DD_COMP_012_ERR_NONE = 0x00U,`
`DD_COMP_012_ERR_NULL = 0x01U,`
`DD_COMP_012_ERR_INVALID = 0x02U,`
`DD_COMP_012_ERR_TIMEOUT = 0x03U`
`} DD_COMP_012_ERR_t;`
	Error code enum for FaultManager

main.c - What to Add

Item	Purpose

`static FaultManager_t faultstate = {0};`	Module-level zero-initialised state for FaultManager

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: collect fault signals`
`* Step 3: evaluate fault rules`
`* Step 4: update fault state`
`* Step 5: record events`
`* Step 6: notify listeners`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initFaults() once in main() init sequence. Call processFaults() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

15. DD-COMP-013 - SafeStateController

DD-COMP-013	**SafeStateController** | Service | Module: SafetyModule | Pattern: State | SW-REQ-008

`@trace DD-COMP-013 SW-REQ-008`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initSafeState(void)`	void	Initialise all state for SafeStateController
main.c	`static uint8_t processSafeState(void *systemStatus)`	uint8_t - 0 success, non-zero error code	Execute core logic for SafeStateController
main.c	`static uint8_t checkTriggers(void)`	uint8_t	Helper function: checkTriggers
main.c	`static uint8_t applyPolicies(void)`	uint8_t	Helper function: applyPolicies
main.c	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState

Doxygen Comments

```
/**
* @brief Initialise SafeStateController.
* @trace DD-COMP-013 SW-REQ-008
*/
/**
* @brief Run SafeStateController process cycle.
* @param systemStatus Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-013 SW-REQ-008
*/
/**
* @brief Checktriggers.
* @trace DD-COMP-013 SW-REQ-008
*/
/**
* @brief Applypolicies.
```

```
 * @trace DD-COMP-013 SW-REQ-008
 */
 /**
 * @brief Updatestate.
 * @trace DD-COMP-013 SW-REQ-008
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load current system state
2	validate safety triggers
3	evaluate safe-state policies
4	apply state transitions
5	record actions and status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded thresholds
2	silent failures
3	blocking calls in service loop
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'safe' (e.g. saferesult).`
2	`State struct: static SafeStateController_t safestate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_013_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SafeStateController_t;`
	State struct for SafeStateController

`void initSafeState(void);`	Prototype: initialise SafeStateController
`uint8_t processSafeState(void *p_data);`	Prototype: run SafeStateController process cycle
`uint8_t checkTriggers(void);`	Prototype: checkTriggers
`uint8_t applyPolicies(void);`	Prototype: applyPolicies
`uint8_t updateState(void);`	Prototype: updateState

`typedef enum {`
`DD_COMP_013_ERR_NONE = 0x00U,`
`DD_COMP_013_ERR_NULL = 0x01U,`
`DD_COMP_013_ERR_INVALID = 0x02U,`
`DD_COMP_013_ERR_TIMEOUT = 0x03U`
`} DD_COMP_013_ERR_t;`
	Error code enum for SafeStateController

main.c - What to Add

Item	Purpose
`static SafeStateController_t safestate = {0};`	Module-level zero-initialised state for SafeStateController

`/* Process function logic order:`
`* Step 1: load current system state`
`* Step 2: validate safety triggers`
`* Step 3: evaluate safe-state policies`
`* Step 4: apply state transitions`
`* Step 5: record actions and status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initSafeState() once in main() init sequence. Call processSafeState() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

16. DD-COMP-014 - CalibrationStore

DD-COMP-014 **CalibrationStore** | Database | Module: SafetyModule | Pattern: Repository | SW-REQ-008

`@trace DD-COMP-014 SW-REQ-008`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_store(void)`	void	Initialise all state for CalibrationStore
main.c	`static uint8_t process_cal(void *cal_key)`	uint8_t - 0 success, non-zero error code	Execute core logic for CalibrationStore
main.c	`static uint8_t load_record(void)`	uint8_t	Helper function: load record
main.c	`static uint8_t save_record(void)`	uint8_t	Helper function: save record
main.c	`static uint8_t validate_key(void)`	uint8_t	Helper function: validate key

Doxygen Comments

```
/**
* @brief Initialise CalibrationStore.
* @trace DD-COMP-014 SW-REQ-008
*/
/**
* @brief Run CalibrationStore process cycle.
* @param cal_key Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-014 SW-REQ-008
*/
/**
* @brief Load record.
* @trace DD-COMP-014 SW-REQ-008
*/
/**
* @brief Save record.
```

```
 * @trace DD-COMP-014 SW-REQ-008
 */
 /**
 * @brief Validate key.
 * @trace DD-COMP-014 SW-REQ-008
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	open calibration store
2	read or update calibration entry
3	validate calibration data
4	commit changes to storage
5	return status or data

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	silent failures
3	unvalidated writes
4	No dynamic struct allocation
5	Array sizes must be compile-time #define constants
6	No pointer arithmetic on struct members

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'cal' (e.g. calresult).`
2	`State struct: static CalibrationStore_t calstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`

4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_014_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} CalibrationStore_t;`
	State struct for CalibrationStore

`void init_store(void);`	Prototype: initialise CalibrationStore
`uint8_t process_cal(void *p_data);`	Prototype: run CalibrationStore process cycle
`uint8_t load_record(void);`	Prototype: load record
`uint8_t save_record(void);`	Prototype: save record
`uint8_t validate_key(void);`	Prototype: validate key

`typedef enum {`
`DD_COMP_014_ERR_NONE = 0x00U,`
`DD_COMP_014_ERR_NULL = 0x01U,`
`DD_COMP_014_ERR_INVALID = 0x02U,`
`DD_COMP_014_ERR_TIMEOUT = 0x03U`
`} DD_COMP_014_ERR_t;`
	Error code enum for CalibrationStore

main.c - What to Add

Item	Purpose
`static CalibrationStore_t calstate = {0};`	Module-level zero-initialised state for CalibrationStore

`/* Process function logic order:`
`* Step 1: open calibration store`
`* Step 2: read or update calibration entry`
`* Step 3: validate calibration data`
`* Step 4: commit changes to storage`
`* Step 5: return status or data`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call init_store() once in main() init sequence. Call process_cal() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

17. DD-COMP-015 - MISRA Compliance Process

DD-COMP-015	**MISRA Compliance Process** | Module | Module: ComplianceModule | Pattern: N/A | SW-REQ-009

`@trace DD-COMP-015 SW-REQ-009`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void misra_init(void)`	void	Initialise all state for MISRA Compliance Process
`main.c`	`static uint8_t misra_process(void *ruleset_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for MISRA Compliance Process
`main.c`	`static uint8_t load_rules(void)`	uint8_t	Helper function: load rules
`main.c`	`static uint8_t scan_sources(void)`	uint8_t	Helper function: scan sources
`main.c`	`static uint8_t report_findings(void)`	uint8_t	Helper function: report findings

Doxygen Comments

```
/**
* @brief Initialise MISRA Compliance Process.
* @trace DD-COMP-015 SW-REQ-009
*/
/**
* @brief Run MISRA Compliance Process process cycle.
* @param ruleset_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-015 SW-REQ-009
*/
/**
* @brief Load rules.
* @trace DD-COMP-015 SW-REQ-009
*/
/**
```

```
 * @brief Scan sources.
 * @trace DD-COMP-015 SW-REQ-009
 */
 /**
 * @brief Report findings.
 * @trace DD-COMP-015 SW-REQ-009
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load MISRA ruleset
2	collect source files
3	run compliance checks
4	aggregate violations
5	generate summary report

Forbidden - Must NOT Do

#	Forbidden Rule
1	using non-MISRA keywords
2	dynamic memory
3	unchecked casts
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern
8	ISR body must not exceed 10 lines
9	No function calls inside ISR except flag-set and HAL_ClearFlag

Variable Naming Rules

#	Rule

1	All local variables prefixed 'misra' (e.g. misraresult).
2	State struct: static MisraCompliance_t misrastate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_015_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} MisraCompliance_t;`
	State struct for MISRA Compliance Process

`void misra_init(void);`	Prototype: initialise MISRA Compliance Process
`uint8_t misra_process(void *p_data);`	Prototype: run MISRA Compliance Process process cycle
`uint8_t load_rules(void);`	Prototype: load rules
`uint8_t scan_sources(void);`	Prototype: scan sources
`uint8_t report_findings(void);`	Prototype: report findings

`typedef enum {`
`DD_COMP_015_ERR_NONE = 0x00U,`
`DD_COMP_015_ERR_NULL = 0x01U,`
`DD_COMP_015_ERR_INVALID = 0x02U,`
`DD_COMP_015_ERR_TIMEOUT = 0x03U`
`} DD_COMP_015_ERR_t;`
	Error code enum for MISRA Compliance Process

main.c - What to Add

Item	Purpose
`static MisraCompliance_t misrastate = {0};`	Module-level zero-initialised state for MISRA Compliance Process

`/* Process function logic order:`
`* Step 1: load MISRA ruleset`
`* Step 2: collect source files`
`* Step 3: run compliance checks`
`* Step 4: aggregate violations`
`* Step 5: generate summary report`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call misra_init() once in main() init sequence. Call misra_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

18. DD-COMP-016 - Supply Voltage Assumption

DD-COMP-016	**Supply Voltage Assumption** | Class | Module: ControlModule | Pattern: Singleton | SW-REQ-010

`@trace DD-COMP-016 SW-REQ-010`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void SupplyVoltage_Init(void)`	void	Initialise all state for Supply Voltage Assumption
`main.c`	`static uint8_t SupplyVoltage_Update(void *voltage_sample)`	uint8_t - 0 success, non-zero error code	Execute core logic for Supply Voltage Assumption
`main.c`	`static uint8_t ReadVoltage(void)`	uint8_t	Helper function: ReadVoltage
`main.c`	`static uint8_t ValidateRange(void)`	uint8_t	Helper function: ValidateRange
`main.c`	`static uint8_t SetAssumption(void)`	uint8_t	Helper function: SetAssumption

Doxygen Comments

```
/**
* @brief Initialise Supply Voltage Assumption.
* @trace DD-COMP-016 SW-REQ-010
*/
/**
* @brief Run Supply Voltage Assumption process cycle.
* @param voltage_sample Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-016 SW-REQ-010
*/
/**
* @brief Readvoltage.
* @trace DD-COMP-016 SW-REQ-010
*/
/**
* @brief Validaterange.
```

```
 * @trace DD-COMP-016 SW-REQ-010
 */
 /**
 * @brief Setassumption.
 * @trace DD-COMP-016 SW-REQ-010
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configured nominal voltage
2	read current supply voltage
3	check voltage within expected range
4	select assumed voltage level
5	flag any out-of-range condition

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded voltage limits
2	implicit unit conversions
3	silent failures

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'sva' (e.g. svaresult).`
2	`State struct: static SupplyVoltageAssumption_t svastate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`
6	`Error returns: use DD_COMP_016_ERR_t enum values, never raw integers.`

7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SupplyVoltageAssumption_t;`
	State struct for Supply Voltage Assumption

`void SupplyVoltage_Init(void);`	Prototype: initialise Supply Voltage Assumption
`uint8_t SupplyVoltage_Update(void *p_data);`	Prototype: run Supply Voltage Assumption process cycle
`uint8_t ReadVoltage(void);`	Prototype: ReadVoltage
`uint8_t ValidateRange(void);`	Prototype: ValidateRange
`uint8_t SetAssumption(void);`	Prototype: SetAssumption

`typedef enum {`
`DD_COMP_016_ERR_NONE = 0x00U,`
`DD_COMP_016_ERR_NULL = 0x01U,`
`DD_COMP_016_ERR_INVALID = 0x02U,`
`DD_COMP_016_ERR_TIMEOUT = 0x03U`
`} DD_COMP_016_ERR_t;`
	Error code enum for Supply Voltage Assumption

main.c - What to Add

Item	Purpose
`static SupplyVoltageAssumption_t svastate = {0};`	Module-level zero-initialised state for Supply Voltage Assumption

`/* Process function logic order:`
`* Step 1: load configured nominal voltage`
`* Step 2: read current supply voltage`
`* Step 3: check voltage within expected range`
`* Step 4: select assumed voltage level`
`* Step 5: flag any out-of-range condition`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call SupplyVoltage_Init() once in main() init sequence. Call SupplyVoltage_Update() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

19. DD-COMP-017 - ADC Channel Configuration

DD-COMP-017	**ADC Channel Configuration** | Service | Module: HALModule | Pattern: Factory | SW-REQ-011

`@trace DD-COMP-017 SW-REQ-011`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void adcch_init(void)`	void	Initialise all state for ADC Channel Configuration
main.c	`static uint8_t adcch_process(void *cfg_data)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADC Channel Configuration
main.c	`static uint8_t load_cfg(void)`	uint8_t	Helper function: load cfg
main.c	`static uint8_t validate_cfg(void)`	uint8_t	Helper function: validate cfg
main.c	`static uint8_t apply_cfg(void)`	uint8_t	Helper function: apply cfg

Doxygen Comments

```
/**
* @brief Initialise ADC Channel Configuration.
* @trace DD-COMP-017 SW-REQ-011
*/
/**
* @brief Run ADC Channel Configuration process cycle.
* @param cfg_data Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-017 SW-REQ-011
*/
/**
* @brief Load cfg.
* @trace DD-COMP-017 SW-REQ-011
*/
/**
* @brief Validate cfg.
```

```
 * @trace DD-COMP-017 SW-REQ-011
 */
 /**
 * @brief Apply cfg.
 * @trace DD-COMP-017 SW-REQ-011
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration data
2	validate channel parameters
3	configure ADC registers
4	enable selected channel
5	apply reference settings
6	report configuration status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded addresses
2	blocking delays
3	unchecked parameters
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcch' (e.g. adcchresult).`

2	State struct: static AdcChannelCfg_t adcchstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_017_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcChannelCfg_t;`
	State struct for ADC Channel Configuration

`void adcch_init(void);`	Prototype: initialise ADC Channel Configuration
`uint8_t adcch_process(void *p_data);`	Prototype: run ADC Channel Configuration process cycle
`uint8_t load_cfg(void);`	Prototype: load cfg
`uint8_t validate_cfg(void);`	Prototype: validate cfg
`uint8_t apply_cfg(void);`	Prototype: apply cfg

`typedef enum {`
`DD_COMP_017_ERR_NONE = 0x00U,`
`DD_COMP_017_ERR_NULL = 0x01U,`
`DD_COMP_017_ERR_INVALID = 0x02U,`
`DD_COMP_017_ERR_TIMEOUT = 0x03U`
`} DD_COMP_017_ERR_t;`
	Error code enum for ADC Channel Configuration

main.c - What to Add

Item	Purpose

`static AdcChannelCfg_t adcchstate = {0};`	Module-level zero-initialised state for ADC Channel Configuration

`/* Process function logic order:`
`* Step 1: load configuration data`
`* Step 2: validate channel parameters`
`* Step 3: configure ADC registers`
`* Step 4: enable selected channel`
`* Step 5: apply reference settings`
`* Step 6: report configuration status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call adcch_init() once in main() init sequence. Call adcch_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

20. DD-COMP-018 - ADC Read Service

DD-COMP-018	**ADC Read Service** | Service | Module: HALModule | Pattern: N/A | SW-REQ-012

`@trace DD-COMP-018 SW-REQ-012`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void adcReadInit(void)`	void	Initialise all state for ADC Read Service
main.c	`static uint8_t adcReadProcess(void *adc_channel)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADC Read Service
main.c	`static uint8_t adcStartConversion(void)`	uint8_t	Helper function: adcStartConversion
main.c	`static uint8_t adcCheckReady(void)`	uint8_t	Helper function: adcCheckReady
main.c	`static uint8_t adcFetchValue(void)`	uint8_t	Helper function: adcFetchValue

Doxygen Comments

```
/**
* @brief Initialise ADC Read Service.
* @trace DD-COMP-018 SW-REQ-012
*/
/**
* @brief Run ADC Read Service process cycle.
* @param adc_channel Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-018 SW-REQ-012
*/
/**
* @brief Adcstartconversion.
* @trace DD-COMP-018 SW-REQ-012
*/
/**
* @brief Adccheckready.
```

```
 * @trace DD-COMP-018 SW-REQ-012
 */
 /**
 * @brief Adcfetchvalue.
 * @trace DD-COMP-018 SW-REQ-012
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize ADC hardware and channels
2	start a conversion cycle
3	poll or wait for conversion complete
4	read raw ADC value
5	apply scaling or calibration
6	store or publish the result
7	handle errors or timeouts

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking too long in ISR
2	hardcoding channel numbers
3	ignoring error flags
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'adc' (e.g. adcresult).
2	State struct: static AdcReadService_t adcstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_018_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcReadService_t;`
	State struct for ADC Read Service

`void adcReadInit(void);`	Prototype: initialise ADC Read Service
`uint8_t adcReadProcess(void *p_data);`	Prototype: run ADC Read Service process cycle
`uint8_t adcStartConversion(void);`	Prototype: adcStartConversion
`uint8_t adcCheckReady(void);`	Prototype: adcCheckReady
`uint8_t adcFetchValue(void);`	Prototype: adcFetchValue

`typedef enum {`
`DD_COMP_018_ERR_NONE = 0x00U,`
`DD_COMP_018_ERR_NULL = 0x01U,`
`DD_COMP_018_ERR_INVALID = 0x02U,`
`DD_COMP_018_ERR_TIMEOUT = 0x03U`
`} DD_COMP_018_ERR_t;`
	Error code enum for ADC Read Service

main.c - What to Add

Item	Purpose

`static AdcReadService_t adcstate = {0};`	Module-level zero-initialised state for ADC Read Service

`/* Process function logic order:`
`* Step 1: initialize ADC hardware and channels`
`* Step 2: start a conversion cycle`
`* Step 3: poll or wait for conversion complete`
`* Step 4: read raw ADC value`
`* Step 5: apply scaling or calibration`
`* Step 6: store or publish the result`
`* Step 7: handle errors or timeouts`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call adcReadInit() once in main() init sequence. Call adcReadProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

21. DD-COMP-019 - Position Scaling Algorithm

DD-COMP-019	**Position Scaling Algorithm** | Algorithm | Module: SignalProcessingModule | Pattern: N/A | SW-REQ-013

`@trace DD-COMP-019 SW-REQ-013`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void posScaleInit(void)`	void	Initialise all state for Position Scaling Algorithm
main.c	`static uint8_t posScaleRun(void *raw_position)`	uint8_t - 0 success, non-zero error code	Execute core logic for Position Scaling Algorithm
main.c	`static uint8_t calcScale(void)`	uint8_t	Helper function: calcScale
main.c	`static uint8_t applyScale(void)`	uint8_t	Helper function: applyScale
main.c	`static uint8_t clampValue(void)`	uint8_t	Helper function: clampValue

Doxygen Comments

```
/**
* @brief Initialise Position Scaling Algorithm.
* @trace DD-COMP-019 SW-REQ-013
*/
/**
* @brief Run Position Scaling Algorithm process cycle.
* @param raw_position Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-019 SW-REQ-013
*/
/**
* @brief Calcscale.
* @trace DD-COMP-019 SW-REQ-013
*/
/**
```

```
 * @brief Applyscale.
 * @trace DD-COMP-019 SW-REQ-013
 */
 /**
 * @brief Clampvalue.
 * @trace DD-COMP-019 SW-REQ-013
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load scaling parameters
2	validate input positions
3	compute scale factors
4	apply scaling to positions
5	clamp to limits
6	store outputs

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded limits
2	division by zero
3	stateful globals
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ps' (e.g. psresult).`

2	State struct: static PositionScaling_t psstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_019_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} PositionScaling_t;`
	State struct for Position Scaling Algorithm

`void posScaleInit(void);`	Prototype: initialise Position Scaling Algorithm
`uint8_t posScaleRun(void *p_data);`	Prototype: run Position Scaling Algorithm process cycle
`uint8_t calcScale(void);`	Prototype: calcScale
`uint8_t applyScale(void);`	Prototype: applyScale
`uint8_t clampValue(void);`	Prototype: clampValue

`typedef enum {`
`DD_COMP_019_ERR_NONE = 0x00U,`
`DD_COMP_019_ERR_NULL = 0x01U,`
`DD_COMP_019_ERR_INVALID = 0x02U,`
`DD_COMP_019_ERR_TIMEOUT = 0x03U`
`} DD_COMP_019_ERR_t;`
	Error code enum for Position Scaling Algorithm

main.c - What to Add

Item	Purpose

`static PositionScaling_t psstate = {0};`	Module-level zero-initialised state for Position Scaling Algorithm

`/* Process function logic order:`
`* Step 1: load scaling parameters`
`* Step 2: validate input positions`
`* Step 3: compute scale factors`
`* Step 4: apply scaling to positions`
`* Step 5: clamp to limits`
`* Step 6: store outputs`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call posScaleInit() once in main() init sequence. Call posScaleRun() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

22. DD-COMP-020 - Control Output Calculator

DD-COMP-020	**Control Output Calculator** | Class | Module: ControlModule | Pattern: Strategy | SW-REQ-014

`@trace DD-COMP-020 SW-REQ-014`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void ctrlOutInit(void)`	void	Initialise all state for Control Output Calculator
`main.c`	`static uint8_t ctrlOutCompute(void *controlInput)`	uint8_t - 0 success, non-zero error code	Execute core logic for Control Output Calculator
`main.c`	`static uint8_t clampOutput(void)`	uint8_t	Helper function: clampOutput
`main.c`	`static uint8_t applyLimits(void)`	uint8_t	Helper function: applyLimits
`main.c`	`static uint8_t blendModes(void)`	uint8_t	Helper function: blendModes

Doxygen Comments

```
/**
* @brief Initialise Control Output Calculator.
* @trace DD-COMP-020 SW-REQ-014
*/
/**
* @brief Run Control Output Calculator process cycle.
* @param controlInput Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-020 SW-REQ-014
*/
/**
* @brief Clampoutput.
* @trace DD-COMP-020 SW-REQ-014
*/
/**
* @brief Applylimits.
```

```
* @trace DD-COMP-020 SW-REQ-014
*/
/**
* @brief Blendmodes.
* @trace DD-COMP-020 SW-REQ-014
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	Load current control inputs
2	Validate and sanitize inputs
3	Compute raw output value
4	Apply limits and constraints
5	Blend mode adjustments if needed
6	Store and return final output

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded limits
2	side effects in helpers
3	silent overflow

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ctrlOut' (e.g. ctrlOutresult).`
2	`State struct: static ControlOutputCalculator_t ctrlOutstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_020_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ControlOutputCalculator_t;`
	State struct for Control Output Calculator

`void ctrlOutInit(void);`	Prototype: initialise Control Output Calculator
`uint8_t ctrlOutCompute(void *p_data);`	Prototype: run Control Output Calculator process cycle
`uint8_t clampOutput(void);`	Prototype: clampOutput
`uint8_t applyLimits(void);`	Prototype: applyLimits
`uint8_t blendModes(void);`	Prototype: blendModes

`typedef enum {`
`DD_COMP_020_ERR_NONE = 0x00U,`
`DD_COMP_020_ERR_NULL = 0x01U,`
`DD_COMP_020_ERR_INVALID = 0x02U,`
`DD_COMP_020_ERR_TIMEOUT = 0x03U`
`} DD_COMP_020_ERR_t;`
	Error code enum for Control Output Calculator

main.c - What to Add

Item	Purpose
`static ControlOutputCalculator_t ctrlOutstate = {0};`	Module-level zero-initialised state for Control Output Calculator

`/* Process function logic order:`
`* Step 1: Load current control inputs`
`* Step 2: Validate and sanitize inputs`
`* Step 3: Compute raw output value`
`* Step 4: Apply limits and constraints`
`* Step 5: Blend mode adjustments if needed`
`* Step 6: Store and return final output`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Strategy. Call ctrlOutInit() once in main() init sequence. Call ctrlOutCompute() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

23. DD-COMP-021 - Control Loop Scheduler

DD-COMP-021	**Control Loop Scheduler** | Service | Module: SchedulerModule | Pattern: Observer | SW-REQ-015

`@trace DD-COMP-021 SW-REQ-015`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void sched_init(void)`	void	Initialise all state for Control Loop Scheduler
main.c	`static uint8_t sched_run(void *time_tick)`	uint8_t - 0 success, non-zero error code	Execute core logic for Control Loop Scheduler
main.c	`static uint8_t tick_update(void)`	uint8_t	Helper function: tick update
main.c	`static uint8_t next_task(void)`	uint8_t	Helper function: next task
main.c	`static uint8_t dispatch_task(void)`	uint8_t	Helper function: dispatch task

Doxygen Comments

```
/**
* @brief Initialise Control Loop Scheduler.
* @trace DD-COMP-021 SW-REQ-015
*/
/**
* @brief Run Control Loop Scheduler process cycle.
* @param time_tick Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-021 SW-REQ-015
*/
/**
* @brief Tick update.
* @trace DD-COMP-021 SW-REQ-015
*/
/**
```

```
 * @brief Next task.
 * @trace DD-COMP-021 SW-REQ-015
 */
 /**
 * @brief Dispatch task.
 * @trace DD-COMP-021 SW-REQ-015
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	update time base
3	check due control loops
4	select next loop
5	execute loop handler
6	record execution status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	shared state without locks
3	unbounded loops
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'sch' (e.g. schresult).`

2	State struct: static ControlScheduler_t schstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_021_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ControlScheduler_t;`
	State struct for Control Loop Scheduler

`void sched_init(void);`	Prototype: initialise Control Loop Scheduler
`uint8_t sched_run(void *p_data);`	Prototype: run Control Loop Scheduler process cycle
`uint8_t tick_update(void);`	Prototype: tick update
`uint8_t next_task(void);`	Prototype: next task
`uint8_t dispatch_task(void);`	Prototype: dispatch task

`typedef enum {`
`DD_COMP_021_ERR_NONE = 0x00U,`
`DD_COMP_021_ERR_NULL = 0x01U,`
`DD_COMP_021_ERR_INVALID = 0x02U,`
`DD_COMP_021_ERR_TIMEOUT = 0x03U`
`} DD_COMP_021_ERR_t;`
	Error code enum for Control Loop Scheduler

main.c - What to Add

Item	Purpose

`static ControlScheduler_t schstate = {0};`	Module-level zero-initialised state for Control Loop Scheduler

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: update time base`
`* Step 3: check due control loops`
`* Step 4: select next loop`
`* Step 5: execute loop handler`
`* Step 6: record execution status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call sched_init() once in main() init sequence. Call sched_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

24. DD-COMP-022 - Feedback Fault Monitor

DD-COMP-022	**Feedback Fault Monitor** | Class | Module: SafetyModule | Pattern: State | SW-REQ-016

`@trace DD-COMP-022 SW-REQ-016`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void ffm_init(void)`	void	Initialise all state for Feedback Fault Monitor
main.c	`static uint8_t ffm_process(void *feedback_value)`	uint8_t - 0 success, non-zero error code	Execute core logic for Feedback Fault Monitor
main.c	`static uint8_t check_limits(void)`	uint8_t	Helper function: check limits
main.c	`static uint8_t update_fault_state(void)`	uint8_t	Helper function: update fault state
main.c	`static uint8_t log_event(void)`	uint8_t	Helper function: log event

Doxygen Comments

```
/**
* @brief Initialise Feedback Fault Monitor.
* @trace DD-COMP-022 SW-REQ-016
*/
/**
* @brief Run Feedback Fault Monitor process cycle.
* @param feedback_value Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-022 SW-REQ-016
*/
/**
* @brief Check limits.
* @trace DD-COMP-022 SW-REQ-016
*/
/**
* @brief Update fault state.
```

```
 * @trace DD-COMP-022 SW-REQ-016
 */
 /**
 * @brief Log event.
 * @trace DD-COMP-022 SW-REQ-016
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Load configuration thresholds
2	Read current feedback signals
3	Compare feedback to expected range
4	Update fault counters and latches
5	Generate fault status output
6	Record diagnostic event if needed

Forbidden - Must NOT Do

#	Forbidden Rule
1	hard-coded thresholds
2	blocking I/O in process
3	silent overflow in counters

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ffm' (e.g. ffmresult).`
2	`State struct: static FeedbackFaultMonitor_t ffmstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_022_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FeedbackFaultMonitor_t;`
	State struct for Feedback Fault Monitor

`void ffm_init(void);`	Prototype: initialise Feedback Fault Monitor
`uint8_t ffm_process(void *p_data);`	Prototype: run Feedback Fault Monitor process cycle
`uint8_t check_limits(void);`	Prototype: check limits
`uint8_t update_fault_state(void);`	Prototype: update fault state
`uint8_t log_event(void);`	Prototype: log event

`typedef enum {`
`DD_COMP_022_ERR_NONE = 0x00U,`
`DD_COMP_022_ERR_NULL = 0x01U,`
`DD_COMP_022_ERR_INVALID = 0x02U,`
`DD_COMP_022_ERR_TIMEOUT = 0x03U`
`} DD_COMP_022_ERR_t;`
	Error code enum for Feedback Fault Monitor

main.c - What to Add

Item	Purpose
`static FeedbackFaultMonitor_t ffmstate = {0};`	Module-level zero-initialised state for Feedback Fault Monitor

`/* Process function logic order:`
`* Step 1: Load configuration thresholds`
`* Step 2: Read current feedback signals`
`* Step 3: Compare feedback to expected range`
`* Step 4: Update fault counters and latches`
`* Step 5: Generate fault status output`
`* Step 6: Record diagnostic event if needed`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call ffm_init() once in main() init sequence. Call ffm_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

25. DD-COMP-023 - Fault Reporter

DD-COMP-023	**Fault Reporter** | Service | Module: SafetyModule | Pattern: N/A | SW-REQ-016

`@trace DD-COMP-023 SW-REQ-016`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_reporter(void)`	void	Initialise all state for Fault Reporter
main.c	`static uint8_t report_faults(void *fault_events)`	uint8_t - 0 success, non-zero error code	Execute core logic for Fault Reporter
main.c	`static uint8_t collect_faults(void)`	uint8_t	Helper function: collect faults
main.c	`static uint8_t format_report(void)`	uint8_t	Helper function: format report
main.c	`static uint8_t send_report(void)`	uint8_t	Helper function: send report

Doxygen Comments

```
/**
* @brief Initialise Fault Reporter.
* @trace DD-COMP-023 SW-REQ-016
*/
/**
* @brief Run Fault Reporter process cycle.
* @param fault_events Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-023 SW-REQ-016
*/
/**
* @brief Collect faults.
* @trace DD-COMP-023 SW-REQ-016
*/
/**
* @brief Format report.
```

```
 * @trace DD-COMP-023 SW-REQ-016
 */
 /**
 * @brief Send report.
 * @trace DD-COMP-023 SW-REQ-016
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	collect fault data
3	deduplicate and prioritize faults
4	format report payload
5	dispatch report to targets
6	log delivery status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking calls in main path
2	leaking sensitive data
3	duplicate reporting
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fr' (e.g. frresult).`

2	State struct: static FaultReporter_t frstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_023_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultReporter_t;`
	State struct for Fault Reporter

`void init_reporter(void);`	Prototype: initialise Fault Reporter
`uint8_t report_faults(void *p_data);`	Prototype: run Fault Reporter process cycle
`uint8_t collect_faults(void);`	Prototype: collect faults
`uint8_t format_report(void);`	Prototype: format report
`uint8_t send_report(void);`	Prototype: send report

`typedef enum {`
`DD_COMP_023_ERR_NONE = 0x00U,`
`DD_COMP_023_ERR_NULL = 0x01U,`
`DD_COMP_023_ERR_INVALID = 0x02U,`
`DD_COMP_023_ERR_TIMEOUT = 0x03U`
`} DD_COMP_023_ERR_t;`
	Error code enum for Fault Reporter

main.c - What to Add

Item	Purpose

`static FaultReporter_t frstate = {0};`	Module-level zero-initialised state for Fault Reporter

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: collect fault data`
`* Step 3: deduplicate and prioritize faults`
`* Step 4: format report payload`
`* Step 5: dispatch report to targets`
`* Step 6: log delivery status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call init_reporter() once in main() init sequence. Call report_faults() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

26. DD-COMP-024 - Actuator Drive Output

DD-COMP-024	**Actuator Drive Output** | Service | Module: HALModule | Pattern: N/A | SW-REQ-014

`@trace DD-COMP-024 SW-REQ-014`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void ActDriveOut_Init(void)`	void	Initialise all state for Actuator Drive Output
main.c	`static uint8_t ActDriveOut_Process(void *desired_cmd)`	uint8_t - 0 success, non-zero error code	Execute core logic for Actuator Drive Output
main.c	`static uint8_t ApplyLimits(void)`	uint8_t	Helper function: ApplyLimits
main.c	`static uint8_t MapCommand(void)`	uint8_t	Helper function: MapCommand
main.c	`static uint8_t UpdateStatus(void)`	uint8_t	Helper function: UpdateStatus

Doxygen Comments

```
/**
* @brief Initialise Actuator Drive Output.
* @trace DD-COMP-024 SW-REQ-014
*/
/**
* @brief Run Actuator Drive Output process cycle.
* @param desired_cmd Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-024 SW-REQ-014
*/
/**
* @brief Applylimits.
* @trace DD-COMP-024 SW-REQ-014
*/
/**
* @brief Mapcommand.
```

```
 * @trace DD-COMP-024 SW-REQ-014
 */
 /**
 * @brief Updatestatus.
 * @trace DD-COMP-024 SW-REQ-014
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Load configuration and limits
2	Read desired actuator command
3	Validate command range
4	Map command to drive output units
5	Apply safety and limit checks
6	Write drive output signal
7	Update output status and faults

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	hardcoded limits
3	silent saturation
4	unchecked null pointers
5	No malloc/calloc/free anywhere in this component
6	No recursive function calls - use iterative logic only
7	No processing inside ISR - set flag and return only
8	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	All local variables prefixed 'act' (e.g. actresult).
2	State struct: static ActuatorDriveOut_t actstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_024_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorDriveOut_t;`
	State struct for Actuator Drive Output

`void ActDriveOut_Init(void);`	Prototype: initialise Actuator Drive Output
`uint8_t ActDriveOut_Process(void *p_data);`	Prototype: run Actuator Drive Output process cycle
`uint8_t ApplyLimits(void);`	Prototype: ApplyLimits
`uint8_t MapCommand(void);`	Prototype: MapCommand
`uint8_t UpdateStatus(void);`	Prototype: UpdateStatus

`typedef enum {`
`DD_COMP_024_ERR_NONE = 0x00U,`
`DD_COMP_024_ERR_NULL = 0x01U,`
`DD_COMP_024_ERR_INVALID = 0x02U,`
`DD_COMP_024_ERR_TIMEOUT = 0x03U`
`} DD_COMP_024_ERR_t;`
	Error code enum for Actuator Drive Output

main.c - What to Add

Item	Purpose
`static ActuatorDriveOut_t actstate = {0};`	Module-level zero-initialised state for Actuator Drive Output

`/* Process function logic order:`
`* Step 1: Load configuration and limits`
`* Step 2: Read desired actuator command`
`* Step 3: Validate command range`
`* Step 4: Map command to drive output units`
`* Step 5: Apply safety and limit checks`
`* Step 6: Write drive output signal`
`* Step 7: Update output status and faults`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call ActDriveOut_Init() once in main() init sequence. Call ActDriveOut_Process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

27. DD-COMP-025 - ThresholdConfigRepository

DD-COMP-025	**ThresholdConfigRepository** | Class | Module: ConfigModule | Pattern: Repository | SW-REQ-017

`@trace DD-COMP-025 SW-REQ-017`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void initRepo(void)`	void	Initialise all state for ThresholdConfigRepository
`main.c`	`static uint8_t loadThresholds(void *configPath)`	uint8_t - 0 success, non-zero error code	Execute core logic for ThresholdConfigRepository
`main.c`	`static uint8_t readConfig(void)`	uint8_t	Helper function: readConfig
`main.c`	`static uint8_t validateThresholds(void)`	uint8_t	Helper function: validateThresholds
`main.c`	`static uint8_t cacheThresholds(void)`	uint8_t	Helper function: cacheThresholds

Doxygen Comments

```
/**
* @brief Initialise ThresholdConfigRepository.
* @trace DD-COMP-025 SW-REQ-017
*/
/**
* @brief Run ThresholdConfigRepository process cycle.
* @param configPath Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-025 SW-REQ-017
*/
/**
* @brief Readconfig.
* @trace DD-COMP-025 SW-REQ-017
*/
/**
```

```
* @brief Validatethresholds.
* @trace DD-COMP-025 SW-REQ-017
*/
/**
* @brief Cachethresholds.
* @trace DD-COMP-025 SW-REQ-017
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	locate config source
2	read raw threshold data
3	parse entries into structures
4	validate ranges and types
5	cache thresholds for access
6	return load status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	silent parse failures
3	mutable global state

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'tcr' (e.g. tcrresult).`
2	`State struct: static ThresholdConfigRepository_t tcrstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_025_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ThresholdConfigRepository_t;`
	State struct for ThresholdConfigRepository

`void initRepo(void);`	Prototype: initialise ThresholdConfigRepository
`uint8_t loadThresholds(void *p_data);`	Prototype: run ThresholdConfigRepository process cycle
`uint8_t readConfig(void);`	Prototype: readConfig
`uint8_t validateThresholds(void);`	Prototype: validateThresholds
`uint8_t cacheThresholds(void);`	Prototype: cacheThresholds

`typedef enum {`
`DD_COMP_025_ERR_NONE = 0x00U,`
`DD_COMP_025_ERR_NULL = 0x01U,`
`DD_COMP_025_ERR_INVALID = 0x02U,`
`DD_COMP_025_ERR_TIMEOUT = 0x03U`
`} DD_COMP_025_ERR_t;`
	Error code enum for ThresholdConfigRepository

main.c - What to Add

Item	Purpose
`static ThresholdConfigRepository_t tcrstate = {0};`	Module-level zero-initialised state for ThresholdConfigRepository

`/* Process function logic order:`
`* Step 1: locate config source`
`* Step 2: read raw threshold data`
`* Step 3: parse entries into structures`
`* Step 4: validate ranges and types`
`* Step 5: cache thresholds for access`
`* Step 6: return load status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call initRepo() once in main() init sequence. Call loadThresholds() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

28. DD-COMP-026 - ThresholdConfigService

DD-COMP-026	**ThresholdConfigService** | Service | Module: ConfigModule | Pattern: Singleton | SW-REQ-017

`@trace DD-COMP-026 SW-REQ-017`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_threshold_cfg(void)`	void	Initialise all state for ThresholdConfigService
main.c	`static uint8_t process_threshold_cfg(void *config_request)`	uint8_t - 0 success, non-zero error code	Execute core logic for ThresholdConfigService
main.c	`static uint8_t load_defaults(void)`	uint8_t	Helper function: load defaults
main.c	`static uint8_t validate_limits(void)`	uint8_t	Helper function: validate limits
main.c	`static uint8_t apply_updates(void)`	uint8_t	Helper function: apply updates

Doxygen Comments

```
/**
* @brief Initialise ThresholdConfigService.
* @trace DD-COMP-026 SW-REQ-017
*/
/**
* @brief Run ThresholdConfigService process cycle.
* @param config_request Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-026 SW-REQ-017
*/
/**
* @brief Load defaults.
* @trace DD-COMP-026 SW-REQ-017
*/
/**
* @brief Validate limits.
```

```
 * @trace DD-COMP-026 SW-REQ-017
 */
 /**
 * @brief Apply updates.
 * @trace DD-COMP-026 SW-REQ-017
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load current threshold settings
2	merge incoming configuration
3	validate threshold ranges
4	persist updated settings
5	emit configuration status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoding thresholds
2	silently ignoring invalid values
3	unbounded retries
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'thr' (e.g. thrresult).`
2	`State struct: static ThresholdConfigService_t thrstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_026_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ThresholdConfigService_t;`
	State struct for ThresholdConfigService

`void init_threshold_cfg(void);`	Prototype: initialise ThresholdConfigService
`uint8_t process_threshold_cfg(void *p_data);`	Prototype: run ThresholdConfigService process cycle
`uint8_t load_defaults(void);`	Prototype: load defaults
`uint8_t validate_limits(void);`	Prototype: validate limits
`uint8_t apply_updates(void);`	Prototype: apply updates

`typedef enum {`
`DD_COMP_026_ERR_NONE = 0x00U,`
`DD_COMP_026_ERR_NULL = 0x01U,`
`DD_COMP_026_ERR_INVALID = 0x02U,`
`DD_COMP_026_ERR_TIMEOUT = 0x03U`
`} DD_COMP_026_ERR_t;`
	Error code enum for ThresholdConfigService

main.c - What to Add

Item	Purpose
`static ThresholdConfigService_t thrstate = {0};`	Module-level zero-initialised state for ThresholdConfigService

`/* Process function logic order:`
`* Step 1: load current threshold settings`
`* Step 2: merge incoming configuration`
`* Step 3: validate threshold ranges`
`* Step 4: persist updated settings`
`* Step 5: emit configuration status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call init_threshold_cfg() once in main() init sequence. Call process_threshold_cfg() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

29. DD-COMP-027 - ShippingPositionStopLogic

DD-COMP-027	**ShippingPositionStopLogic** | Algorithm | Module: ControlModule | Pattern: Observer | SW-REQ-018

`@trace DD-COMP-027 SW-REQ-018`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initStopLogic(void)`	void	Initialise all state for ShippingPositionStopLogic
main.c	`static uint8_t processStopLogic(void *currentPos)`	uint8_t - 0 success, non-zero error code	Execute core logic for ShippingPositionStopLogic
main.c	`static uint8_t calcStopDistance(void)`	uint8_t	Helper function: calcStopDistance
main.c	`static uint8_t updateStopState(void)`	uint8_t	Helper function: updateStopState
main.c	`static uint8_t isStopCondition(void)`	uint8_t	Helper function: isStopCondition

Doxygen Comments

```
/**
* @brief Initialise ShippingPositionStopLogic.
* @trace DD-COMP-027 SW-REQ-018
*/
/**
* @brief Run ShippingPositionStopLogic process cycle.
* @param currentPos Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-027 SW-REQ-018
*/
/**
* @brief Calcstopdistance.
* @trace DD-COMP-027 SW-REQ-018
*/
/**
```

```
 * @brief Updatestopstate.
 * @trace DD-COMP-027 SW-REQ-018
 */
 /**
 * @brief Isstopcondition.
 * @trace DD-COMP-027 SW-REQ-018
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load current position and target
2	evaluate stop conditions
3	compute remaining distance
4	update stop state flags
5	emit stop command if needed
6	store updated state

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded thresholds
2	mutable global state
3	blocking delays
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'stop' (e.g. stopresult).`

2	State struct: static ShippingPositionStopLogic_t stopstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_027_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ShippingPositionStopLogic_t;`
	State struct for ShippingPositionStopLogic

`void initStopLogic(void);`	Prototype: initialise ShippingPositionStopLogic
`uint8_t processStopLogic(void *p_data);`	Prototype: run ShippingPositionStopLogic process cycle
`uint8_t calcStopDistance(void);`	Prototype: calcStopDistance
`uint8_t updateStopState(void);`	Prototype: updateStopState
`uint8_t isStopCondition(void);`	Prototype: isStopCondition

`typedef enum {`
`DD_COMP_027_ERR_NONE = 0x00U,`
`DD_COMP_027_ERR_NULL = 0x01U,`
`DD_COMP_027_ERR_INVALID = 0x02U,`
`DD_COMP_027_ERR_TIMEOUT = 0x03U`
`} DD_COMP_027_ERR_t;`
	Error code enum for ShippingPositionStopLogic

main.c - What to Add

Item	Purpose

`static ShippingPositionStopLogic_t stopstate = {0};`	Module-level zero-initialised state for ShippingPositionStopLogic

`/* Process function logic order:`
`* Step 1: load current position and target`
`* Step 2: evaluate stop conditions`
`* Step 3: compute remaining distance`
`* Step 4: update stop state flags`
`* Step 5: emit stop command if needed`
`* Step 6: store updated state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initStopLogic() once in main() init sequence. Call processStopLogic() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

30. DD-COMP-028 - UartCommandParser

DD-COMP-028	**UartCommandParser** | Service | Module: UartModule | Pattern: Observer | SW-REQ-019

`@trace DD-COMP-028 SW-REQ-019`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void Ucp_Init(void)`	void	Initialise all state for UartCommandParser
main.c	`static uint8_t Ucp_Process(void *uart_rx_byte)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartCommandParser
main.c	`static uint8_t Ucp_ReadByte(void)`	uint8_t	Helper function: Ucp ReadByte
main.c	`static uint8_t Ucp_ParseCmd(void)`	uint8_t	Helper function: Ucp ParseCmd
main.c	`static uint8_t Ucp_Reset(void)`	uint8_t	Helper function: Ucp Reset

Doxygen Comments

```
/**
* @brief Initialise UartCommandParser.
* @trace DD-COMP-028 SW-REQ-019
*/
/**
* @brief Run UartCommandParser process cycle.
* @param uart_rx_byte Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-028 SW-REQ-019
*/
/**
* @brief Ucp readbyte.
* @trace DD-COMP-028 SW-REQ-019
*/
/**
* @brief Ucp parsecmd.
```

```
 * @trace DD-COMP-028 SW-REQ-019
 */
 /**
 * @brief Ucp reset.
 * @trace DD-COMP-028 SW-REQ-019
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read incoming byte
2	append to command buffer
3	detect command terminator
4	validate command format
5	decode command fields
6	signal command ready
7	reset buffer on completion

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UART reads
2	buffer overflow
3	heap allocations
4	tight hardware coupling
5	No malloc/calloc/free anywhere in this component
6	No recursive function calls - use iterative logic only
7	No processing inside ISR - set flag and return only
8	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	All local variables prefixed 'ucp' (e.g. ucpresult).
2	State struct: static UartCommandParser_t ucpstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_028_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartCommandParser_t;`
	State struct for UartCommandParser

`void Ucp_Init(void);`	Prototype: initialise UartCommandParser
`uint8_t Ucp_Process(void *p_data);`	Prototype: run UartCommandParser process cycle
`uint8_t Ucp_ReadByte(void);`	Prototype: Ucp ReadByte
`uint8_t Ucp_ParseCmd(void);`	Prototype: Ucp ParseCmd
`uint8_t Ucp_Reset(void);`	Prototype: Ucp Reset

`typedef enum {`
`DD_COMP_028_ERR_NONE = 0x00U,`
`DD_COMP_028_ERR_NULL = 0x01U,`
`DD_COMP_028_ERR_INVALID = 0x02U,`
`DD_COMP_028_ERR_TIMEOUT = 0x03U`
`} DD_COMP_028_ERR_t;`
	Error code enum for UartCommandParser

main.c - What to Add

Item	Purpose
`static UartCommandParser_t ucpstate = {0};`	Module-level zero-initialised state for UartCommandParser

`/* Process function logic order:`
`* Step 1: read incoming byte`
`* Step 2: append to command buffer`
`* Step 3: detect command terminator`
`* Step 4: validate command format`
`* Step 5: decode command fields`
`* Step 6: signal command ready`
`* Step 7: reset buffer on completion`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call Ucp_Init() once in main() init sequence. Call Ucp_Process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

31. DD-COMP-029 - UartCommandParser_OFF

DD-COMP-029	**UartCommandParser_OFF** | Service | Module: UartModule | Pattern: Observer | SW-REQ-020

`@trace DD-COMP-029 SW-REQ-020`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void uart_parser_off_init(void)`	void	Initialise all state for UartCommandParser_OFF
main.c	`static uint8_t uart_parser_off_process(void *uart_rx_bytes)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartCommandParser_OFF
main.c	`static uint8_t scan_off_cmd(void)`	uint8_t	Helper function: scan off cmd
main.c	`static uint8_t validate_off_crc(void)`	uint8_t	Helper function: validate off crc
main.c	`static uint8_t reset_off_state(void)`	uint8_t	Helper function: reset off state

Doxygen Comments

```
/**
* @brief Initialise UartCommandParser_OFF.
* @trace DD-COMP-029 SW-REQ-020
*/
/**
* @brief Run UartCommandParser_OFF process cycle.
* @param uart_rx_bytes Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-029 SW-REQ-020
*/
/**
* @brief Scan off cmd.
* @trace DD-COMP-029 SW-REQ-020
*/
/**
* @brief Validate off crc.
```

```
 * @trace DD-COMP-029 SW-REQ-020
 */
 /**
 * @brief Reset off state.
 * @trace DD-COMP-029 SW-REQ-020
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read incoming UART bytes
2	buffer bytes until end marker
3	detect OFF command pattern
4	validate checksum if present
5	emit OFF command event
6	clear buffer and state

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking reads
2	buffer overflow
3	silent parse errors
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uoff' (e.g. uoffresult).`

2	State struct: static UartCommandParserOff_t uoffstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_029_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartCommandParserOff_t;`
	State struct for UartCommandParser_OFF

`void uart_parser_off_init(void);`	Prototype: initialise UartCommandParser_OFF
`uint8_t uart_parser_off_process(void *p_data);`	Prototype: run UartCommandParser_OFF process cycle
`uint8_t scan_off_cmd(void);`	Prototype: scan off cmd
`uint8_t validate_off_crc(void);`	Prototype: validate off crc
`uint8_t reset_off_state(void);`	Prototype: reset off state

`typedef enum {`
`DD_COMP_029_ERR_NONE = 0x00U,`
`DD_COMP_029_ERR_NULL = 0x01U,`
`DD_COMP_029_ERR_INVALID = 0x02U,`
`DD_COMP_029_ERR_TIMEOUT = 0x03U`
`} DD_COMP_029_ERR_t;`
	Error code enum for UartCommandParser_OFF

main.c - What to Add

Item	Purpose

`static UartCommandParserOff_t uoffstate = {0};`	Module-level zero-initialised state for UartCommandParser_OFF

`/* Process function logic order:`
`* Step 1: read incoming UART bytes`
`* Step 2: buffer bytes until end marker`
`* Step 3: detect OFF command pattern`
`* Step 4: validate checksum if present`
`* Step 5: emit OFF command event`
`* Step 6: clear buffer and state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call uart_parser_off_init() once in main() init sequence. Call uart_parser_off_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

32. DD-COMP-030 - ActuatorStateMachine

DD-COMP-030	**ActuatorStateMachine** | Class | Module: ControlModule | Pattern: State | SW-REQ-019

`@trace DD-COMP-030 SW-REQ-019`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_state(void)`	void	Initialise all state for ActuatorStateMachine
main.c	`static uint8_t run_state(void *event_flags)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorStateMachine
main.c	`static uint8_t eval_event(void)`	uint8_t	Helper function: eval event
main.c	`static uint8_t update_state(void)`	uint8_t	Helper function: update state
main.c	`static uint8_t apply_output(void)`	uint8_t	Helper function: apply output

Doxygen Comments

```
/**
* @brief Initialise ActuatorStateMachine.
* @trace DD-COMP-030 SW-REQ-019
*/
/**
* @brief Run ActuatorStateMachine process cycle.
* @param event_flags Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-030 SW-REQ-019
*/
/**
* @brief Eval event.
* @trace DD-COMP-030 SW-REQ-019
*/
/**
* @brief Update state.
```

```
 * @trace DD-COMP-030 SW-REQ-019
 */
 /**
 * @brief Apply output.
 * @trace DD-COMP-030 SW-REQ-019
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read inputs and events
2	evaluate transition conditions
3	update current state
4	compute desired actuator command
5	publish outputs
6	log or update diagnostics

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	implicit state changes
3	shared mutable globals

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'asm' (e.g. asmresult).`
2	`State struct: static ActuatorStateMachine_t asmstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_030_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorStateMachine_t;`
	State struct for ActuatorStateMachine

`void init_state(void);`	Prototype: initialise ActuatorStateMachine
`uint8_t run_state(void *p_data);`	Prototype: run ActuatorStateMachine process cycle
`uint8_t eval_event(void);`	Prototype: eval event
`uint8_t update_state(void);`	Prototype: update state
`uint8_t apply_output(void);`	Prototype: apply output

`typedef enum {`
`DD_COMP_030_ERR_NONE = 0x00U,`
`DD_COMP_030_ERR_NULL = 0x01U,`
`DD_COMP_030_ERR_INVALID = 0x02U,`
`DD_COMP_030_ERR_TIMEOUT = 0x03U`
`} DD_COMP_030_ERR_t;`
	Error code enum for ActuatorStateMachine

main.c - What to Add

Item	Purpose
`static ActuatorStateMachine_t asmstate = {0};`	Module-level zero-initialised state for ActuatorStateMachine

`/* Process function logic order:`
`* Step 1: read inputs and events`
`* Step 2: evaluate transition conditions`
`* Step 3: update current state`
`* Step 4: compute desired actuator command`
`* Step 5: publish outputs`
`* Step 6: log or update diagnostics`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call init_state() once in main() init sequence. Call run_state() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

33. DD-COMP-031 - ActuatorStateMachine_OFF

DD-COMP-031	**ActuatorStateMachine_OFF** | Class | Module: ControlModule | Pattern: State | SW-REQ-020

`@trace DD-COMP-031 SW-REQ-020`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_off_state(void)`	void	Initialise all state for ActuatorStateMachine_OFF
main.c	`static uint8_t process_off_state(void *req_on)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorStateMachine_OFF
main.c	`static uint8_t check_request(void)`	uint8_t	Helper function: check request
main.c	`static uint8_t clear_outputs(void)`	uint8_t	Helper function: clear outputs
main.c	`static uint8_t set_next_state(void)`	uint8_t	Helper function: set next state

Doxygen Comments

```
/**
* @brief Initialise ActuatorStateMachine_OFF.
* @trace DD-COMP-031 SW-REQ-020
*/
/**
* @brief Run ActuatorStateMachine_OFF process cycle.
* @param req_on Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-031 SW-REQ-020
*/
/**
* @brief Check request.
* @trace DD-COMP-031 SW-REQ-020
*/
/**
* @brief Clear outputs.
```

```
 * @trace DD-COMP-031 SW-REQ-020
 */
 /**
 * @brief Set next state.
 * @trace DD-COMP-031 SW-REQ-020
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Read inputs and current state
2	Check for activation request
3	Maintain actuator off outputs
4	Reset timers and latches
5	Update next state if requested

Forbidden - Must NOT Do

#	Forbidden Rule
1	changing outputs without request
2	leaving timers running
3	ignoring fault flags

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'off' (e.g. offresult).`
2	`State struct: static ActuatorOffState_t offstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`
6	`Error returns: use DD_COMP_031_ERR_t enum values, never raw integers.`

7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorOffState_t;`
	State struct for ActuatorStateMachine_OFF

`void init_off_state(void);`	Prototype: initialise ActuatorStateMachine_OFF
`uint8_t process_off_state(void *p_data);`	Prototype: run ActuatorStateMachine_OFF process cycle
`uint8_t check_request(void);`	Prototype: check request
`uint8_t clear_outputs(void);`	Prototype: clear outputs
`uint8_t set_next_state(void);`	Prototype: set next state

`typedef enum {`
`DD_COMP_031_ERR_NONE = 0x00U,`
`DD_COMP_031_ERR_NULL = 0x01U,`
`DD_COMP_031_ERR_INVALID = 0x02U,`
`DD_COMP_031_ERR_TIMEOUT = 0x03U`
`} DD_COMP_031_ERR_t;`
	Error code enum for ActuatorStateMachine_OFF

main.c - What to Add

Item	Purpose
`static ActuatorOffState_t offstate = {0};`	Module-level zero-initialised state for ActuatorStateMachine_OFF

`/* Process function logic order:`
`* Step 1: Read inputs and current state`
`* Step 2: Check for activation request`
`* Step 3: Maintain actuator off outputs`
`* Step 4: Reset timers and latches`
`* Step 5: Update next state if requested`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call init_off_state() once in main() init sequence. Call process_off_state() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

34. DD-COMP-032 - UartAckService

DD-COMP-032	**UartAckService** | Service | Module: UartModule | Pattern: Observer | SW-REQ-021

`@trace DD-COMP-032 SW-REQ-021`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void uartAckInit(void)`	void	Initialise all state for UartAckService
`main.c`	`static uint8_t uartAckProcess(void *rxBuffer)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartAckService
`main.c`	`static uint8_t parseFrame(void)`	uint8_t	Helper function: parseFrame
`main.c`	`static uint8_t sendAck(void)`	uint8_t	Helper function: sendAck
`main.c`	`static uint8_t checkTimeout(void)`	uint8_t	Helper function: checkTimeout

Doxygen Comments

```
/**
* @brief Initialise UartAckService.
* @trace DD-COMP-032 SW-REQ-021
*/
/**
* @brief Run UartAckService process cycle.
* @param rxBuffer Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-032 SW-REQ-021
*/
/**
* @brief Parseframe.
* @trace DD-COMP-032 SW-REQ-021
*/
/**
* @brief Sendack.
```

```
 * @trace DD-COMP-032 SW-REQ-021
 */
 /**
 * @brief Checktimeout.
 * @trace DD-COMP-032 SW-REQ-021
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read incoming bytes
2	assemble UART frame
3	validate checksum
4	detect ack request
5	send ack reply
6	update timers/state
7	report errors

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	unchecked buffer bounds
3	hidden global state
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'ack' (e.g. ackresult).
2	State struct: static UartAckService_t ackstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_032_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartAckService_t;`
	State struct for UartAckService

`void uartAckInit(void);`	Prototype: initialise UartAckService
`uint8_t uartAckProcess(void *p_data);`	Prototype: run UartAckService process cycle
`uint8_t parseFrame(void);`	Prototype: parseFrame
`uint8_t sendAck(void);`	Prototype: sendAck
`uint8_t checkTimeout(void);`	Prototype: checkTimeout

`typedef enum {`
`DD_COMP_032_ERR_NONE = 0x00U,`
`DD_COMP_032_ERR_NULL = 0x01U,`
`DD_COMP_032_ERR_INVALID = 0x02U,`
`DD_COMP_032_ERR_TIMEOUT = 0x03U`
`} DD_COMP_032_ERR_t;`
	Error code enum for UartAckService

main.c - What to Add

Item	Purpose

`static UartAckService_t ackstate = {0};`	Module-level zero-initialised state for UartAckService

`/* Process function logic order:`
`* Step 1: read incoming bytes`
`* Step 2: assemble UART frame`
`* Step 3: validate checksum`
`* Step 4: detect ack request`
`* Step 5: send ack reply`
`* Step 6: update timers/state`
`* Step 7: report errors`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call uartAckInit() once in main() init sequence. Call uartAckProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

35. DD-COMP-033 - AdcDriverConfig

DD-COMP-033	**AdcDriverConfig** | Class | Module: AdcModule | Pattern: Facade | SW-REQ-023

`@trace DD-COMP-033 SW-REQ-023`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_cfg(void)`	void	Initialise all state for AdcDriverConfig
main.c	`static uint8_t apply_cfg(void *config_data)`	uint8_t - 0 success, non-zero error code	Execute core logic for AdcDriverConfig
main.c	`static uint8_t load_params(void)`	uint8_t	Helper function: load params
main.c	`static uint8_t validate_cfg(void)`	uint8_t	Helper function: validate cfg
main.c	`static uint8_t write_regs(void)`	uint8_t	Helper function: write regs

Doxygen Comments

```
/**
* @brief Initialise AdcDriverConfig.
* @trace DD-COMP-033 SW-REQ-023
*/
/**
* @brief Run AdcDriverConfig process cycle.
* @param config_data Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-033 SW-REQ-023
*/
/**
* @brief Load params.
* @trace DD-COMP-033 SW-REQ-023
*/
/**
* @brief Validate cfg.
```

```
 * @trace DD-COMP-033 SW-REQ-023
 */
 /**
 * @brief Write regs.
 * @trace DD-COMP-033 SW-REQ-023
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load default parameters
2	read configuration inputs
3	validate ranges and channels
4	map settings to registers
5	write configuration to ADC
6	confirm configuration status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded magic numbers
2	blocking waits in config
3	invalid channel indices

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adc' (e.g. adcresult).`
2	`State struct: static AdcDriverConfig_t adcstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_033_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcDriverConfig_t;`
	State struct for AdcDriverConfig

`void init_cfg(void);`	Prototype: initialise AdcDriverConfig
`uint8_t apply_cfg(void *p_data);`	Prototype: run AdcDriverConfig process cycle
`uint8_t load_params(void);`	Prototype: load params
`uint8_t validate_cfg(void);`	Prototype: validate cfg
`uint8_t write_regs(void);`	Prototype: write regs

`typedef enum {`
`DD_COMP_033_ERR_NONE = 0x00U,`
`DD_COMP_033_ERR_NULL = 0x01U,`
`DD_COMP_033_ERR_INVALID = 0x02U,`
`DD_COMP_033_ERR_TIMEOUT = 0x03U`
`} DD_COMP_033_ERR_t;`
	Error code enum for AdcDriverConfig

main.c - What to Add

Item	Purpose
`static AdcDriverConfig_t adcstate = {0};`	Module-level zero-initialised state for AdcDriverConfig

`/* Process function logic order:`
`* Step 1: load default parameters`
`* Step 2: read configuration inputs`
`* Step 3: validate ranges and channels`
`* Step 4: map settings to registers`
`* Step 5: write configuration to ADC`
`* Step 6: confirm configuration status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call init_cfg() once in main() init sequence. Call apply_cfg() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

36. DD-COMP-034 - AdcSamplingScheduler

DD-COMP-034	**AdcSamplingScheduler** | Service | Module: AdcModule | Pattern: Producer-Consumer | SW-REQ-024

`@trace DD-COMP-034 SW-REQ-024`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void AdcSchedInit(void)`	void	Initialise all state for AdcSamplingScheduler
`main.c`	`static uint8_t AdcSchedRun(void *config)`	uint8_t - 0 success, non-zero error code	Execute core logic for AdcSamplingScheduler
`main.c`	`static uint8_t LoadConfig(void)`	uint8_t	Helper function: LoadConfig
`main.c`	`static uint8_t ScheduleNext(void)`	uint8_t	Helper function: ScheduleNext
`main.c`	`static uint8_t TriggerAdc(void)`	uint8_t	Helper function: TriggerAdc

Doxygen Comments

```
/**
* @brief Initialise AdcSamplingScheduler.
* @trace DD-COMP-034 SW-REQ-024
*/
/**
* @brief Run AdcSamplingScheduler process cycle.
* @param config Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-034 SW-REQ-024
*/
/**
* @brief Loadconfig.
* @trace DD-COMP-034 SW-REQ-024
*/
/**
```

```
 * @brief Schedulenext.
 * @trace DD-COMP-034 SW-REQ-024
 */
 /**
 * @brief Triggeradc.
 * @trace DD-COMP-034 SW-REQ-024
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load scheduler configuration
2	check timing and pending requests
3	select next channel to sample
4	trigger ADC conversion
5	collect conversion status
6	update next schedule time

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	busy-wait loops
3	hardcoded timings
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'sched' (e.g. schedresult).`

2	State struct: static AdcSamplingScheduler_t schedstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_034_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcSamplingScheduler_t;`
	State struct for AdcSamplingScheduler

`void AdcSchedInit(void);`	Prototype: initialise AdcSamplingScheduler
`uint8_t AdcSchedRun(void *p_data);`	Prototype: run AdcSamplingScheduler process cycle
`uint8_t LoadConfig(void);`	Prototype: LoadConfig
`uint8_t ScheduleNext(void);`	Prototype: ScheduleNext
`uint8_t TriggerAdc(void);`	Prototype: TriggerAdc

`typedef enum {`
`DD_COMP_034_ERR_NONE = 0x00U,`
`DD_COMP_034_ERR_NULL = 0x01U,`
`DD_COMP_034_ERR_INVALID = 0x02U,`
`DD_COMP_034_ERR_TIMEOUT = 0x03U`
`} DD_COMP_034_ERR_t;`
	Error code enum for AdcSamplingScheduler

main.c - What to Add

Item	Purpose

`static AdcSamplingScheduler_t schedstate = {0};`	Module-level zero-initialised state for AdcSamplingScheduler

`/* Process function logic order:`
`* Step 1: load scheduler configuration`
`* Step 2: check timing and pending requests`
`* Step 3: select next channel to sample`
`* Step 4: trigger ADC conversion`
`* Step 5: collect conversion status`
`* Step 6: update next schedule time`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Producer-Consumer. Call AdcSchedInit() once in main() init sequence. Call AdcSchedRun() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

37. DD-COMP-035 - AdcContinuousAcquisition

DD-COMP-035	**AdcContinuousAcquisition** | Service | Module: AdcModule | Pattern: Observer | SW-REQ-022

`@trace DD-COMP-035 SW-REQ-022`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void adcContInit(void)`	void	Initialise all state for AdcContinuousAcquisition
main.c	`static uint8_t adcContProcess(void *adc_handle)`	uint8_t - 0 success, non-zero error code	Execute core logic for AdcContinuousAcquisition
main.c	`static uint8_t adcCfgDma(void)`	uint8_t	Helper function: adcCfgDma
main.c	`static uint8_t adcStartStream(void)`	uint8_t	Helper function: adcStartStream
main.c	`static uint8_t adcStopStream(void)`	uint8_t	Helper function: adcStopStream

Doxygen Comments

```
/**
* @brief Initialise AdcContinuousAcquisition.
* @trace DD-COMP-035 SW-REQ-022
*/
/**
* @brief Run AdcContinuousAcquisition process cycle.
* @param adc_handle Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-035 SW-REQ-022
*/
/**
* @brief Adccfgdma.
* @trace DD-COMP-035 SW-REQ-022
*/
/**
* @brief Adcstartstream.
```

```
 * @trace DD-COMP-035 SW-REQ-022
 */
 /**
 * @brief Adcstopstream.
 * @trace DD-COMP-035 SW-REQ-022
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Initialize ADC and DMA resources
2	Configure sampling rate and channels
3	Start continuous conversion
4	Collect samples into buffer
5	Check for buffer full or errors
6	Publish or store acquired data
7	Handle stop request or reset

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays in process loop
2	unbounded buffer growth
3	reconfiguring ADC while running
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'adcCont' (e.g. adcContresult).
2	State struct: static AdcContinuousAcquisition_t adcContstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_035_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcContinuousAcquisition_t;`
	State struct for AdcContinuousAcquisition

`void adcContInit(void);`	Prototype: initialise AdcContinuousAcquisition
`uint8_t adcContProcess(void *p_data);`	Prototype: run AdcContinuousAcquisition process cycle
`uint8_t adcCfgDma(void);`	Prototype: adcCfgDma
`uint8_t adcStartStream(void);`	Prototype: adcStartStream
`uint8_t adcStopStream(void);`	Prototype: adcStopStream

`typedef enum {`
`DD_COMP_035_ERR_NONE = 0x00U,`
`DD_COMP_035_ERR_NULL = 0x01U,`
`DD_COMP_035_ERR_INVALID = 0x02U,`
`DD_COMP_035_ERR_TIMEOUT = 0x03U`
`} DD_COMP_035_ERR_t;`
	Error code enum for AdcContinuousAcquisition

main.c - What to Add

Item	Purpose
`static AdcContinuousAcquisition_t adcContstate = {0};`	Module-level zero-initialised state for AdcContinuousAcquisition

`/* Process function logic order:`
`* Step 1: Initialize ADC and DMA resources`
`* Step 2: Configure sampling rate and channels`
`* Step 3: Start continuous conversion`
`* Step 4: Collect samples into buffer`
`* Step 5: Check for buffer full or errors`
`* Step 6: Publish or store acquired data`
`* Step 7: Handle stop request or reset`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call adcContInit() once in main() init sequence. Call adcContProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

38. DD-COMP-036 - ADC Sample Buffer

DD-COMP-036	**ADC Sample Buffer** | Class | Module: SensingModule | Pattern: Singleton | SW-REQ-025

`@trace DD-COMP-036 SW-REQ-025`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void adcbuf_init(void)`	void	Initialise all state for ADC Sample Buffer
`main.c`	`static uint8_t adcbuf_update(void *adc_sample)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADC Sample Buffer
`main.c`	`static uint8_t adcbuf_push(void)`	uint8_t	Helper function: adcbuf push
`main.c`	`static uint8_t adcbuf_pop(void)`	uint8_t	Helper function: adcbuf pop
`main.c`	`static uint8_t adcbuf_clear(void)`	uint8_t	Helper function: adcbuf clear

Doxygen Comments

```
/**
* @brief Initialise ADC Sample Buffer.
* @trace DD-COMP-036 SW-REQ-025
*/
/**
* @brief Run ADC Sample Buffer process cycle.
* @param adc_sample Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-036 SW-REQ-025
*/
/**
* @brief Adcbuf push.
* @trace DD-COMP-036 SW-REQ-025
*/
/**
* @brief Adcbuf pop.
```

```
 * @trace DD-COMP-036 SW-REQ-025
 */
 /**
 * @brief Adcbuf clear.
 * @trace DD-COMP-036 SW-REQ-025
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize buffer indices and size
2	receive new ADC sample
3	store sample in ring buffer
4	advance write index with wraparound
5	update count and status flags
6	provide buffered samples on request

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking in ISR
2	buffer overflow
3	dynamic allocation

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcbuf' (e.g. adcbufresult).`
2	`State struct: static AdcSampleBuffer_t adcbufstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_036_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcSampleBuffer_t;`
	State struct for ADC Sample Buffer

`void adcbuf_init(void);`	Prototype: initialise ADC Sample Buffer
`uint8_t adcbuf_update(void *p_data);`	Prototype: run ADC Sample Buffer process cycle
`uint8_t adcbuf_push(void);`	Prototype: adcbuf push
`uint8_t adcbuf_pop(void);`	Prototype: adcbuf pop
`uint8_t adcbuf_clear(void);`	Prototype: adcbuf clear

`typedef enum {`
`DD_COMP_036_ERR_NONE = 0x00U,`
`DD_COMP_036_ERR_NULL = 0x01U,`
`DD_COMP_036_ERR_INVALID = 0x02U,`
`DD_COMP_036_ERR_TIMEOUT = 0x03U`
`} DD_COMP_036_ERR_t;`
	Error code enum for ADC Sample Buffer

main.c - What to Add

Item	Purpose
`static AdcSampleBuffer_t adcbufstate = {0};`	Module-level zero-initialised state for ADC Sample Buffer

`/* Process function logic order:`
`* Step 1: initialize buffer indices and size`
`* Step 2: receive new ADC sample`
`* Step 3: store sample in ring buffer`
`* Step 4: advance write index with wraparound`
`* Step 5: update count and status flags`
`* Step 6: provide buffered samples on request`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call adcbuf_init() once in main() init sequence. Call adcbuf_update() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

39. DD-COMP-037 - Flap Control Computation

DD-COMP-037	**Flap Control Computation** | Service | Module: ControlModule | Pattern: Strategy | SW-REQ-026

`@trace DD-COMP-037 SW-REQ-026`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void flapInit(void)`	void	Initialise all state for Flap Control Computation
`main.c`	`static uint8_t flapCompute(void *cmdAngle)`	uint8_t - 0 success, non-zero error code	Execute core logic for Flap Control Computation
`main.c`	`static uint8_t calcDemand(void)`	uint8_t	Helper function: calcDemand
`main.c`	`static uint8_t limitRate(void)`	uint8_t	Helper function: limitRate
`main.c`	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState

Doxygen Comments

```
/**
* @brief Initialise Flap Control Computation.
* @trace DD-COMP-037 SW-REQ-026
*/
/**
* @brief Run Flap Control Computation process cycle.
* @param cmdAngle Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-037 SW-REQ-026
*/
/**
* @brief Calcdemand.
* @trace DD-COMP-037 SW-REQ-026
*/
/**
* @brief Limitrate.
```

```
 * @trace DD-COMP-037 SW-REQ-026
 */
 /**
 * @brief Updatestate.
 * @trace DD-COMP-037 SW-REQ-026
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Read current airspeed and command inputs
2	Compute desired flap position
3	Apply rate limiting and bounds
4	Update internal state
5	Output actuator command

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded limits
2	blocking delays
3	unstable feedback loops
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'flap' (e.g. flapresult).`
2	`State struct: static FlapControl_t flapstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_037_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FlapControl_t;`
	State struct for Flap Control Computation

`void flapInit(void);`	Prototype: initialise Flap Control Computation
`uint8_t flapCompute(void *p_data);`	Prototype: run Flap Control Computation process cycle
`uint8_t calcDemand(void);`	Prototype: calcDemand
`uint8_t limitRate(void);`	Prototype: limitRate
`uint8_t updateState(void);`	Prototype: updateState

`typedef enum {`
`DD_COMP_037_ERR_NONE = 0x00U,`
`DD_COMP_037_ERR_NULL = 0x01U,`
`DD_COMP_037_ERR_INVALID = 0x02U,`
`DD_COMP_037_ERR_TIMEOUT = 0x03U`
`} DD_COMP_037_ERR_t;`
	Error code enum for Flap Control Computation

main.c - What to Add

Item	Purpose
`static FlapControl_t flapstate = {0};`	Module-level zero-initialised state for Flap Control Computation

`/* Process function logic order:`
`* Step 1: Read current airspeed and command inputs`
`* Step 2: Compute desired flap position`
`* Step 3: Apply rate limiting and bounds`
`* Step 4: Update internal state`
`* Step 5: Output actuator command`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Strategy. Call flapInit() once in main() init sequence. Call flapCompute() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

40. DD-COMP-038 - Actuation Range Guard

DD-COMP-038 **Actuation Range Guard** | Algorithm | Module: SafetyModule | Pattern: Decorator | SW-REQ-027

`@trace DD-COMP-038 SW-REQ-027`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initActRange(void)`	void	Initialise all state for Actuation Range Guard
main.c	`static uint8_t guardActRange(void *actuation_req)`	uint8_t - 0 success, non-zero error code	Execute core logic for Actuation Range Guard
main.c	`static uint8_t clampActuation(void)`	uint8_t	Helper function: clampActuation
main.c	`static uint8_t checkLimits(void)`	uint8_t	Helper function: checkLimits
main.c	`static uint8_t flagSaturation(void)`	uint8_t	Helper function: flagSaturation

Doxygen Comments

```
/**
* @brief Initialise Actuation Range Guard.
* @trace DD-COMP-038 SW-REQ-027
*/
/**
* @brief Run Actuation Range Guard process cycle.
* @param actuation_req Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-038 SW-REQ-027
*/
/**
* @brief Clampactuation.
* @trace DD-COMP-038 SW-REQ-027
*/
/**
* @brief Checklimits.
```

```
 * @trace DD-COMP-038 SW-REQ-027
 */
 /**
 * @brief Flagsaturation.
 * @trace DD-COMP-038 SW-REQ-027
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load range limits
2	read requested actuation
3	clamp to min/max bounds
4	set saturation flag
5	output guarded actuation

Forbidden - Must NOT Do

#	Forbidden Rule
1	magic numbers for limits
2	silent overflow
3	stateful side effects without reset
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'arg' (e.g. argresult).`
2	`State struct: static ActuationRangeGuard_t argstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_038_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuationRangeGuard_t;`
	State struct for Actuation Range Guard

`void initActRange(void);`	Prototype: initialise Actuation Range Guard
`uint8_t guardActRange(void *p_data);`	Prototype: run Actuation Range Guard process cycle
`uint8_t clampActuation(void);`	Prototype: clampActuation
`uint8_t checkLimits(void);`	Prototype: checkLimits
`uint8_t flagSaturation(void);`	Prototype: flagSaturation

`typedef enum {`
`DD_COMP_038_ERR_NONE = 0x00U,`
`DD_COMP_038_ERR_NULL = 0x01U,`
`DD_COMP_038_ERR_INVALID = 0x02U,`
`DD_COMP_038_ERR_TIMEOUT = 0x03U`
`} DD_COMP_038_ERR_t;`
	Error code enum for Actuation Range Guard

main.c - What to Add

Item	Purpose
`static ActuationRangeGuard_t argstate = {0};`	Module-level zero-initialised state for Actuation Range Guard

`/* Process function logic order:`
`* Step 1: load range limits`
`* Step 2: read requested actuation`
`* Step 3: clamp to min/max bounds`
`* Step 4: set saturation flag`
`* Step 5: output guarded actuation`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Decorator. Call initActRange() once in main() init sequence. Call guardActRange() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

41. DD-COMP-039 - Real-Time ADC Freshness Check

DD-COMP-039	**Real-Time ADC Freshness Check** | Algorithm | Module: SensingModule | Pattern: N/A | SW-REQ-028

`@trace DD-COMP-039 SW-REQ-028`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void FreshChk_Init(void)`	void	Initialise all state for Real-Time ADC Freshness Check
main.c	`static uint8_t FreshChk_Run(void *adc_timestamp)`	uint8_t - 0 success, non-zero error code	Execute core logic for Real-Time ADC Freshness Check
main.c	`static uint8_t UpdateTimestamp(void)`	uint8_t	Helper function: UpdateTimestamp
main.c	`static uint8_t IsStale(void)`	uint8_t	Helper function: IsStale
main.c	`static uint8_t FlagStatus(void)`	uint8_t	Helper function: FlagStatus

Doxygen Comments

```
/**
* @brief Initialise Real-Time ADC Freshness Check.
* @trace DD-COMP-039 SW-REQ-028
*/
/**
* @brief Run Real-Time ADC Freshness Check process cycle.
* @param adc_timestamp Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-039 SW-REQ-028
*/
/**
* @brief Updatetimestamp.
* @trace DD-COMP-039 SW-REQ-028
*/
/**
```

```
 * @brief Isstale.
 * @trace DD-COMP-039 SW-REQ-028
 */
 /**
 * @brief Flagstatus.
 * @trace DD-COMP-039 SW-REQ-028
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration thresholds
2	read current ADC timestamp
3	compute age since last sample
4	compare age to freshness limit
5	update freshness status flag
6	report or log status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	hardcoded timeouts
3	overflow in time math
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fresh' (e.g. freshresult).`

2	State struct: static AdcFreshCheck_t freshstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_039_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcFreshCheck_t;`
	State struct for Real-Time ADC Freshness Check

`void FreshChk_Init(void);`	Prototype: initialise Real-Time ADC Freshness Check
`uint8_t FreshChk_Run(void *p_data);`	Prototype: run Real-Time ADC Freshness Check process cycle
`uint8_t UpdateTimestamp(void);`	Prototype: UpdateTimestamp
`uint8_t IsStale(void);`	Prototype: IsStale
`uint8_t FlagStatus(void);`	Prototype: FlagStatus

`typedef enum {`
`DD_COMP_039_ERR_NONE = 0x00U,`
`DD_COMP_039_ERR_NULL = 0x01U,`
`DD_COMP_039_ERR_INVALID = 0x02U,`
`DD_COMP_039_ERR_TIMEOUT = 0x03U`
`} DD_COMP_039_ERR_t;`
	Error code enum for Real-Time ADC Freshness Check

main.c - What to Add

Item	Purpose

`static AdcFreshCheck_t freshstate = {0};`	Module-level zero-initialised state for Real-Time ADC Freshness Check

`/* Process function logic order:`
`* Step 1: load configuration thresholds`
`* Step 2: read current ADC timestamp`
`* Step 3: compute age since last sample`
`* Step 4: compare age to freshness limit`
`* Step 5: update freshness status flag`
`* Step 6: report or log status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call FreshChk_Init() once in main() init sequence. Call FreshChk_Run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

42. DD-COMP-040 - Open-Loop Inhibit Gate

DD-COMP-040	**Open-Loop Inhibit Gate** | Service | Module: SafetyModule | Pattern: Chain of Responsibility | SW-REQ-029

`@trace DD-COMP-040 SW-REQ-029`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void olig_init(void)`	void	Initialise all state for Open-Loop Inhibit Gate
`main.c`	`static uint8_t olig_process(void *inhibit_cmd)`	uint8_t - 0 success, non-zero error code	Execute core logic for Open-Loop Inhibit Gate
`main.c`	`static uint8_t olig_check_inhibit(void)`	uint8_t	Helper function: olig check inhibit
`main.c`	`static uint8_t olig_update_state(void)`	uint8_t	Helper function: olig update state
`main.c`	`static uint8_t olig_emit_output(void)`	uint8_t	Helper function: olig emit output

Doxygen Comments

```
/**
* @brief Initialise Open-Loop Inhibit Gate.
* @trace DD-COMP-040 SW-REQ-029
*/
/**
* @brief Run Open-Loop Inhibit Gate process cycle.
* @param inhibit_cmd Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-040 SW-REQ-029
*/
/**
* @brief Olig check inhibit.
* @trace DD-COMP-040 SW-REQ-029
*/
/**
```

```
 * @brief Olig update state.
 * @trace DD-COMP-040 SW-REQ-029
 */
 /**
 * @brief Olig emit output.
 * @trace DD-COMP-040 SW-REQ-029
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read current inhibit command
2	read open-loop control signal
3	evaluate inhibit condition
4	apply gate logic to control signal
5	update internal state flags
6	produce gated output

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays in process
2	hidden global state
3	ambiguous inhibit priority
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'olig' (e.g. oligresult).`

2	State struct: static OpenLoopInhibitGate_t oligstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_040_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} OpenLoopInhibitGate_t;`
	State struct for Open-Loop Inhibit Gate

`void olig_init(void);`	Prototype: initialise Open-Loop Inhibit Gate
`uint8_t olig_process(void *p_data);`	Prototype: run Open-Loop Inhibit Gate process cycle
`uint8_t olig_check_inhibit(void);`	Prototype: olig check inhibit
`uint8_t olig_update_state(void);`	Prototype: olig update state
`uint8_t olig_emit_output(void);`	Prototype: olig emit output

`typedef enum {`
`DD_COMP_040_ERR_NONE = 0x00U,`
`DD_COMP_040_ERR_NULL = 0x01U,`
`DD_COMP_040_ERR_INVALID = 0x02U,`
`DD_COMP_040_ERR_TIMEOUT = 0x03U`
`} DD_COMP_040_ERR_t;`
	Error code enum for Open-Loop Inhibit Gate

main.c - What to Add

Item	Purpose

`static OpenLoopInhibitGate_t oligstate = {0};`	Module-level zero-initialised state for Open-Loop Inhibit Gate

`/* Process function logic order:`
`* Step 1: read current inhibit command`
`* Step 2: read open-loop control signal`
`* Step 3: evaluate inhibit condition`
`* Step 4: apply gate logic to control signal`
`* Step 5: update internal state flags`
`* Step 6: produce gated output`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Chain of Responsibility. Call olig_init() once in main() init sequence. Call olig_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

43. DD-COMP-041 - ADC Validity Monitor

DD-COMP-041	**ADC Validity Monitor** | Service | Module: SensingModule | Pattern: Observer | SW-REQ-030

`@trace DD-COMP-041 SW-REQ-030`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void AdcValid_Init(void)`	void	Initialise all state for ADC Validity Monitor
main.c	`static uint8_t AdcValid_Process(void *adc_samples)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADC Validity Monitor
main.c	`static uint8_t CheckRange(void)`	uint8_t	Helper function: CheckRange
main.c	`static uint8_t UpdateStatus(void)`	uint8_t	Helper function: UpdateStatus
main.c	`static uint8_t ApplyFilter(void)`	uint8_t	Helper function: ApplyFilter

Doxygen Comments

```
/**
* @brief Initialise ADC Validity Monitor.
* @trace DD-COMP-041 SW-REQ-030
*/
/**
* @brief Run ADC Validity Monitor process cycle.
* @param adc_samples Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-041 SW-REQ-030
*/
/**
* @brief Checkrange.
* @trace DD-COMP-041 SW-REQ-030
*/
/**
* @brief Updatestatus.
```

```
 * @trace DD-COMP-041 SW-REQ-030
 */
 /**
 * @brief Applyfilter.
 * @trace DD-COMP-041 SW-REQ-030
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration and thresholds
2	read latest ADC samples
3	filter or average samples if needed
4	compare samples against limits
5	update validity status flags
6	report status to system

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking calls in process loop
2	hardcoded thresholds
3	mixing units without conversion
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adc' (e.g. adcresult).`

2	State struct: static AdcValidityMonitor_t adcstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_041_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcValidityMonitor_t;`
	State struct for ADC Validity Monitor

`void AdcValid_Init(void);`	Prototype: initialise ADC Validity Monitor
`uint8_t AdcValid_Process(void *p_data);`	Prototype: run ADC Validity Monitor process cycle
`uint8_t CheckRange(void);`	Prototype: CheckRange
`uint8_t UpdateStatus(void);`	Prototype: UpdateStatus
`uint8_t ApplyFilter(void);`	Prototype: ApplyFilter

`typedef enum {`
`DD_COMP_041_ERR_NONE = 0x00U,`
`DD_COMP_041_ERR_NULL = 0x01U,`
`DD_COMP_041_ERR_INVALID = 0x02U,`
`DD_COMP_041_ERR_TIMEOUT = 0x03U`
`} DD_COMP_041_ERR_t;`
	Error code enum for ADC Validity Monitor

main.c - What to Add

Item	Purpose

`static AdcValidityMonitor_t adcstate = {0};`	Module-level zero-initialised state for ADC Validity Monitor

`/* Process function logic order:`
`* Step 1: load configuration and thresholds`
`* Step 2: read latest ADC samples`
`* Step 3: filter or average samples if needed`
`* Step 4: compare samples against limits`
`* Step 5: update validity status flags`
`* Step 6: report status to system`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call AdcValid_Init() once in main() init sequence. Call AdcValid_Process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

44. DD-COMP-042 - Drive Permission Check

DD-COMP-042 **Drive Permission Check** | Algorithm | Module: SafetyModule | Pattern: N/A | SW-REQ-031

`@trace DD-COMP-042 SW-REQ-031`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initPerm(void)`	void	Initialise all state for Drive Permission Check
main.c	`static uint8_t checkPerm(void *user_id)`	uint8_t - 0 success, non-zero error code	Execute core logic for Drive Permission Check
main.c	`static uint8_t loadRules(void)`	uint8_t	Helper function: loadRules
main.c	`static uint8_t matchRole(void)`	uint8_t	Helper function: matchRole
main.c	`static uint8_t logDecision(void)`	uint8_t	Helper function: logDecision

Doxygen Comments

```
/**
* @brief Initialise Drive Permission Check.
* @trace DD-COMP-042 SW-REQ-031
*/
/**
* @brief Run Drive Permission Check process cycle.
* @param user_id Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-042 SW-REQ-031
*/
/**
* @brief Loadrules.
* @trace DD-COMP-042 SW-REQ-031
*/
/**
* @brief Matchrole.
```

```
 * @trace DD-COMP-042 SW-REQ-031
 */
 /**
 * @brief Logdecision.
 * @trace DD-COMP-042 SW-REQ-031
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load permission rules
2	validate input identifiers
3	resolve user role
4	match role against rules
5	determine access decision
6	record decision outcome

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded rule IDs
2	silent failures
3	blocking I/O in main path
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'perm' (e.g. permresult).`

2	State struct: static DrivePermCheck_t permstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_042_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} DrivePermCheck_t;`
	State struct for Drive Permission Check

`void initPerm(void);`	Prototype: initialise Drive Permission Check
`uint8_t checkPerm(void *p_data);`	Prototype: run Drive Permission Check process cycle
`uint8_t loadRules(void);`	Prototype: loadRules
`uint8_t matchRole(void);`	Prototype: matchRole
`uint8_t logDecision(void);`	Prototype: logDecision

`typedef enum {`
`DD_COMP_042_ERR_NONE = 0x00U,`
`DD_COMP_042_ERR_NULL = 0x01U,`
`DD_COMP_042_ERR_INVALID = 0x02U,`
`DD_COMP_042_ERR_TIMEOUT = 0x03U`
`} DD_COMP_042_ERR_t;`
	Error code enum for Drive Permission Check

main.c - What to Add

Item	Purpose

`static DrivePermCheck_t permstate = {0};`	Module-level zero-initialised state for Drive Permission Check

`/* Process function logic order:`
`* Step 1: load permission rules`
`* Step 2: validate input identifiers`
`* Step 3: resolve user role`
`* Step 4: match role against rules`
`* Step 5: determine access decision`
`* Step 6: record decision outcome`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call initPerm() once in main() init sequence. Call checkPerm() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

45. DD-COMP-043 - Target ADC Validator

DD-COMP-043	**Target ADC Validator** | Service | Module: ConfigModule | Pattern: Factory | SW-REQ-032

`@trace DD-COMP-043 SW-REQ-032`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_adc_val(void)`	void	Initialise all state for Target ADC Validator
main.c	`static uint8_t proc_adc_val(void *adc_sample)`	uint8_t - 0 success, non-zero error code	Execute core logic for Target ADC Validator
main.c	`static uint8_t load_adc_cfg(void)`	uint8_t	Helper function: load adc cfg
main.c	`static uint8_t check_adc_range(void)`	uint8_t	Helper function: check adc range
main.c	`static uint8_t report_adc_err(void)`	uint8_t	Helper function: report adc err

Doxygen Comments

```
/**
* @brief Initialise Target ADC Validator.
* @trace DD-COMP-043 SW-REQ-032
*/
/**
* @brief Run Target ADC Validator process cycle.
* @param adc_sample Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-043 SW-REQ-032
*/
/**
* @brief Load adc cfg.
* @trace DD-COMP-043 SW-REQ-032
*/
/**
* @brief Check adc range.
```

```
 * @trace DD-COMP-043 SW-REQ-032
 */
 /**
 * @brief Report adc err.
 * @trace DD-COMP-043 SW-REQ-032
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load validator configuration
2	ingest raw ADC sample
3	convert to engineering units
4	check limits and plausibility
5	flag invalid conditions
6	emit validation status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded limits
2	silent failures
3	blocking I/O
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcval' (e.g. adcvalresult).`

2	State struct: static TargetAdcValidator_t adcvalstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_043_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} TargetAdcValidator_t;`
	State struct for Target ADC Validator

`void init_adc_val(void);`	Prototype: initialise Target ADC Validator
`uint8_t proc_adc_val(void *p_data);`	Prototype: run Target ADC Validator process cycle
`uint8_t load_adc_cfg(void);`	Prototype: load adc cfg
`uint8_t check_adc_range(void);`	Prototype: check adc range
`uint8_t report_adc_err(void);`	Prototype: report adc err

`typedef enum {`
`DD_COMP_043_ERR_NONE = 0x00U,`
`DD_COMP_043_ERR_NULL = 0x01U,`
`DD_COMP_043_ERR_INVALID = 0x02U,`
`DD_COMP_043_ERR_TIMEOUT = 0x03U`
`} DD_COMP_043_ERR_t;`
	Error code enum for Target ADC Validator

main.c - What to Add

Item	Purpose

`static TargetAdcValidator_t adcvalstate = {0};`	Module-level zero-initialised state for Target ADC Validator

`/* Process function logic order:`
`* Step 1: load validator configuration`
`* Step 2: ingest raw ADC sample`
`* Step 3: convert to engineering units`
`* Step 4: check limits and plausibility`
`* Step 5: flag invalid conditions`
`* Step 6: emit validation status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call init_adc_val() once in main() init sequence. Call proc_adc_val() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

46. DD-COMP-044 - ActuatorSafetyMonitor

DD-COMP-044	**ActuatorSafetyMonitor** | Class | Module: ControlLoopModule | Pattern: Observer | SW-REQ-033

`@trace DD-COMP-044 SW-REQ-033`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initSafety(void)`	void	Initialise all state for ActuatorSafetyMonitor
main.c	`static uint8_t monitorSafety(void *actuator_cmd)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorSafetyMonitor
main.c	`static uint8_t checkLimits(void)`	uint8_t	Helper function: checkLimits
main.c	`static uint8_t detectFault(void)`	uint8_t	Helper function: detectFault
main.c	`static uint8_t updateStatus(void)`	uint8_t	Helper function: updateStatus

Doxygen Comments

```
/**
* @brief Initialise ActuatorSafetyMonitor.
* @trace DD-COMP-044 SW-REQ-033
*/
/**
* @brief Run ActuatorSafetyMonitor process cycle.
* @param actuator_cmd Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-044 SW-REQ-033
*/
/**
* @brief Checklimits.
* @trace DD-COMP-044 SW-REQ-033
*/
/**
* @brief Detectfault.
```

```
 * @trace DD-COMP-044 SW-REQ-033
 */
 /**
 * @brief Updatestatus.
 * @trace DD-COMP-044 SW-REQ-033
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read actuator state and commands
2	validate against safety limits
3	detect fault or anomaly conditions
4	update internal safety status
5	generate inhibit or alert outputs
6	log or record safety events

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits in processing loop
2	uncleared stale fault flags
3	hardcoded limits instead of config
4	ambiguous status codes

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'safe' (e.g. saferesult).`
2	`State struct: static ActuatorSafetyMonitor_t safestate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`

5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_044_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorSafetyMonitor_t;`
	State struct for ActuatorSafetyMonitor

`void initSafety(void);`	Prototype: initialise ActuatorSafetyMonitor
`uint8_t monitorSafety(void *p_data);`	Prototype: run ActuatorSafetyMonitor process cycle
`uint8_t checkLimits(void);`	Prototype: checkLimits
`uint8_t detectFault(void);`	Prototype: detectFault
`uint8_t updateStatus(void);`	Prototype: updateStatus

`typedef enum {`
`DD_COMP_044_ERR_NONE = 0x00U,`
`DD_COMP_044_ERR_NULL = 0x01U,`
`DD_COMP_044_ERR_INVALID = 0x02U,`
`DD_COMP_044_ERR_TIMEOUT = 0x03U`
`} DD_COMP_044_ERR_t;`
	Error code enum for ActuatorSafetyMonitor

main.c - What to Add

Item	Purpose
`static ActuatorSafetyMonitor_t safestate = {0};`	Module-level zero-initialised state for ActuatorSafetyMonitor

`/* Process function logic order:`
`* Step 1: read actuator state and commands`
`* Step 2: validate against safety limits`
`* Step 3: detect fault or anomaly conditions`
`* Step 4: update internal safety status`
`* Step 5: generate inhibit or alert outputs`
`* Step 6: log or record safety events`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initSafety() once in main() init sequence. Call monitorSafety() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

47. DD-COMP-045 - ActuatorTargetStopController

DD-COMP-045	**ActuatorTargetStopController** | Class | Module: ControlLoopModule | Pattern: Strategy | SW-REQ-034

`@trace DD-COMP-045 SW-REQ-034`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init(void)`	void	Initialise all state for ActuatorTargetStopController
`main.c`	`static uint8_t update(void *targetPos)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorTargetStopController
`main.c`	`static uint8_t computeStop(void)`	uint8_t	Helper function: computeStop
`main.c`	`static uint8_t applyLimits(void)`	uint8_t	Helper function: applyLimits
`main.c`	`static uint8_t resetState(void)`	uint8_t	Helper function: resetState

Doxygen Comments

```
/**
* @brief Initialise ActuatorTargetStopController.
* @trace DD-COMP-045 SW-REQ-034
*/
/**
* @brief Run ActuatorTargetStopController process cycle.
* @param targetPos Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-045 SW-REQ-034
*/
/**
* @brief Computestop.
* @trace DD-COMP-045 SW-REQ-034
*/
/**
```

```
* @brief Applylimits.
* @trace DD-COMP-045 SW-REQ-034
*/
/**
* @brief Resetstate.
* @trace DD-COMP-045 SW-REQ-034
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	read inputs and current state
2	determine if stop is requested
3	compute stop target position
4	apply safety and range limits
5	update internal state
6	output stop target and status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	global mutable state
3	magic numbers
4	unbounded outputs

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'atsc' (e.g. atscresult).`
2	`State struct: static ActuatorTargetStopController_t atscstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`

5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_045_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorTargetStopController_t;`
	State struct for ActuatorTargetStopController

`void init(void);`	Prototype: initialise ActuatorTargetStopController
`uint8_t update(void *p_data);`	Prototype: run ActuatorTargetStopController process cycle
`uint8_t computeStop(void);`	Prototype: computeStop
`uint8_t applyLimits(void);`	Prototype: applyLimits
`uint8_t resetState(void);`	Prototype: resetState

`typedef enum {`
`DD_COMP_045_ERR_NONE = 0x00U,`
`DD_COMP_045_ERR_NULL = 0x01U,`
`DD_COMP_045_ERR_INVALID = 0x02U,`
`DD_COMP_045_ERR_TIMEOUT = 0x03U`
`} DD_COMP_045_ERR_t;`
	Error code enum for ActuatorTargetStopController

main.c - What to Add

Item	Purpose
`static ActuatorTargetStopController_t atscstate = {0};`	Module-level zero-initialised state for ActuatorTargetStopController

`/* Process function logic order:`
`* Step 1: read inputs and current state`
`* Step 2: determine if stop is requested`
`* Step 3: compute stop target position`
`* Step 4: apply safety and range limits`
`* Step 5: update internal state`
`* Step 6: output stop target and status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Strategy. Call init() once in main() init sequence. Call update() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

48. DD-COMP-046 - SupplyVoltageADCChannelConfig

DD-COMP-046	**SupplyVoltageADCChannelConfig** | Service | Module: HardwareAbstractionLayer | Pattern: Facade | SW-REQ-035

`@trace DD-COMP-046 SW-REQ-035`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initSupplyAdcCfg(void)`	void	Initialise all state for SupplyVoltageADCChannelConfig
main.c	`static uint8_t processSupplyAdcCfg(void *adcChannelId)`	uint8_t - 0 success, non-zero error code	Execute core logic for SupplyVoltageADCChannelConfig
main.c	`static uint8_t loadAdcDefaults(void)`	uint8_t	Helper function: loadAdcDefaults
main.c	`static uint8_t validateAdcCfg(void)`	uint8_t	Helper function: validateAdcCfg
main.c	`static uint8_t applyAdcChannel(void)`	uint8_t	Helper function: applyAdcChannel

Doxygen Comments

```
/**
* @brief Initialise SupplyVoltageADCChannelConfig.
* @trace DD-COMP-046 SW-REQ-035
*/
/**
* @brief Run SupplyVoltageADCChannelConfig process cycle.
* @param adcChannelId Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-046 SW-REQ-035
*/
/**
* @brief Loadadcdefaults.
* @trace DD-COMP-046 SW-REQ-035
*/
```

```
/**
* @brief Validateadccfg.
* @trace DD-COMP-046 SW-REQ-035
*/
/**
* @brief Applyadcchannel.
* @trace DD-COMP-046 SW-REQ-035
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load default ADC channel parameters
2	read requested supply voltage channel settings
3	validate channel and scaling values
4	apply configuration to ADC driver
5	store active configuration state

Forbidden - Must NOT Do

#	Forbidden Rule
1	hard-coded channel numbers
2	skipping validation
3	blocking delays in process
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'supplyAdc' (e.g. supplyAdcresult).`

2	State struct: static SupplyVoltageAdcConfig_t supplyAdcstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_046_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SupplyVoltageAdcConfig_t;`
	State struct for SupplyVoltageADCChannelConfig

`void initSupplyAdcCfg(void);`	Prototype: initialise SupplyVoltageADCChannelConfig
`uint8_t processSupplyAdcCfg(void *p_data);`	Prototype: run SupplyVoltageADCChannelConfig process cycle
`uint8_t loadAdcDefaults(void);`	Prototype: loadAdcDefaults
`uint8_t validateAdcCfg(void);`	Prototype: validateAdcCfg
`uint8_t applyAdcChannel(void);`	Prototype: applyAdcChannel

`typedef enum {`
`DD_COMP_046_ERR_NONE = 0x00U,`
`DD_COMP_046_ERR_NULL = 0x01U,`
`DD_COMP_046_ERR_INVALID = 0x02U,`
`DD_COMP_046_ERR_TIMEOUT = 0x03U`
`} DD_COMP_046_ERR_t;`
	Error code enum for SupplyVoltageADCChannelConfig

main.c - What to Add

Item	Purpose

`static SupplyVoltageAdcConfig_t supplyAdcstate = {0};`	Module-level zero-initialised state for SupplyVoltageADCChannelConfig

`/* Process function logic order:`
`* Step 1: load default ADC channel parameters`
`* Step 2: read requested supply voltage channel settings`
`* Step 3: validate channel and scaling values`
`* Step 4: apply configuration to ADC driver`
`* Step 5: store active configuration state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call initSupplyAdcCfg() once in main() init sequence. Call processSupplyAdcCfg() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

49. DD-COMP-047 - SupplyVoltageScaler

DD-COMP-047 **SupplyVoltageScaler** | Algorithm | Module: VoltageMonitorModule | Pattern: N/A | SW-REQ-036

`@trace DD-COMP-047 SW-REQ-036`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void svs_init(void)`	void	Initialise all state for SupplyVoltageScaler
main.c	`static uint8_t svs_process(void *raw_voltage)`	uint8_t - 0 success, non-zero error code	Execute core logic for SupplyVoltageScaler
main.c	`static uint8_t svs_validate(void)`	uint8_t	Helper function: svs validate
main.c	`static uint8_t svs_compute(void)`	uint8_t	Helper function: svs compute
main.c	`static uint8_t svs_limit(void)`	uint8_t	Helper function: svs limit

Doxygen Comments

```
/**
* @brief Initialise SupplyVoltageScaler.
* @trace DD-COMP-047 SW-REQ-036
*/
/**
* @brief Run SupplyVoltageScaler process cycle.
* @param raw_voltage Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-047 SW-REQ-036
*/
/**
* @brief Svs validate.
* @trace DD-COMP-047 SW-REQ-036
*/
/**
* @brief Svs compute.
```

```
 * @trace DD-COMP-047 SW-REQ-036
 */
 /**
 * @brief Svs limit.
 * @trace DD-COMP-047 SW-REQ-036
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration parameters
2	read input supply voltage
3	validate and clamp input range
4	apply scaling factor and offset
5	limit output to safe bounds
6	update output value

Forbidden - Must NOT Do

#	Forbidden Rule
1	hard-coded constants
2	division by zero
3	overflow without clamping
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'svs' (e.g. svsresult).`

2	State struct: static SupplyVoltageScaler_t svsstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_047_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SupplyVoltageScaler_t;`
	State struct for SupplyVoltageScaler

`void svs_init(void);`	Prototype: initialise SupplyVoltageScaler
`uint8_t svs_process(void *p_data);`	Prototype: run SupplyVoltageScaler process cycle
`uint8_t svs_validate(void);`	Prototype: svs validate
`uint8_t svs_compute(void);`	Prototype: svs compute
`uint8_t svs_limit(void);`	Prototype: svs limit

`typedef enum {`
`DD_COMP_047_ERR_NONE = 0x00U,`
`DD_COMP_047_ERR_NULL = 0x01U,`
`DD_COMP_047_ERR_INVALID = 0x02U,`
`DD_COMP_047_ERR_TIMEOUT = 0x03U`
`} DD_COMP_047_ERR_t;`
	Error code enum for SupplyVoltageScaler

main.c - What to Add

Item	Purpose

`static SupplyVoltageScaler_t svsstate = {0};`	Module-level zero-initialised state for SupplyVoltageScaler

`/* Process function logic order:`
`* Step 1: load configuration parameters`
`* Step 2: read input supply voltage`
`* Step 3: validate and clamp input range`
`* Step 4: apply scaling factor and offset`
`* Step 5: limit output to safe bounds`
`* Step 6: update output value`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call svs_init() once in main() init sequence. Call svs_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

50. DD-COMP-048 - SupplyVoltageSampler

DD-COMP-048	**SupplyVoltageSampler** | Service | Module: VoltageMonitorModule | Pattern: Observer | SW-REQ-037

`@trace DD-COMP-048 SW-REQ-037`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void svs_init(void)`	void	Initialise all state for SupplyVoltageSampler
`main.c`	`static uint8_t svs_process(void *adc_channel)`	uint8_t - 0 success, non-zero error code	Execute core logic for SupplyVoltageSampler
`main.c`	`static uint8_t svs_read_adc(void)`	uint8_t	Helper function: svs read adc
`main.c`	`static uint8_t svs_filter(void)`	uint8_t	Helper function: svs filter
`main.c`	`static uint8_t svs_update_status(void)`	uint8_t	Helper function: svs update status

Doxygen Comments

```
/**
* @brief Initialise SupplyVoltageSampler.
* @trace DD-COMP-048 SW-REQ-037
*/
/**
* @brief Run SupplyVoltageSampler process cycle.
* @param adc_channel Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-048 SW-REQ-037
*/
/**
* @brief Svs read adc.
* @trace DD-COMP-048 SW-REQ-037
*/
/**
```

```
* @brief Svs filter.
* @trace DD-COMP-048 SW-REQ-037
*/
/**
* @brief Svs update status.
* @trace DD-COMP-048 SW-REQ-037
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	trigger ADC sampling
2	read raw voltage value
3	apply scaling and filtering
4	check against thresholds
5	store latest sample
6	publish status output

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking ADC reads
2	magic constants
3	unbounded buffers
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'svs' (e.g. svsresult).`

2	State struct: static SupplyVoltageSampler_t svsstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_048_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SupplyVoltageSampler_t;`
	State struct for SupplyVoltageSampler

`void svs_init(void);`	Prototype: initialise SupplyVoltageSampler
`uint8_t svs_process(void *p_data);`	Prototype: run SupplyVoltageSampler process cycle
`uint8_t svs_read_adc(void);`	Prototype: svs read adc
`uint8_t svs_filter(void);`	Prototype: svs filter
`uint8_t svs_update_status(void);`	Prototype: svs update status

`typedef enum {`
`DD_COMP_048_ERR_NONE = 0x00U,`
`DD_COMP_048_ERR_NULL = 0x01U,`
`DD_COMP_048_ERR_INVALID = 0x02U,`
`DD_COMP_048_ERR_TIMEOUT = 0x03U`
`} DD_COMP_048_ERR_t;`
	Error code enum for SupplyVoltageSampler

main.c - What to Add

Item	Purpose

`static SupplyVoltageSampler_t svsstate = {0};`	Module-level zero-initialised state for SupplyVoltageSampler

`/* Process function logic order:`
`* Step 1: trigger ADC sampling`
`* Step 2: read raw voltage value`
`* Step 3: apply scaling and filtering`
`* Step 4: check against thresholds`
`* Step 5: store latest sample`
`* Step 6: publish status output`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call svs_init() once in main() init sequence. Call svs_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

51. DD-COMP-049 - OvervoltageThresholdStore

DD-COMP-049	**OvervoltageThresholdStore** | Service | Module: ConfigModule | Pattern: Repository | SW-REQ-038

`@trace DD-COMP-049 SW-REQ-038`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initStore(void)`	void	Initialise all state for OvervoltageThresholdStore
main.c	`static uint8_t processStore(void *newThreshold)`	uint8_t - 0 success, non-zero error code	Execute core logic for OvervoltageThresholdStore
main.c	`static uint8_t loadDefaults(void)`	uint8_t	Helper function: loadDefaults
main.c	`static uint8_t saveThreshold(void)`	uint8_t	Helper function: saveThreshold
main.c	`static uint8_t validateLevel(void)`	uint8_t	Helper function: validateLevel

Doxygen Comments

```
/**
* @brief Initialise OvervoltageThresholdStore.
* @trace DD-COMP-049 SW-REQ-038
*/
/**
* @brief Run OvervoltageThresholdStore process cycle.
* @param newThreshold Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-049 SW-REQ-038
*/
/**
* @brief Loaddefaults.
* @trace DD-COMP-049 SW-REQ-038
*/
/**
```

```
 * @brief Savethreshold.
 * @trace DD-COMP-049 SW-REQ-038
 */
 /**
 * @brief Validatelevel.
 * @trace DD-COMP-049 SW-REQ-038
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load current threshold from storage
2	validate incoming threshold value
3	update stored threshold if valid
4	persist changes to nonvolatile memory
5	report status/result

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking I/O in main process
2	unsafe writes without validation
3	magic numbers for limits
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ovt' (e.g. ovtresult).`
2	`State struct: static OvervoltageThresholdStore_t ovtstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_049_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} OvervoltageThresholdStore_t;`
	State struct for OvervoltageThresholdStore

`void initStore(void);`	Prototype: initialise OvervoltageThresholdStore
`uint8_t processStore(void *p_data);`	Prototype: run OvervoltageThresholdStore process cycle
`uint8_t loadDefaults(void);`	Prototype: loadDefaults
`uint8_t saveThreshold(void);`	Prototype: saveThreshold
`uint8_t validateLevel(void);`	Prototype: validateLevel

`typedef enum {`
`DD_COMP_049_ERR_NONE = 0x00U,`
`DD_COMP_049_ERR_NULL = 0x01U,`
`DD_COMP_049_ERR_INVALID = 0x02U,`
`DD_COMP_049_ERR_TIMEOUT = 0x03U`
`} DD_COMP_049_ERR_t;`
	Error code enum for OvervoltageThresholdStore

main.c - What to Add

Item	Purpose

`static OvervoltageThresholdStore_t ovtstate = {0};`	Module-level zero-initialised state for OvervoltageThresholdStore

`/* Process function logic order:`
`* Step 1: load current threshold from storage`
`* Step 2: validate incoming threshold value`
`* Step 3: update stored threshold if valid`
`* Step 4: persist changes to nonvolatile memory`
`* Step 5: report status/result`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call initStore() once in main() init sequence. Call processStore() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

52. DD-COMP-050 - OvervoltageDetector

DD-COMP-050	**OvervoltageDetector** | Class | Module: VoltageMonitorModule | Pattern: Observer | SW-REQ-039

`@trace DD-COMP-050 SW-REQ-039`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void ovd_init(void)`	void	Initialise all state for OvervoltageDetector
main.c	`static uint8_t ovd_update(void *voltage_in)`	uint8_t - 0 success, non-zero error code	Execute core logic for OvervoltageDetector
main.c	`static uint8_t ovd_read(void)`	uint8_t	Helper function: ovd read
main.c	`static uint8_t ovd_filter(void)`	uint8_t	Helper function: ovd filter
main.c	`static uint8_t ovd_alarm(void)`	uint8_t	Helper function: ovd alarm

Doxygen Comments

```
/**
* @brief Initialise OvervoltageDetector.
* @trace DD-COMP-050 SW-REQ-039
*/
/**
* @brief Run OvervoltageDetector process cycle.
* @param voltage_in Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-050 SW-REQ-039
*/
/**
* @brief Ovd read.
* @trace DD-COMP-050 SW-REQ-039
*/
/**
* @brief Ovd filter.
```

```
 * @trace DD-COMP-050 SW-REQ-039
 */
 /**
 * @brief Ovd alarm.
 * @trace DD-COMP-050 SW-REQ-039
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read voltage input
2	apply filtering or averaging
3	compare to threshold
4	update overvoltage state
5	set alarm output
6	log or latch state as needed

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	hardcoded thresholds
3	unfiltered noise
4	global state

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ovd' (e.g. ovdresult).`
2	`State struct: static OvervoltageDetector_t ovdstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`

5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_050_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} OvervoltageDetector_t;`
	State struct for OvervoltageDetector

`void ovd_init(void);`	Prototype: initialise OvervoltageDetector
`uint8_t ovd_update(void *p_data);`	Prototype: run OvervoltageDetector process cycle
`uint8_t ovd_read(void);`	Prototype: ovd read
`uint8_t ovd_filter(void);`	Prototype: ovd filter
`uint8_t ovd_alarm(void);`	Prototype: ovd alarm

`typedef enum {`
`DD_COMP_050_ERR_NONE = 0x00U,`
`DD_COMP_050_ERR_NULL = 0x01U,`
`DD_COMP_050_ERR_INVALID = 0x02U,`
`DD_COMP_050_ERR_TIMEOUT = 0x03U`
`} DD_COMP_050_ERR_t;`
	Error code enum for OvervoltageDetector

main.c - What to Add

Item	Purpose
`static OvervoltageDetector_t ovdstate = {0};`	Module-level zero-initialised state for OvervoltageDetector

`/* Process function logic order:`
`* Step 1: read voltage input`
`* Step 2: apply filtering or averaging`
`* Step 3: compare to threshold`
`* Step 4: update overvoltage state`
`* Step 5: set alarm output`
`* Step 6: log or latch state as needed`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call ovd_init() once in main() init sequence. Call ovd_update() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

53. DD-COMP-051 - OvervoltageDriveDisabler

DD-COMP-051	**OvervoltageDriveDisabler** | Class | Module: ControlLoopModule | Pattern: Observer | SW-REQ-040

`@trace DD-COMP-051 SW-REQ-040`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initOvd(void)`	void	Initialise all state for OvervoltageDriveDisabler
main.c	`static uint8_t procOvd(void *busVoltage)`	uint8_t - 0 success, non-zero error code	Execute core logic for OvervoltageDriveDisabler
main.c	`static uint8_t checkLimit(void)`	uint8_t	Helper function: checkLimit
main.c	`static uint8_t updateLatch(void)`	uint8_t	Helper function: updateLatch
main.c	`static uint8_t setOutput(void)`	uint8_t	Helper function: setOutput

Doxygen Comments

```
/**
* @brief Initialise OvervoltageDriveDisabler.
* @trace DD-COMP-051 SW-REQ-040
*/
/**
* @brief Run OvervoltageDriveDisabler process cycle.
* @param busVoltage Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-051 SW-REQ-040
*/
/**
* @brief Checklimit.
* @trace DD-COMP-051 SW-REQ-040
*/
/**
```

```
 * @brief Updatelatch.
 * @trace DD-COMP-051 SW-REQ-040
 */
 /**
 * @brief Setoutput.
 * @trace DD-COMP-051 SW-REQ-040
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read voltage input
2	compare to overvoltage threshold
3	latch disable state if exceeded
4	apply optional reset condition
5	set drive enable output
6	report status flag

Forbidden - Must NOT Do

#	Forbidden Rule
1	avoid hardcoded magic thresholds
2	avoid blocking delays in process
3	avoid clearing latch without reset

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ovd' (e.g. ovdresult).`
2	`State struct: static OvervoltageDriveDisabler_t ovdstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_051_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} OvervoltageDriveDisabler_t;`
	State struct for OvervoltageDriveDisabler

`void initOvd(void);`	Prototype: initialise OvervoltageDriveDisabler
`uint8_t procOvd(void *p_data);`	Prototype: run OvervoltageDriveDisabler process cycle
`uint8_t checkLimit(void);`	Prototype: checkLimit
`uint8_t updateLatch(void);`	Prototype: updateLatch
`uint8_t setOutput(void);`	Prototype: setOutput

`typedef enum {`
`DD_COMP_051_ERR_NONE = 0x00U,`
`DD_COMP_051_ERR_NULL = 0x01U,`
`DD_COMP_051_ERR_INVALID = 0x02U,`
`DD_COMP_051_ERR_TIMEOUT = 0x03U`
`} DD_COMP_051_ERR_t;`
	Error code enum for OvervoltageDriveDisabler

main.c - What to Add

Item	Purpose
`static OvervoltageDriveDisabler_t ovdstate = {0};`	Module-level zero-initialised state for OvervoltageDriveDisabler

`/* Process function logic order:`
`* Step 1: read voltage input`
`* Step 2: compare to overvoltage threshold`
`* Step 3: latch disable state if exceeded`
`* Step 4: apply optional reset condition`
`* Step 5: set drive enable output`
`* Step 6: report status flag`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initOvd() once in main() init sequence. Call procOvd() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

54. DD-COMP-052 - ActuatorCommandGate

DD-COMP-052	**ActuatorCommandGate** | Class | Module: ControlModule | Pattern: Guard | SW-REQ-041

`@trace DD-COMP-052 SW-REQ-041`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initGate(void)`	void	Initialise all state for ActuatorCommandGate
main.c	`static uint8_t processGate(void *actuator_cmd)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorCommandGate
main.c	`static uint8_t validateCmd(void)`	uint8_t	Helper function: validateCmd
main.c	`static uint8_t applyGate(void)`	uint8_t	Helper function: applyGate
main.c	`static uint8_t resetGate(void)`	uint8_t	Helper function: resetGate

Doxygen Comments

```
/**
* @brief Initialise ActuatorCommandGate.
* @trace DD-COMP-052 SW-REQ-041
*/
/**
* @brief Run ActuatorCommandGate process cycle.
* @param actuator_cmd Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-052 SW-REQ-041
*/
/**
* @brief Validatecmd.
* @trace DD-COMP-052 SW-REQ-041
*/
/**
* @brief Applygate.
```

```
 * @trace DD-COMP-052 SW-REQ-041
 */
 /**
 * @brief Resetgate.
 * @trace DD-COMP-052 SW-REQ-041
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize gate settings
2	receive actuator command
3	validate command against limits
4	apply gating rules
5	output approved command
6	handle rejection or defaults

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded limits
2	blocking operations
3	ambiguous names

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'gate' (e.g. gateresult).`
2	`State struct: static ActuatorCommandGate_t gatestate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_052_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorCommandGate_t;`
	State struct for ActuatorCommandGate

`void initGate(void);`	Prototype: initialise ActuatorCommandGate
`uint8_t processGate(void *p_data);`	Prototype: run ActuatorCommandGate process cycle
`uint8_t validateCmd(void);`	Prototype: validateCmd
`uint8_t applyGate(void);`	Prototype: applyGate
`uint8_t resetGate(void);`	Prototype: resetGate

`typedef enum {`
`DD_COMP_052_ERR_NONE = 0x00U,`
`DD_COMP_052_ERR_NULL = 0x01U,`
`DD_COMP_052_ERR_INVALID = 0x02U,`
`DD_COMP_052_ERR_TIMEOUT = 0x03U`
`} DD_COMP_052_ERR_t;`
	Error code enum for ActuatorCommandGate

main.c - What to Add

Item	Purpose
`static ActuatorCommandGate_t gatestate = {0};`	Module-level zero-initialised state for ActuatorCommandGate

`/* Process function logic order:`
`* Step 1: initialize gate settings`
`* Step 2: receive actuator command`
`* Step 3: validate command against limits`
`* Step 4: apply gating rules`
`* Step 5: output approved command`
`* Step 6: handle rejection or defaults`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Guard. Call initGate() once in main() init sequence. Call processGate() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

55. DD-COMP-053 - FaultTransitionHandler

DD-COMP-053	**FaultTransitionHandler** | Class | Module: FaultManagementModule | Pattern: Observer | SW-REQ-042

`@trace DD-COMP-053 SW-REQ-042`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void initFaultTrans(void)`	void	Initialise all state for FaultTransitionHandler
`main.c`	`static uint8_t handleFaults(void *fault_state)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultTransitionHandler
`main.c`	`static uint8_t loadFaultRules(void)`	uint8_t	Helper function: loadFaultRules
`main.c`	`static uint8_t checkTransitions(void)`	uint8_t	Helper function: checkTransitions
`main.c`	`static uint8_t applyRecovery(void)`	uint8_t	Helper function: applyRecovery

Doxygen Comments

```
/**
* @brief Initialise FaultTransitionHandler.
* @trace DD-COMP-053 SW-REQ-042
*/
/**
* @brief Run FaultTransitionHandler process cycle.
* @param fault_state Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-053 SW-REQ-042
*/
/**
* @brief Loadfaultrules.
* @trace DD-COMP-053 SW-REQ-042
*/
/**
```

```
 * @brief Checktransitions.
 * @trace DD-COMP-053 SW-REQ-042
 */
 /**
 * @brief Applyrecovery.
 * @trace DD-COMP-053 SW-REQ-042
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load fault transition rules
2	read current fault state
3	evaluate eligible transitions
4	apply transition actions
5	update fault state
6	log transition result

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded rules
2	silent state changes
3	ignoring invalid events

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fault' (e.g. faultresult).`
2	`State struct: static FaultTransitionHandler_t faultstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_053_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultTransitionHandler_t;`
	State struct for FaultTransitionHandler

`void initFaultTrans(void);`	Prototype: initialise FaultTransitionHandler
`uint8_t handleFaults(void *p_data);`	Prototype: run FaultTransitionHandler process cycle
`uint8_t loadFaultRules(void);`	Prototype: loadFaultRules
`uint8_t checkTransitions(void);`	Prototype: checkTransitions
`uint8_t applyRecovery(void);`	Prototype: applyRecovery

`typedef enum {`
`DD_COMP_053_ERR_NONE = 0x00U,`
`DD_COMP_053_ERR_NULL = 0x01U,`
`DD_COMP_053_ERR_INVALID = 0x02U,`
`DD_COMP_053_ERR_TIMEOUT = 0x03U`
`} DD_COMP_053_ERR_t;`
	Error code enum for FaultTransitionHandler

main.c - What to Add

Item	Purpose
`static FaultTransitionHandler_t faultstate = {0};`	Module-level zero-initialised state for FaultTransitionHandler

`/* Process function logic order:`
`* Step 1: load fault transition rules`
`* Step 2: read current fault state`
`* Step 3: evaluate eligible transitions`
`* Step 4: apply transition actions`
`* Step 5: update fault state`
`* Step 6: log transition result`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initFaultTrans() once in main() init sequence. Call handleFaults() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

56. DD-COMP-054 - OvervoltageUartNotifier

DD-COMP-054	**OvervoltageUartNotifier** | Service | Module: DiagnosticsModule | Pattern: Observer | SW-REQ-043

`@trace DD-COMP-054 SW-REQ-043`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void ovun_init(void)`	void	Initialise all state for OvervoltageUartNotifier
`main.c`	`static uint8_t ovun_process(void *voltage_status)`	uint8_t - 0 success, non-zero error code	Execute core logic for OvervoltageUartNotifier
`main.c`	`static uint8_t ovun_check(void)`	uint8_t	Helper function: ovun check
`main.c`	`static uint8_t ovun_format(void)`	uint8_t	Helper function: ovun format
`main.c`	`static uint8_t ovun_send(void)`	uint8_t	Helper function: ovun send

Doxygen Comments

```
/**
* @brief Initialise OvervoltageUartNotifier.
* @trace DD-COMP-054 SW-REQ-043
*/
/**
* @brief Run OvervoltageUartNotifier process cycle.
* @param voltage_status Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-054 SW-REQ-043
*/
/**
* @brief Ovun check.
* @trace DD-COMP-054 SW-REQ-043
*/
/**
```

```
* @brief Ovun format.
* @trace DD-COMP-054 SW-REQ-043
*/
/**
* @brief Ovun send.
* @trace DD-COMP-054 SW-REQ-043
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	read voltage status
2	detect overvoltage event
3	build UART message
4	send message over UART
5	update notification state

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UART calls
2	spamming repeated alerts
3	hardcoded thresholds
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ovun' (e.g. ovunresult).`
2	`State struct: static OvervoltageUartNotifier_t ovunstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_054_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} OvervoltageUartNotifier_t;`
	State struct for OvervoltageUartNotifier

`void ovun_init(void);`	Prototype: initialise OvervoltageUartNotifier
`uint8_t ovun_process(void *p_data);`	Prototype: run OvervoltageUartNotifier process cycle
`uint8_t ovun_check(void);`	Prototype: ovun check
`uint8_t ovun_format(void);`	Prototype: ovun format
`uint8_t ovun_send(void);`	Prototype: ovun send

`typedef enum {`
`DD_COMP_054_ERR_NONE = 0x00U,`
`DD_COMP_054_ERR_NULL = 0x01U,`
`DD_COMP_054_ERR_INVALID = 0x02U,`
`DD_COMP_054_ERR_TIMEOUT = 0x03U`
`} DD_COMP_054_ERR_t;`
	Error code enum for OvervoltageUartNotifier

main.c - What to Add

Item	Purpose
`static OvervoltageUartNotifier_t ovunstate = {0};`	Module-level zero-initialised state for OvervoltageUartNotifier

`/* Process function logic order:`
`* Step 1: read voltage status`
`* Step 2: detect overvoltage event`
`* Step 3: build UART message`
`* Step 4: send message over UART`
`* Step 5: update notification state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call ovun_init() once in main() init sequence. Call ovun_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

57. DD-COMP-055 - FaultStateRetentionLogic

DD-COMP-055	**FaultStateRetentionLogic** | Class | Module: FaultManagementModule | Pattern: State | SW-REQ-044

`@trace DD-COMP-055 SW-REQ-044`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initRetention(void)`	void	Initialise all state for FaultStateRetentionLogic
main.c	`static uint8_t processRetention(void *fault_flags)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultStateRetentionLogic
main.c	`static uint8_t detectFault(void)`	uint8_t	Helper function: detectFault
main.c	`static uint8_t updateHoldTimer(void)`	uint8_t	Helper function: updateHoldTimer
main.c	`static uint8_t clearOnRecovery(void)`	uint8_t	Helper function: clearOnRecovery

Doxygen Comments

```
/**
* @brief Initialise FaultStateRetentionLogic.
* @trace DD-COMP-055 SW-REQ-044
*/
/**
* @brief Run FaultStateRetentionLogic process cycle.
* @param fault_flags Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-055 SW-REQ-044
*/
/**
* @brief Detectfault.
* @trace DD-COMP-055 SW-REQ-044
*/
/**
```

```
* @brief Updateholdtimer.
* @trace DD-COMP-055 SW-REQ-044
*/
/**
* @brief Clearonrecovery.
* @trace DD-COMP-055 SW-REQ-044
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load previous fault state
2	check current fault inputs
3	detect transitions and latch as needed
4	update retention timer or counter
5	apply retention policy
6	clear latch on recovery conditions
7	publish retained fault state

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays in processing
2	global mutable state
3	ambiguous timer units

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ret' (e.g. retresult).`
2	`State struct: static FaultStateRetentionLogic_t retstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`

5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_055_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultStateRetentionLogic_t;`
	State struct for FaultStateRetentionLogic

`void initRetention(void);`	Prototype: initialise FaultStateRetentionLogic
`uint8_t processRetention(void *p_data);`	Prototype: run FaultStateRetentionLogic process cycle
`uint8_t detectFault(void);`	Prototype: detectFault
`uint8_t updateHoldTimer(void);`	Prototype: updateHoldTimer
`uint8_t clearOnRecovery(void);`	Prototype: clearOnRecovery

`typedef enum {`
`DD_COMP_055_ERR_NONE = 0x00U,`
`DD_COMP_055_ERR_NULL = 0x01U,`
`DD_COMP_055_ERR_INVALID = 0x02U,`
`DD_COMP_055_ERR_TIMEOUT = 0x03U`
`} DD_COMP_055_ERR_t;`
	Error code enum for FaultStateRetentionLogic

main.c - What to Add

Item	Purpose
`static FaultStateRetentionLogic_t retstate = {0};`	Module-level zero-initialised state for FaultStateRetentionLogic

`/* Process function logic order:`
`* Step 1: load previous fault state`
`* Step 2: check current fault inputs`
`* Step 3: detect transitions and latch as needed`
`* Step 4: update retention timer or counter`
`* Step 5: apply retention policy`
`* Step 6: clear latch on recovery conditions`
`* Step 7: publish retained fault state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initRetention() once in main() init sequence. Call processRetention() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

58. DD-COMP-056 - FaultExitVoltageChecker

DD-COMP-056	**FaultExitVoltageChecker** | Service | Module: FaultManagementModule | Pattern: Scheduler | SW-REQ-045

`@trace DD-COMP-056 SW-REQ-045`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initFaultExitVoltChk(void)`	void	Initialise all state for FaultExitVoltageChecker
main.c	`static uint8_t runFaultExitVoltChk(void *exit_voltage)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultExitVoltageChecker
main.c	`static uint8_t readExitVoltage(void)`	uint8_t	Helper function: readExitVoltage
main.c	`static uint8_t checkThresholds(void)`	uint8_t	Helper function: checkThresholds
main.c	`static uint8_t updateStatus(void)`	uint8_t	Helper function: updateStatus

Doxygen Comments

```
/**
* @brief Initialise FaultExitVoltageChecker.
* @trace DD-COMP-056 SW-REQ-045
*/
/**
* @brief Run FaultExitVoltageChecker process cycle.
* @param exit_voltage Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-056 SW-REQ-045
*/
/**
* @brief Readexitvoltage.
* @trace DD-COMP-056 SW-REQ-045
*/
```

```
/**
* @brief Checkthresholds.
* @trace DD-COMP-056 SW-REQ-045
*/
/**
* @brief Updatestatus.
* @trace DD-COMP-056 SW-REQ-045
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration thresholds
2	acquire exit voltage signal
3	filter or validate the reading
4	compare voltage against limits
5	set fault or clear state
6	report status and diagnostics

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded thresholds
2	blocking calls in process
3	uncleared fault latching
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'fev' (e.g. fevresult).
2	State struct: static FaultExitVoltageChecker_t fevstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_056_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultExitVoltageChecker_t;`
	State struct for FaultExitVoltageChecker

`void initFaultExitVoltChk(void);`	Prototype: initialise FaultExitVoltageChecker
`uint8_t runFaultExitVoltChk(void *p_data);`	Prototype: run FaultExitVoltageChecker process cycle
`uint8_t readExitVoltage(void);`	Prototype: readExitVoltage
`uint8_t checkThresholds(void);`	Prototype: checkThresholds
`uint8_t updateStatus(void);`	Prototype: updateStatus

`typedef enum {`
`DD_COMP_056_ERR_NONE = 0x00U,`
`DD_COMP_056_ERR_NULL = 0x01U,`
`DD_COMP_056_ERR_INVALID = 0x02U,`
`DD_COMP_056_ERR_TIMEOUT = 0x03U`
`} DD_COMP_056_ERR_t;`
	Error code enum for FaultExitVoltageChecker

main.c - What to Add

Item	Purpose
`static FaultExitVoltageChecker_t fevstate = {0};`	Module-level zero-initialised state for FaultExitVoltageChecker

`/* Process function logic order:`
`* Step 1: load configuration thresholds`
`* Step 2: acquire exit voltage signal`
`* Step 3: filter or validate the reading`
`* Step 4: compare voltage against limits`
`* Step 5: set fault or clear state`
`* Step 6: report status and diagnostics`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Scheduler. Call initFaultExitVoltChk() once in main() init sequence. Call runFaultExitVoltChk() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

59. DD-COMP-057 - OvervoltageMonitor

DD-COMP-057	**OvervoltageMonitor** | Service | Module: PowerSafetyModule | Pattern: Observer | SW-REQ-046

`@trace DD-COMP-057 SW-REQ-046`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void ovm_init(void)`	void	Initialise all state for OvervoltageMonitor
main.c	`static uint8_t ovm_process(void *voltage_adc)`	uint8_t - 0 success, non-zero error code	Execute core logic for OvervoltageMonitor
main.c	`static uint8_t ovm_read_voltage(void)`	uint8_t	Helper function: ovm read voltage
main.c	`static uint8_t ovm_check_threshold(void)`	uint8_t	Helper function: ovm check threshold
main.c	`static uint8_t ovm_update_state(void)`	uint8_t	Helper function: ovm update state

Doxygen Comments

```
/**
* @brief Initialise OvervoltageMonitor.
* @trace DD-COMP-057 SW-REQ-046
*/
/**
* @brief Run OvervoltageMonitor process cycle.
* @param voltage_adc Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-057 SW-REQ-046
*/
/**
* @brief Ovm read voltage.
* @trace DD-COMP-057 SW-REQ-046
*/
/**
```

```
* @brief Ovm check threshold.
* @trace DD-COMP-057 SW-REQ-046
*/
/**
* @brief Ovm update state.
* @trace DD-COMP-057 SW-REQ-046
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	Read latest voltage sample
2	Apply filtering or scaling
3	Compare against threshold
4	Update fault state and timers
5	Publish status or alarms

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	hardcoded thresholds
3	unfiltered noisy input
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ovm' (e.g. ovmresult).`
2	`State struct: static OvervoltageMonitor_t ovmstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_057_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} OvervoltageMonitor_t;`
	State struct for OvervoltageMonitor

`void ovm_init(void);`	Prototype: initialise OvervoltageMonitor
`uint8_t ovm_process(void *p_data);`	Prototype: run OvervoltageMonitor process cycle
`uint8_t ovm_read_voltage(void);`	Prototype: ovm read voltage
`uint8_t ovm_check_threshold(void);`	Prototype: ovm check threshold
`uint8_t ovm_update_state(void);`	Prototype: ovm update state

`typedef enum {`
`DD_COMP_057_ERR_NONE = 0x00U,`
`DD_COMP_057_ERR_NULL = 0x01U,`
`DD_COMP_057_ERR_INVALID = 0x02U,`
`DD_COMP_057_ERR_TIMEOUT = 0x03U`
`} DD_COMP_057_ERR_t;`
	Error code enum for OvervoltageMonitor

main.c - What to Add

Item	Purpose
`static OvervoltageMonitor_t ovmstate = {0};`	Module-level zero-initialised state for OvervoltageMonitor

`/* Process function logic order:`
`* Step 1: Read latest voltage sample`
`* Step 2: Apply filtering or scaling`
`* Step 3: Compare against threshold`
`* Step 4: Update fault state and timers`
`* Step 5: Publish status or alarms`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call ovm_init() once in main() init sequence. Call ovm_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

60. DD-COMP-058 - UartActuatorCommandInterface

DD-COMP-058	**UartActuatorCommandInterface** | Service | Module: UartModule | Pattern: Facade | SW-REQ-047

`@trace DD-COMP-058 SW-REQ-047`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void uart_cmdif_init(void)`	void	Initialise all state for UartActuatorCommandInterface
main.c	`static uint8_t uart_cmdif_process(void *uart_rx_buffer)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartActuatorCommandInterface
main.c	`static uint8_t parse_frame(void)`	uint8_t	Helper function: parse frame
main.c	`static uint8_t validate_cmd(void)`	uint8_t	Helper function: validate cmd
main.c	`static uint8_t send_ack(void)`	uint8_t	Helper function: send ack

Doxygen Comments

```
/**
* @brief Initialise UartActuatorCommandInterface.
* @trace DD-COMP-058 SW-REQ-047
*/
/**
* @brief Run UartActuatorCommandInterface process cycle.
* @param uart_rx_buffer Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-058 SW-REQ-047
*/
/**
* @brief Parse frame.
* @trace DD-COMP-058 SW-REQ-047
*/
/**
```

```
 * @brief Validate cmd.
 * @trace DD-COMP-058 SW-REQ-047
 */
 /**
 * @brief Send ack.
 * @trace DD-COMP-058 SW-REQ-047
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read bytes from UART buffer
2	detect and extract command frame
3	parse command fields
4	validate checksum and bounds
5	dispatch command to actuator layer
6	build and send response

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UART reads
2	unchecked buffer lengths
3	ambiguous command IDs
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uartcmd' (e.g. uartcmdresult).`

2	State struct: static UartActuatorCommandInterface_t uartcmdstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_058_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartActuatorCommandInterface_t;`
	State struct for UartActuatorCommandInterface

`void uart_cmdif_init(void);`	Prototype: initialise UartActuatorCommandInterface
`uint8_t uart_cmdif_process(void *p_data);`	Prototype: run UartActuatorCommandInterface process cycle
`uint8_t parse_frame(void);`	Prototype: parse frame
`uint8_t validate_cmd(void);`	Prototype: validate cmd
`uint8_t send_ack(void);`	Prototype: send ack

`typedef enum {`
`DD_COMP_058_ERR_NONE = 0x00U,`
`DD_COMP_058_ERR_NULL = 0x01U,`
`DD_COMP_058_ERR_INVALID = 0x02U,`
`DD_COMP_058_ERR_TIMEOUT = 0x03U`
`} DD_COMP_058_ERR_t;`
	Error code enum for UartActuatorCommandInterface

main.c - What to Add

Item	Purpose

`static UartActuatorCommandInterface_t uartcmdstate = {0};`	Module-level zero-initialised state for UartActuatorCommandInterface

`/* Process function logic order:`
`* Step 1: read bytes from UART buffer`
`* Step 2: detect and extract command frame`
`* Step 3: parse command fields`
`* Step 4: validate checksum and bounds`
`* Step 5: dispatch command to actuator layer`
`* Step 6: build and send response`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call uart_cmdif_init() once in main() init sequence. Call uart_cmdif_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

61. DD-COMP-059 - ActuatorOnCommandParser

DD-COMP-059 **ActuatorOnCommandParser** | Algorithm | Module: UartModule | Pattern: Factory | SW-REQ-048

`@trace DD-COMP-059 SW-REQ-048`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initParser(void)`	void	Initialise all state for ActuatorOnCommandParser
main.c	`static uint8_t parseCommand(void *cmdBuffer)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorOnCommandParser
main.c	`static uint8_t readToken(void)`	uint8_t	Helper function: readToken
main.c	`static uint8_t validateCmd(void)`	uint8_t	Helper function: validateCmd
main.c	`static uint8_t setActuator(void)`	uint8_t	Helper function: setActuator

Doxygen Comments

```
/**
* @brief Initialise ActuatorOnCommandParser.
* @trace DD-COMP-059 SW-REQ-048
*/
/**
* @brief Run ActuatorOnCommandParser process cycle.
* @param cmdBuffer Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-059 SW-REQ-048
*/
/**
* @brief Readtoken.
* @trace DD-COMP-059 SW-REQ-048
*/
/**
* @brief Validatecmd.
```

```
 * @trace DD-COMP-059 SW-REQ-048
 */
 /**
 * @brief Setactuator.
 * @trace DD-COMP-059 SW-REQ-048
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	reset parser state
2	read next command token
3	validate command format
4	interpret actuator target and action
5	update output action
6	report parse status

Forbidden - Must NOT Do

#	Forbidden Rule
1	dynamic allocation
2	blocking I/O
3	global mutable state
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'aocp' (e.g. aocpresult).`

2	State struct: static ActuatorOnCommandParser_t aocpstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_059_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorOnCommandParser_t;`
	State struct for ActuatorOnCommandParser

`void initParser(void);`	Prototype: initialise ActuatorOnCommandParser
`uint8_t parseCommand(void *p_data);`	Prototype: run ActuatorOnCommandParser process cycle
`uint8_t readToken(void);`	Prototype: readToken
`uint8_t validateCmd(void);`	Prototype: validateCmd
`uint8_t setActuator(void);`	Prototype: setActuator

`typedef enum {`
`DD_COMP_059_ERR_NONE = 0x00U,`
`DD_COMP_059_ERR_NULL = 0x01U,`
`DD_COMP_059_ERR_INVALID = 0x02U,`
`DD_COMP_059_ERR_TIMEOUT = 0x03U`
`} DD_COMP_059_ERR_t;`
	Error code enum for ActuatorOnCommandParser

main.c - What to Add

Item	Purpose

`static ActuatorOnCommandParser_t aocpstate = {0};`	Module-level zero-initialised state for ActuatorOnCommandParser

`/* Process function logic order:`
`* Step 1: reset parser state`
`* Step 2: read next command token`
`* Step 3: validate command format`
`* Step 4: interpret actuator target and action`
`* Step 5: update output action`
`* Step 6: report parse status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call initParser() once in main() init sequence. Call parseCommand() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

62. DD-COMP-060 - UART Command Parser

DD-COMP-060	**UART Command Parser** | Class | Module: UartCommModule | Pattern: N/A | SW-REQ-049

`@trace DD-COMP-060 SW-REQ-049`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initParser(void)`	void	Initialise all state for UART Command Parser
main.c	`static uint8_t processUart(void *uart_rx_byte)`	uint8_t - 0 success, non-zero error code	Execute core logic for UART Command Parser
main.c	`static uint8_t readByte(void)`	uint8_t	Helper function: readByte
main.c	`static uint8_t accumCmd(void)`	uint8_t	Helper function: accumCmd
main.c	`static uint8_t dispatchCmd(void)`	uint8_t	Helper function: dispatchCmd

Doxygen Comments

```
/**
* @brief Initialise UART Command Parser.
* @trace DD-COMP-060 SW-REQ-049
*/
/**
* @brief Run UART Command Parser process cycle.
* @param uart_rx_byte Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-060 SW-REQ-049
*/
/**
* @brief Readbyte.
* @trace DD-COMP-060 SW-REQ-049
*/
/**
* @brief Accumcmd.
```

```
 * @trace DD-COMP-060 SW-REQ-049
 */
 /**
 * @brief Dispatchcmd.
 * @trace DD-COMP-060 SW-REQ-049
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize buffers and state
2	read incoming UART bytes
3	accumulate bytes into command frame
4	validate frame format and checksum
5	parse command fields
6	execute or queue command
7	reset state on completion

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking reads
2	buffer overflow
3	complex parsing in ISR
4	silent parse errors

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uart' (e.g. uartresult).`
2	`State struct: static UartCommandParser_t uartstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`

4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_060_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartCommandParser_t;`
	State struct for UART Command Parser

`void initParser(void);`	Prototype: initialise UART Command Parser
`uint8_t processUart(void *p_data);`	Prototype: run UART Command Parser process cycle
`uint8_t readByte(void);`	Prototype: readByte
`uint8_t accumCmd(void);`	Prototype: accumCmd
`uint8_t dispatchCmd(void);`	Prototype: dispatchCmd

`typedef enum {`
`DD_COMP_060_ERR_NONE = 0x00U,`
`DD_COMP_060_ERR_NULL = 0x01U,`
`DD_COMP_060_ERR_INVALID = 0x02U,`
`DD_COMP_060_ERR_TIMEOUT = 0x03U`
`} DD_COMP_060_ERR_t;`
	Error code enum for UART Command Parser

main.c - What to Add

Item	Purpose
`static UartCommandParser_t uartstate = {0};`	Module-level zero-initialised state for UART Command Parser

`/* Process function logic order:`
`* Step 1: initialize buffers and state`
`* Step 2: read incoming UART bytes`
`* Step 3: accumulate bytes into command frame`
`* Step 4: validate frame format and checksum`
`* Step 5: parse command fields`
`* Step 6: execute or queue command`
`* Step 7: reset state on completion`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call initParser() once in main() init sequence. Call processUart() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

63. DD-COMP-061 - Command Dispatcher

DD-COMP-061	**Command Dispatcher** | Service | Module: CommandModule | Pattern: Command | SW-REQ-050

`@trace DD-COMP-061 SW-REQ-050`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_dispatch(void)`	void	Initialise all state for Command Dispatcher
main.c	`static uint8_t run_dispatch(void *command_packet)`	uint8_t - 0 success, non-zero error code	Execute core logic for Command Dispatcher
main.c	`static uint8_t parse_cmd(void)`	uint8_t	Helper function: parse cmd
main.c	`static uint8_t route_cmd(void)`	uint8_t	Helper function: route cmd
main.c	`static uint8_t log_result(void)`	uint8_t	Helper function: log result

Doxygen Comments

```
/**
* @brief Initialise Command Dispatcher.
* @trace DD-COMP-061 SW-REQ-050
*/
/**
* @brief Run Command Dispatcher process cycle.
* @param command_packet Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-061 SW-REQ-050
*/
/**
* @brief Parse cmd.
* @trace DD-COMP-061 SW-REQ-050
*/
/**
* @brief Route cmd.
```

```
 * @trace DD-COMP-061 SW-REQ-050
 */
 /**
 * @brief Log result.
 * @trace DD-COMP-061 SW-REQ-050
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	receive command input
3	validate command format
4	map command to handler
5	execute handler action
6	collect execution status
7	emit response

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking on I/O
2	hardcoded command maps
3	silent failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'cmd' (e.g. cmdresult).
2	State struct: static CommandDispatcher_t cmdstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_061_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} CommandDispatcher_t;`
	State struct for Command Dispatcher

`void init_dispatch(void);`	Prototype: initialise Command Dispatcher
`uint8_t run_dispatch(void *p_data);`	Prototype: run Command Dispatcher process cycle
`uint8_t parse_cmd(void);`	Prototype: parse cmd
`uint8_t route_cmd(void);`	Prototype: route cmd
`uint8_t log_result(void);`	Prototype: log result

`typedef enum {`
`DD_COMP_061_ERR_NONE = 0x00U,`
`DD_COMP_061_ERR_NULL = 0x01U,`
`DD_COMP_061_ERR_INVALID = 0x02U,`
`DD_COMP_061_ERR_TIMEOUT = 0x03U`
`} DD_COMP_061_ERR_t;`
	Error code enum for Command Dispatcher

main.c - What to Add

Item	Purpose
`static CommandDispatcher_t cmdstate = {0};`	Module-level zero-initialised state for Command Dispatcher

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: receive command input`
`* Step 3: validate command format`
`* Step 4: map command to handler`
`* Step 5: execute handler action`
`* Step 6: collect execution status`
`* Step 7: emit response`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Command. Call init_dispatch() once in main() init sequence. Call run_dispatch() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

64. DD-COMP-062 - Actuator Controller

DD-COMP-062	**Actuator Controller** | Service | Module: ActuatorModule | Pattern: Facade | SW-REQ-051

`@trace DD-COMP-062 SW-REQ-051`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initActuator(void)`	void	Initialise all state for Actuator Controller
main.c	`static uint8_t runActuator(void *cmdStream)`	uint8_t - 0 success, non-zero error code	Execute core logic for Actuator Controller
main.c	`static uint8_t parseCmd(void)`	uint8_t	Helper function: parseCmd
main.c	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState
main.c	`static uint8_t emitStatus(void)`	uint8_t	Helper function: emitStatus

Doxygen Comments

```
/**
* @brief Initialise Actuator Controller.
* @trace DD-COMP-062 SW-REQ-051
*/
/**
* @brief Run Actuator Controller process cycle.
* @param cmdStream Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-062 SW-REQ-051
*/
/**
* @brief Parsecmd.
* @trace DD-COMP-062 SW-REQ-051
*/
/**
* @brief Updatestate.
```

```
 * @trace DD-COMP-062 SW-REQ-051
 */
 /**
 * @brief Emitstatus.
 * @trace DD-COMP-062 SW-REQ-051
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	read command input
3	validate and parse command
4	update actuator state
5	apply control output
6	report status
7	log cycle summary

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	hard-coded limits
3	silent failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'act' (e.g. actresult).
2	State struct: static ActuatorController_t actstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_062_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorController_t;`
	State struct for Actuator Controller

`void initActuator(void);`	Prototype: initialise Actuator Controller
`uint8_t runActuator(void *p_data);`	Prototype: run Actuator Controller process cycle
`uint8_t parseCmd(void);`	Prototype: parseCmd
`uint8_t updateState(void);`	Prototype: updateState
`uint8_t emitStatus(void);`	Prototype: emitStatus

`typedef enum {`
`DD_COMP_062_ERR_NONE = 0x00U,`
`DD_COMP_062_ERR_NULL = 0x01U,`
`DD_COMP_062_ERR_INVALID = 0x02U,`
`DD_COMP_062_ERR_TIMEOUT = 0x03U`
`} DD_COMP_062_ERR_t;`
	Error code enum for Actuator Controller

main.c - What to Add

Item	Purpose

`static ActuatorController_t actstate = {0};`	Module-level zero-initialised state for Actuator Controller

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: read command input`
`* Step 3: validate and parse command`
`* Step 4: update actuator state`
`* Step 5: apply control output`
`* Step 6: report status`
`* Step 7: log cycle summary`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call initActuator() once in main() init sequence. Call runActuator() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

65. DD-COMP-063 - Invalid Command Handler

DD-COMP-063	**Invalid Command Handler** | Service | Module: CommandModule | Pattern: N/A | SW-REQ-052

`@trace DD-COMP-063 SW-REQ-052`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init_badcmd(void)`	void	Initialise all state for Invalid Command Handler
`main.c`	`static uint8_t handle_badcmd(void *raw_command)`	uint8_t - 0 success, non-zero error code	Execute core logic for Invalid Command Handler
`main.c`	`static uint8_t log_badcmd(void)`	uint8_t	Helper function: log badcmd
`main.c`	`static uint8_t format_hint(void)`	uint8_t	Helper function: format hint
`main.c`	`static uint8_t send_error(void)`	uint8_t	Helper function: send error

Doxygen Comments

```
/**
* @brief Initialise Invalid Command Handler.
* @trace DD-COMP-063 SW-REQ-052
*/
/**
* @brief Run Invalid Command Handler process cycle.
* @param raw_command Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-063 SW-REQ-052
*/
/**
* @brief Log badcmd.
* @trace DD-COMP-063 SW-REQ-052
*/
/**
* @brief Format hint.
```

```
 * @trace DD-COMP-063 SW-REQ-052
 */
 /**
 * @brief Send error.
 * @trace DD-COMP-063 SW-REQ-052
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Receive unknown command input
2	Normalize and validate input
3	Generate user-friendly error message
4	Log invalid command details
5	Send error response to client
6	Return control to caller

Forbidden - Must NOT Do

#	Forbidden Rule
1	Exposing internal command lists
2	Throwing unhandled exceptions
3	Blocking on slow I/O
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'badcmd' (e.g. badcmdresult).`

2	State struct: static InvalidCommandHandler_t badcmdstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_063_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} InvalidCommandHandler_t;`
	State struct for Invalid Command Handler

`void init_badcmd(void);`	Prototype: initialise Invalid Command Handler
`uint8_t handle_badcmd(void *p_data);`	Prototype: run Invalid Command Handler process cycle
`uint8_t log_badcmd(void);`	Prototype: log badcmd
`uint8_t format_hint(void);`	Prototype: format hint
`uint8_t send_error(void);`	Prototype: send error

`typedef enum {`
`DD_COMP_063_ERR_NONE = 0x00U,`
`DD_COMP_063_ERR_NULL = 0x01U,`
`DD_COMP_063_ERR_INVALID = 0x02U,`
`DD_COMP_063_ERR_TIMEOUT = 0x03U`
`} DD_COMP_063_ERR_t;`
	Error code enum for Invalid Command Handler

main.c - What to Add

Item	Purpose

`static InvalidCommandHandler_t badcmdstate = {0};`	Module-level zero-initialised state for Invalid Command Handler

`/* Process function logic order:`
`* Step 1: Receive unknown command input`
`* Step 2: Normalize and validate input`
`* Step 3: Generate user-friendly error message`
`* Step 4: Log invalid command details`
`* Step 5: Send error response to client`
`* Step 6: Return control to caller`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call init_badcmd() once in main() init sequence. Call handle_badcmd() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

66. DD-COMP-064 - Invalid Command Logger

DD-COMP-064	**Invalid Command Logger** | Service | Module: LoggingModule | Pattern: Repository | SW-REQ-053

`@trace DD-COMP-064 SW-REQ-053`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init_logger(void)`	void	Initialise all state for Invalid Command Logger
`main.c`	`static uint8_t log_invalid_cmd(void *invalid_cmd)`	uint8_t - 0 success, non-zero error code	Execute core logic for Invalid Command Logger
`main.c`	`static uint8_t open_logfile(void)`	uint8_t	Helper function: open logfile
`main.c`	`static uint8_t format_entry(void)`	uint8_t	Helper function: format entry
`main.c`	`static uint8_t write_entry(void)`	uint8_t	Helper function: write entry

Doxygen Comments

```
/**
* @brief Initialise Invalid Command Logger.
* @trace DD-COMP-064 SW-REQ-053
*/
/**
* @brief Run Invalid Command Logger process cycle.
* @param invalid_cmd Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-064 SW-REQ-053
*/
/**
* @brief Open logfile.
* @trace DD-COMP-064 SW-REQ-053
*/
/**
```

```
* @brief Format entry.
* @trace DD-COMP-064 SW-REQ-053
*/
/**
* @brief Write entry.
* @trace DD-COMP-064 SW-REQ-053
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load logger configuration
2	validate log destination
3	format invalid command details
4	append entry to log
5	handle write errors
6	confirm logging success

Forbidden - Must NOT Do

#	Forbidden Rule
1	logging sensitive data
2	blocking main thread
3	silent failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'iclog' (e.g. iclogresult).`

2	State struct: static InvalidCommandLogger_t iclogstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_064_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} InvalidCommandLogger_t;`
	State struct for Invalid Command Logger

`void init_logger(void);`	Prototype: initialise Invalid Command Logger
`uint8_t log_invalid_cmd(void *p_data);`	Prototype: run Invalid Command Logger process cycle
`uint8_t open_logfile(void);`	Prototype: open logfile
`uint8_t format_entry(void);`	Prototype: format entry
`uint8_t write_entry(void);`	Prototype: write entry

`typedef enum {`
`DD_COMP_064_ERR_NONE = 0x00U,`
`DD_COMP_064_ERR_NULL = 0x01U,`
`DD_COMP_064_ERR_INVALID = 0x02U,`
`DD_COMP_064_ERR_TIMEOUT = 0x03U`
`} DD_COMP_064_ERR_t;`
	Error code enum for Invalid Command Logger

main.c - What to Add

Item	Purpose

`static InvalidCommandLogger_t iclogstate = {0};`	Module-level zero-initialised state for Invalid Command Logger

`/* Process function logic order:`
`* Step 1: load logger configuration`
`* Step 2: validate log destination`
`* Step 3: format invalid command details`
`* Step 4: append entry to log`
`* Step 5: handle write errors`
`* Step 6: confirm logging success`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call init_logger() once in main() init sequence. Call log_invalid_cmd() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

67. DD-COMP-065 - Voltage Configuration Store

DD-COMP-065	**Voltage Configuration Store** | Database | Module: ConfigModule | Pattern: Singleton | SW-REQ-054

`@trace DD-COMP-065 SW-REQ-054`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void vconf_init(void)`	void	Initialise all state for Voltage Configuration Store
main.c	`static uint8_t vconf_process(void *config_key)`	uint8_t - 0 success, non-zero error code	Execute core logic for Voltage Configuration Store
main.c	`static uint8_t vconf_load(void)`	uint8_t	Helper function: vconf load
main.c	`static uint8_t vconf_save(void)`	uint8_t	Helper function: vconf save
main.c	`static uint8_t vconf_validate(void)`	uint8_t	Helper function: vconf validate

Doxygen Comments

```
/**
* @brief Initialise Voltage Configuration Store.
* @trace DD-COMP-065 SW-REQ-054
*/
/**
* @brief Run Voltage Configuration Store process cycle.
* @param config_key Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-065 SW-REQ-054
*/
/**
* @brief Vconf load.
* @trace DD-COMP-065 SW-REQ-054
*/
/**
```

```
* @brief Vconf save.
* @trace DD-COMP-065 SW-REQ-054
*/
/**
* @brief Vconf validate.
* @trace DD-COMP-065 SW-REQ-054
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize storage backend
2	load existing voltage configuration
3	validate configuration entries
4	apply updates from input
5	persist changes to storage
6	return operation status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	blocking I/O in main loop
3	silent failures
4	No dynamic struct allocation
5	Array sizes must be compile-time #define constants
6	No pointer arithmetic on struct members

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'vconf' (e.g. vconfresult).`
2	`State struct: static VoltageConfigStore_t vconfstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_065_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} VoltageConfigStore_t;`
	State struct for Voltage Configuration Store

`void vconf_init(void);`	Prototype: initialise Voltage Configuration Store
`uint8_t vconf_process(void *p_data);`	Prototype: run Voltage Configuration Store process cycle
`uint8_t vconf_load(void);`	Prototype: vconf load
`uint8_t vconf_save(void);`	Prototype: vconf save
`uint8_t vconf_validate(void);`	Prototype: vconf validate

`typedef enum {`
`DD_COMP_065_ERR_NONE = 0x00U,`
`DD_COMP_065_ERR_NULL = 0x01U,`
`DD_COMP_065_ERR_INVALID = 0x02U,`
`DD_COMP_065_ERR_TIMEOUT = 0x03U`
`} DD_COMP_065_ERR_t;`
	Error code enum for Voltage Configuration Store

main.c - What to Add

Item	Purpose
`static VoltageConfigStore_t vconfstate = {0};`	Module-level zero-initialised state for Voltage Configuration Store

`/* Process function logic order:`
`* Step 1: initialize storage backend`
`* Step 2: load existing voltage configuration`
`* Step 3: validate configuration entries`
`* Step 4: apply updates from input`
`* Step 5: persist changes to storage`
`* Step 6: return operation status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call vconf_init() once in main() init sequence. Call vconf_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

68. DD-COMP-066 - Voltage Range Configuration

DD-COMP-066	**Voltage Range Configuration** | Class | Module: ConfigModule | Pattern: N/A | SW-REQ-055

`@trace DD-COMP-066 SW-REQ-055`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initRangeCfg(void)`	void	Initialise all state for Voltage Range Configuration
main.c	`static uint8_t applyRangeCfg(void *requested_min)`	uint8_t - 0 success, non-zero error code	Execute core logic for Voltage Range Configuration
main.c	`static uint8_t loadDefaults(void)`	uint8_t	Helper function: loadDefaults
main.c	`static uint8_t validateRange(void)`	uint8_t	Helper function: validateRange
main.c	`static uint8_t commitRange(void)`	uint8_t	Helper function: commitRange

Doxygen Comments

```
/**
* @brief Initialise Voltage Range Configuration.
* @trace DD-COMP-066 SW-REQ-055
*/
/**
* @brief Run Voltage Range Configuration process cycle.
* @param requested_min Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-066 SW-REQ-055
*/
/**
* @brief Loaddefaults.
* @trace DD-COMP-066 SW-REQ-055
*/
/**
* @brief Validaterange.
```

```
 * @trace DD-COMP-066 SW-REQ-055
 */
 /**
 * @brief Commitrange.
 * @trace DD-COMP-066 SW-REQ-055
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load default voltage limits
2	read requested range settings
3	validate min/max values
4	clamp to allowed boundaries
5	apply range to hardware interface
6	store current configuration

Forbidden - Must NOT Do

#	Forbidden Rule
1	hard-coded limits
2	silent invalid ranges
3	blocking hardware calls

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'vr' (e.g. vrresult).`
2	`State struct: static VoltageRangeConfig_t vrstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_066_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} VoltageRangeConfig_t;`
	State struct for Voltage Range Configuration

`void initRangeCfg(void);`	Prototype: initialise Voltage Range Configuration
`uint8_t applyRangeCfg(void *p_data);`	Prototype: run Voltage Range Configuration process cycle
`uint8_t loadDefaults(void);`	Prototype: loadDefaults
`uint8_t validateRange(void);`	Prototype: validateRange
`uint8_t commitRange(void);`	Prototype: commitRange

`typedef enum {`
`DD_COMP_066_ERR_NONE = 0x00U,`
`DD_COMP_066_ERR_NULL = 0x01U,`
`DD_COMP_066_ERR_INVALID = 0x02U,`
`DD_COMP_066_ERR_TIMEOUT = 0x03U`
`} DD_COMP_066_ERR_t;`
	Error code enum for Voltage Range Configuration

main.c - What to Add

Item	Purpose
`static VoltageRangeConfig_t vrstate = {0};`	Module-level zero-initialised state for Voltage Range Configuration

`/* Process function logic order:`
`* Step 1: load default voltage limits`
`* Step 2: read requested range settings`
`* Step 3: validate min/max values`
`* Step 4: clamp to allowed boundaries`
`* Step 5: apply range to hardware interface`
`* Step 6: store current configuration`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call initRangeCfg() once in main() init sequence. Call applyRangeCfg() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

69. DD-COMP-067 - Startup Safety Manager

DD-COMP-067	**Startup Safety Manager** | Service | Module: StartupModule | Pattern: State | SW-REQ-056

`@trace DD-COMP-067 SW-REQ-056`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void ssm_setup(void)`	void	Initialise all state for Startup Safety Manager
`main.c`	`static uint8_t ssm_run(void *config_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for Startup Safety Manager
`main.c`	`static uint8_t load_policies(void)`	uint8_t	Helper function: load policies
`main.c`	`static uint8_t check_readiness(void)`	uint8_t	Helper function: check readiness
`main.c`	`static uint8_t log_event(void)`	uint8_t	Helper function: log event

Doxygen Comments

```
/**
* @brief Initialise Startup Safety Manager.
* @trace DD-COMP-067 SW-REQ-056
*/
/**
* @brief Run Startup Safety Manager process cycle.
* @param config_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-067 SW-REQ-056
*/
/**
* @brief Load policies.
* @trace DD-COMP-067 SW-REQ-056
*/
/**
* @brief Check readiness.
```

```
 * @trace DD-COMP-067 SW-REQ-056
 */
 /**
 * @brief Log event.
 * @trace DD-COMP-067 SW-REQ-056
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load safety configuration
2	verify system prerequisites
3	evaluate startup conditions
4	apply safety policies
5	emit startup status
6	record audit log

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded safety rules
2	blocking waits without timeouts
3	silent failure handling
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ssm' (e.g. ssmresult).`

2	State struct: static StartupSafetyManager_t ssmstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_067_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} StartupSafetyManager_t;`
	State struct for Startup Safety Manager

`void ssm_setup(void);`	Prototype: initialise Startup Safety Manager
`uint8_t ssm_run(void *p_data);`	Prototype: run Startup Safety Manager process cycle
`uint8_t load_policies(void);`	Prototype: load policies
`uint8_t check_readiness(void);`	Prototype: check readiness
`uint8_t log_event(void);`	Prototype: log event

`typedef enum {`
`DD_COMP_067_ERR_NONE = 0x00U,`
`DD_COMP_067_ERR_NULL = 0x01U,`
`DD_COMP_067_ERR_INVALID = 0x02U,`
`DD_COMP_067_ERR_TIMEOUT = 0x03U`
`} DD_COMP_067_ERR_t;`
	Error code enum for Startup Safety Manager

main.c - What to Add

Item	Purpose

`static StartupSafetyManager_t ssmstate = {0};`	Module-level zero-initialised state for Startup Safety Manager

`/* Process function logic order:`
`* Step 1: load safety configuration`
`* Step 2: verify system prerequisites`
`* Step 3: evaluate startup conditions`
`* Step 4: apply safety policies`
`* Step 5: emit startup status`
`* Step 6: record audit log`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call ssm_setup() once in main() init sequence. Call ssm_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

70. DD-COMP-068 - StartupDelayManager

DD-COMP-068	**StartupDelayManager** | Class | Module: StartupModule | Pattern: State | SW-REQ-057

`@trace DD-COMP-068 SW-REQ-057`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initDelay(void)`	void	Initialise all state for StartupDelayManager
main.c	`static uint8_t runDelay(void *config)`	uint8_t - 0 success, non-zero error code	Execute core logic for StartupDelayManager
main.c	`static uint8_t calcDelay(void)`	uint8_t	Helper function: calcDelay
main.c	`static uint8_t checkReady(void)`	uint8_t	Helper function: checkReady
main.c	`static uint8_t resetTimer(void)`	uint8_t	Helper function: resetTimer

Doxygen Comments

```
/**
* @brief Initialise StartupDelayManager.
* @trace DD-COMP-068 SW-REQ-057
*/
/**
* @brief Run StartupDelayManager process cycle.
* @param config Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-068 SW-REQ-057
*/
/**
* @brief Calcdelay.
* @trace DD-COMP-068 SW-REQ-057
*/
/**
* @brief Checkready.
```

```
 * @trace DD-COMP-068 SW-REQ-057
 */
 /**
 * @brief Resettimer.
 * @trace DD-COMP-068 SW-REQ-057
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	initialize timer
3	monitor startup status
4	apply delay window
5	verify readiness
6	signal completion

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	global state coupling
3	hardcoded delays

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'sdm' (e.g. sdmresult).`
2	`State struct: static StartupDelayManager_t sdmstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_068_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} StartupDelayManager_t;`
	State struct for StartupDelayManager

`void initDelay(void);`	Prototype: initialise StartupDelayManager
`uint8_t runDelay(void *p_data);`	Prototype: run StartupDelayManager process cycle
`uint8_t calcDelay(void);`	Prototype: calcDelay
`uint8_t checkReady(void);`	Prototype: checkReady
`uint8_t resetTimer(void);`	Prototype: resetTimer

`typedef enum {`
`DD_COMP_068_ERR_NONE = 0x00U,`
`DD_COMP_068_ERR_NULL = 0x01U,`
`DD_COMP_068_ERR_INVALID = 0x02U,`
`DD_COMP_068_ERR_TIMEOUT = 0x03U`
`} DD_COMP_068_ERR_t;`
	Error code enum for StartupDelayManager

main.c - What to Add

Item	Purpose
`static StartupDelayManager_t sdmstate = {0};`	Module-level zero-initialised state for StartupDelayManager

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: initialize timer`
`* Step 3: monitor startup status`
`* Step 4: apply delay window`
`* Step 5: verify readiness`
`* Step 6: signal completion`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initDelay() once in main() init sequence. Call runDelay() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

71. DD-COMP-069 - ADCResolutionConfigurator

DD-COMP-069 **ADCResolutionConfigurator** | Service | Module: ADCModule | Pattern: N/A | SW-REQ-058

`@trace DD-COMP-069 SW-REQ-058`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void adcResCfgInit(void)`	void	Initialise all state for ADCResolutionConfigurator
main.c	`static uint8_t adcResCfgRun(void *requestedResolution)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCResolutionConfigurator
main.c	`static uint8_t validateRes(void)`	uint8_t	Helper function: validateRes
main.c	`static uint8_t applyRes(void)`	uint8_t	Helper function: applyRes
main.c	`static uint8_t cacheRes(void)`	uint8_t	Helper function: cacheRes

Doxygen Comments

```
/**
* @brief Initialise ADCResolutionConfigurator.
* @trace DD-COMP-069 SW-REQ-058
*/
/**
* @brief Run ADCResolutionConfigurator process cycle.
* @param requestedResolution Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-069 SW-REQ-058
*/
/**
* @brief Validateres.
* @trace DD-COMP-069 SW-REQ-058
*/
/**
* @brief Applyres.
```

```
 * @trace DD-COMP-069 SW-REQ-058
 */
 /**
 * @brief Cacheres.
 * @trace DD-COMP-069 SW-REQ-058
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load requested resolution
2	validate against supported modes
3	compare with current setting
4	apply new resolution to ADC
5	update cached state
6	report status

Forbidden - Must NOT Do

#	Forbidden Rule
1	changing resolution during active conversion
2	ignoring hardware limits
3	blocking wait loops
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcRes' (e.g. adcResresult).`

2	State struct: static ADCResolutionConfigurator_t adcResstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_069_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ADCResolutionConfigurator_t;`
	State struct for ADCResolutionConfigurator

`void adcResCfgInit(void);`	Prototype: initialise ADCResolutionConfigurator
`uint8_t adcResCfgRun(void *p_data);`	Prototype: run ADCResolutionConfigurator process cycle
`uint8_t validateRes(void);`	Prototype: validateRes
`uint8_t applyRes(void);`	Prototype: applyRes
`uint8_t cacheRes(void);`	Prototype: cacheRes

`typedef enum {`
`DD_COMP_069_ERR_NONE = 0x00U,`
`DD_COMP_069_ERR_NULL = 0x01U,`
`DD_COMP_069_ERR_INVALID = 0x02U,`
`DD_COMP_069_ERR_TIMEOUT = 0x03U`
`} DD_COMP_069_ERR_t;`
	Error code enum for ADCResolutionConfigurator

main.c - What to Add

Item	Purpose

`static ADCResolutionConfigurator_t adcResstate = {0};`	Module-level zero-initialised state for ADCResolutionConfigurator

`/* Process function logic order:`
`* Step 1: load requested resolution`
`* Step 2: validate against supported modes`
`* Step 3: compare with current setting`
`* Step 4: apply new resolution to ADC`
`* Step 5: update cached state`
`* Step 6: report status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call adcResCfgInit() once in main() init sequence. Call adcResCfgRun() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

72. DD-COMP-070 - ADCReferenceConfigurator

DD-COMP-070	**ADCReferenceConfigurator** | Service | Module: ADCModule | Pattern: N/A | SW-REQ-059

`@trace DD-COMP-070 SW-REQ-059`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void initAdcRef(void)`	void	Initialise all state for ADCReferenceConfigurator
`main.c`	`static uint8_t processAdcRef(void *ref_mode)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCReferenceConfigurator
`main.c`	`static uint8_t loadRefConfig(void)`	uint8_t	Helper function: loadRefConfig
`main.c`	`static uint8_t applyRefSettings(void)`	uint8_t	Helper function: applyRefSettings
`main.c`	`static uint8_t validateRefMode(void)`	uint8_t	Helper function: validateRefMode

Doxygen Comments

```
/**
* @brief Initialise ADCReferenceConfigurator.
* @trace DD-COMP-070 SW-REQ-059
*/
/**
* @brief Run ADCReferenceConfigurator process cycle.
* @param ref_mode Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-070 SW-REQ-059
*/
/**
* @brief Loadrefconfig.
* @trace DD-COMP-070 SW-REQ-059
*/
/**
* @brief Applyrefsettings.
```

```
 * @trace DD-COMP-070 SW-REQ-059
 */
 /**
 * @brief Validaterefmode.
 * @trace DD-COMP-070 SW-REQ-059
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read reference configuration inputs
2	validate selected reference mode
3	apply reference voltage settings
4	update ADC hardware registers
5	verify reference stabilization
6	report configuration status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays during setup
2	hardcoded reference values
3	writing registers without validation
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcRef' (e.g. adcRefresult).`

2	State struct: static AdcReferenceConfig_t adcRefstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_070_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcReferenceConfig_t;`
	State struct for ADCReferenceConfigurator

`void initAdcRef(void);`	Prototype: initialise ADCReferenceConfigurator
`uint8_t processAdcRef(void *p_data);`	Prototype: run ADCReferenceConfigurator process cycle
`uint8_t loadRefConfig(void);`	Prototype: loadRefConfig
`uint8_t applyRefSettings(void);`	Prototype: applyRefSettings
`uint8_t validateRefMode(void);`	Prototype: validateRefMode

`typedef enum {`
`DD_COMP_070_ERR_NONE = 0x00U,`
`DD_COMP_070_ERR_NULL = 0x01U,`
`DD_COMP_070_ERR_INVALID = 0x02U,`
`DD_COMP_070_ERR_TIMEOUT = 0x03U`
`} DD_COMP_070_ERR_t;`
	Error code enum for ADCReferenceConfigurator

main.c - What to Add

Item	Purpose

`static AdcReferenceConfig_t adcRefstate = {0};`	Module-level zero-initialised state for ADCReferenceConfigurator

`/* Process function logic order:`
`* Step 1: read reference configuration inputs`
`* Step 2: validate selected reference mode`
`* Step 3: apply reference voltage settings`
`* Step 4: update ADC hardware registers`
`* Step 5: verify reference stabilization`
`* Step 6: report configuration status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call initAdcRef() once in main() init sequence. Call processAdcRef() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

73. DD-COMP-071 - ADCSamplingScheduler

DD-COMP-071	**ADCSamplingScheduler** | Service | Module: ADCModule | Pattern: Observer | SW-REQ-060

`@trace DD-COMP-071 SW-REQ-060`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void adcSchedInit(void)`	void	Initialise all state for ADCSamplingScheduler
main.c	`static uint8_t adcSchedRun(void *tick)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCSamplingScheduler
main.c	`static uint8_t loadSchedule(void)`	uint8_t	Helper function: loadSchedule
main.c	`static uint8_t tickUpdate(void)`	uint8_t	Helper function: tickUpdate
main.c	`static uint8_t triggerSample(void)`	uint8_t	Helper function: triggerSample

Doxygen Comments

```
/**
* @brief Initialise ADCSamplingScheduler.
* @trace DD-COMP-071 SW-REQ-060
*/
/**
* @brief Run ADCSamplingScheduler process cycle.
* @param tick Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-071 SW-REQ-060
*/
/**
* @brief Loadschedule.
* @trace DD-COMP-071 SW-REQ-060
*/
/**
* @brief Tickupdate.
```

```
 * @trace DD-COMP-071 SW-REQ-060
 */
 /**
 * @brief Triggersample.
 * @trace DD-COMP-071 SW-REQ-060
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize scheduler state
2	load or update sampling schedule
3	advance time or tick counter
4	check if sampling due
5	trigger ADC sampling event
6	record next scheduled time

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	hard-coded timing constants
3	missing enable checks
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcSch' (e.g. adcSchresult).`

2	State struct: static AdcSamplingScheduler_t adcSchstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_071_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcSamplingScheduler_t;`
	State struct for ADCSamplingScheduler

`void adcSchedInit(void);`	Prototype: initialise ADCSamplingScheduler
`uint8_t adcSchedRun(void *p_data);`	Prototype: run ADCSamplingScheduler process cycle
`uint8_t loadSchedule(void);`	Prototype: loadSchedule
`uint8_t tickUpdate(void);`	Prototype: tickUpdate
`uint8_t triggerSample(void);`	Prototype: triggerSample

`typedef enum {`
`DD_COMP_071_ERR_NONE = 0x00U,`
`DD_COMP_071_ERR_NULL = 0x01U,`
`DD_COMP_071_ERR_INVALID = 0x02U,`
`DD_COMP_071_ERR_TIMEOUT = 0x03U`
`} DD_COMP_071_ERR_t;`
	Error code enum for ADCSamplingScheduler

main.c - What to Add

Item	Purpose

`static AdcSamplingScheduler_t adcSchstate = {0};`	Module-level zero-initialised state for ADCSamplingScheduler

`/* Process function logic order:`
`* Step 1: initialize scheduler state`
`* Step 2: load or update sampling schedule`
`* Step 3: advance time or tick counter`
`* Step 4: check if sampling due`
`* Step 5: trigger ADC sampling event`
`* Step 6: record next scheduled time`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call adcSchedInit() once in main() init sequence. Call adcSchedRun() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

74. DD-COMP-072 - ADCSaturationDetector

DD-COMP-072 **ADCSaturationDetector** | Algorithm | Module: ADCModule | Pattern: State | SW-REQ-061

`@trace DD-COMP-072 SW-REQ-061`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void satdet_init(void)`	void	Initialise all state for ADCSaturationDetector
main.c	`static uint8_t satdet_process(void *adc_samples)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCSaturationDetector
main.c	`static uint8_t check_limits(void)`	uint8_t	Helper function: check limits
main.c	`static uint8_t update_state(void)`	uint8_t	Helper function: update state
main.c	`static uint8_t flag_event(void)`	uint8_t	Helper function: flag event

Doxygen Comments

```
/**
* @brief Initialise ADCSaturationDetector.
* @trace DD-COMP-072 SW-REQ-061
*/
/**
* @brief Run ADCSaturationDetector process cycle.
* @param adc_samples Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-072 SW-REQ-061
*/
/**
* @brief Check limits.
* @trace DD-COMP-072 SW-REQ-061
*/
/**
* @brief Update state.
```

```
 * @trace DD-COMP-072 SW-REQ-061
 */
 /**
 * @brief Flag event.
 * @trace DD-COMP-072 SW-REQ-061
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load config thresholds
2	read latest ADC samples
3	compare samples to limits
4	update saturation counters
5	set saturation flags
6	store status output

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded limits
2	blocking operations
3	unused globals
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'sat' (e.g. satresult).`

2	State struct: static AdcSaturationDetector_t satstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_072_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcSaturationDetector_t;`
	State struct for ADCSaturationDetector

`void satdet_init(void);`	Prototype: initialise ADCSaturationDetector
`uint8_t satdet_process(void *p_data);`	Prototype: run ADCSaturationDetector process cycle
`uint8_t check_limits(void);`	Prototype: check limits
`uint8_t update_state(void);`	Prototype: update state
`uint8_t flag_event(void);`	Prototype: flag event

`typedef enum {`
`DD_COMP_072_ERR_NONE = 0x00U,`
`DD_COMP_072_ERR_NULL = 0x01U,`
`DD_COMP_072_ERR_INVALID = 0x02U,`
`DD_COMP_072_ERR_TIMEOUT = 0x03U`
`} DD_COMP_072_ERR_t;`
	Error code enum for ADCSaturationDetector

main.c - What to Add

Item	Purpose

`static AdcSaturationDetector_t satstate = {0};`	Module-level zero-initialised state for ADCSaturationDetector

`/* Process function logic order:`
`* Step 1: load config thresholds`
`* Step 2: read latest ADC samples`
`* Step 3: compare samples to limits`
`* Step 4: update saturation counters`
`* Step 5: set saturation flags`
`* Step 6: store status output`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call satdet_init() once in main() init sequence. Call satdet_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

75. DD-COMP-073 - ADCPinInitializer

DD-COMP-073	**ADCPinInitializer** | Service | Module: BoardSupportModule | Pattern: N/A | SW-REQ-062

`@trace DD-COMP-073 SW-REQ-062`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initADCPins(void)`	void	Initialise all state for ADCPinInitializer
main.c	`static uint8_t processADCPins(void *pinConfig)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCPinInitializer
main.c	`static uint8_t cfgPin(void)`	uint8_t	Helper function: cfgPin
main.c	`static uint8_t setMode(void)`	uint8_t	Helper function: setMode
main.c	`static uint8_t verifyPin(void)`	uint8_t	Helper function: verifyPin

Doxygen Comments

```
/**
* @brief Initialise ADCPinInitializer.
* @trace DD-COMP-073 SW-REQ-062
*/
/**
* @brief Run ADCPinInitializer process cycle.
* @param pinConfig Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-073 SW-REQ-062
*/
/**
* @brief Cfgpin.
* @trace DD-COMP-073 SW-REQ-062
*/
/**
* @brief Setmode.
```

```
 * @trace DD-COMP-073 SW-REQ-062
 */
 /**
 * @brief Verifypin.
 * @trace DD-COMP-073 SW-REQ-062
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load pin configuration
2	validate pin numbers
3	configure pin mode
4	set default states
5	apply ADC settings
6	verify initialization status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded pin numbers
2	blocking delays
3	global state leakage
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adcPin' (e.g. adcPinresult).`

2	State struct: static ADCPinInitializer_t adcPinstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_073_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ADCPinInitializer_t;`
	State struct for ADCPinInitializer

`void initADCPins(void);`	Prototype: initialise ADCPinInitializer
`uint8_t processADCPins(void *p_data);`	Prototype: run ADCPinInitializer process cycle
`uint8_t cfgPin(void);`	Prototype: cfgPin
`uint8_t setMode(void);`	Prototype: setMode
`uint8_t verifyPin(void);`	Prototype: verifyPin

`typedef enum {`
`DD_COMP_073_ERR_NONE = 0x00U,`
`DD_COMP_073_ERR_NULL = 0x01U,`
`DD_COMP_073_ERR_INVALID = 0x02U,`
`DD_COMP_073_ERR_TIMEOUT = 0x03U`
`} DD_COMP_073_ERR_t;`
	Error code enum for ADCPinInitializer

main.c - What to Add

Item	Purpose

`static ADCPinInitializer_t adcPinstate = {0};`	Module-level zero-initialised state for ADCPinInitializer

`/* Process function logic order:`
`* Step 1: load pin configuration`
`* Step 2: validate pin numbers`
`* Step 3: configure pin mode`
`* Step 4: set default states`
`* Step 5: apply ADC settings`
`* Step 6: verify initialization status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call initADCPins() once in main() init sequence. Call processADCPins() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

76. DD-COMP-074 - ActuatorGPIOInitializer

DD-COMP-074 **ActuatorGPIOInitializer** | Service | Module: BoardSupportModule | Pattern: N/A | SW-REQ-063

`@trace DD-COMP-074 SW-REQ-063`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_gpio(void)`	void	Initialise all state for ActuatorGPIOInitializer
main.c	`static uint8_t process_gpio(void *config_table)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorGPIOInitializer
main.c	`static uint8_t cfg_pins(void)`	uint8_t	Helper function: cfg pins
main.c	`static uint8_t set_modes(void)`	uint8_t	Helper function: set modes
main.c	`static uint8_t apply_defaults(void)`	uint8_t	Helper function: apply defaults

Doxygen Comments

```
/**
* @brief Initialise ActuatorGPIOInitializer.
* @trace DD-COMP-074 SW-REQ-063
*/
/**
* @brief Run ActuatorGPIOInitializer process cycle.
* @param config_table Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-074 SW-REQ-063
*/
/**
* @brief Cfg pins.
* @trace DD-COMP-074 SW-REQ-063
*/
/**
* @brief Set modes.
```

```
 * @trace DD-COMP-074 SW-REQ-063
 */
 /**
 * @brief Apply defaults.
 * @trace DD-COMP-074 SW-REQ-063
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load actuator GPIO configuration
2	validate pin assignments
3	configure pin modes
4	set initial output states
5	enable GPIO peripherals
6	report initialization status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoding pin numbers
2	blocking delays in init
3	reconfiguring active pins
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'gpio' (e.g. gpioresult).`

2	State struct: static ActuatorGPIOInitializer_t gpiostate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_074_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorGPIOInitializer_t;`
	State struct for ActuatorGPIOInitializer

`void init_gpio(void);`	Prototype: initialise ActuatorGPIOInitializer
`uint8_t process_gpio(void *p_data);`	Prototype: run ActuatorGPIOInitializer process cycle
`uint8_t cfg_pins(void);`	Prototype: cfg pins
`uint8_t set_modes(void);`	Prototype: set modes
`uint8_t apply_defaults(void);`	Prototype: apply defaults

`typedef enum {`
`DD_COMP_074_ERR_NONE = 0x00U,`
`DD_COMP_074_ERR_NULL = 0x01U,`
`DD_COMP_074_ERR_INVALID = 0x02U,`
`DD_COMP_074_ERR_TIMEOUT = 0x03U`
`} DD_COMP_074_ERR_t;`
	Error code enum for ActuatorGPIOInitializer

main.c - What to Add

Item	Purpose

`static ActuatorGPIOInitializer_t gpiostate = {0};`	Module-level zero-initialised state for ActuatorGPIOInitializer

`/* Process function logic order:`
`* Step 1: load actuator GPIO configuration`
`* Step 2: validate pin assignments`
`* Step 3: configure pin modes`
`* Step 4: set initial output states`
`* Step 5: enable GPIO peripherals`
`* Step 6: report initialization status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call init_gpio() once in main() init sequence. Call process_gpio() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

77. DD-COMP-075 - PWMTimerInitializer

DD-COMP-075 **PWMTimerInitializer** | Service | Module: ActuatorControlModule | Pattern: Factory | SW-REQ-064

`@trace DD-COMP-075 SW-REQ-064`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void pwmInit(void)`	void	Initialise all state for PWMTimerInitializer
main.c	`static uint8_t pwmStart(void *timerId)`	uint8_t - 0 success, non-zero error code	Execute core logic for PWMTimerInitializer
main.c	`static uint8_t cfgTimer(void)`	uint8_t	Helper function: cfgTimer
main.c	`static uint8_t cfgChannel(void)`	uint8_t	Helper function: cfgChannel
main.c	`static uint8_t setDuty(void)`	uint8_t	Helper function: setDuty

Doxygen Comments

```
/**
* @brief Initialise PWMTimerInitializer.
* @trace DD-COMP-075 SW-REQ-064
*/
/**
* @brief Run PWMTimerInitializer process cycle.
* @param timerId Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-075 SW-REQ-064
*/
/**
* @brief Cfgtimer.
* @trace DD-COMP-075 SW-REQ-064
*/
/**
* @brief Cfgchannel.
```

```
 * @trace DD-COMP-075 SW-REQ-064
 */
 /**
 * @brief Setduty.
 * @trace DD-COMP-075 SW-REQ-064
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Validate input parameters
2	Enable timer clock
3	Configure timer base settings
4	Configure PWM channel settings
5	Set initial duty cycle
6	Start timer output

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded registers
2	blocking delays
3	unused channels
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'pwm' (e.g. pwmresult).`

2	State struct: static PwmTimerInit_t pwmstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_075_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} PwmTimerInit_t;`
	State struct for PWMTimerInitializer

`void pwmInit(void);`	Prototype: initialise PWMTimerInitializer
`uint8_t pwmStart(void *p_data);`	Prototype: run PWMTimerInitializer process cycle
`uint8_t cfgTimer(void);`	Prototype: cfgTimer
`uint8_t cfgChannel(void);`	Prototype: cfgChannel
`uint8_t setDuty(void);`	Prototype: setDuty

`typedef enum {`
`DD_COMP_075_ERR_NONE = 0x00U,`
`DD_COMP_075_ERR_NULL = 0x01U,`
`DD_COMP_075_ERR_INVALID = 0x02U,`
`DD_COMP_075_ERR_TIMEOUT = 0x03U`
`} DD_COMP_075_ERR_t;`
	Error code enum for PWMTimerInitializer

main.c - What to Add

Item	Purpose

`static PwmTimerInit_t pwmstate = {0};`	Module-level zero-initialised state for PWMTimerInitializer

`/* Process function logic order:`
`* Step 1: Validate input parameters`
`* Step 2: Enable timer clock`
`* Step 3: Configure timer base settings`
`* Step 4: Configure PWM channel settings`
`* Step 5: Set initial duty cycle`
`* Step 6: Start timer output`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call pwmInit() once in main() init sequence. Call pwmStart() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

78. DD-COMP-076 - UART Peripheral Initializer

DD-COMP-076	**UART Peripheral Initializer** | Class | Module: HAL | Pattern: Factory | SW-REQ-065

`@trace DD-COMP-076 SW-REQ-065`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void uart_init(void)`	void	Initialise all state for UART Peripheral Initializer
`main.c`	`static uint8_t uart_run(void *cfg)`	uint8_t - 0 success, non-zero error code	Execute core logic for UART Peripheral Initializer
`main.c`	`static uint8_t uart_config(void)`	uint8_t	Helper function: uart config
`main.c`	`static uint8_t uart_enable(void)`	uint8_t	Helper function: uart enable
`main.c`	`static uint8_t uart_set_baud(void)`	uint8_t	Helper function: uart set baud

Doxygen Comments

```
/**
* @brief Initialise UART Peripheral Initializer.
* @trace DD-COMP-076 SW-REQ-065
*/
/**
* @brief Run UART Peripheral Initializer process cycle.
* @param cfg Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-076 SW-REQ-065
*/
/**
* @brief Uart config.
* @trace DD-COMP-076 SW-REQ-065
*/
/**
* @brief Uart enable.
```

```
 * @trace DD-COMP-076 SW-REQ-065
 */
 /**
 * @brief Uart set baud.
 * @trace DD-COMP-076 SW-REQ-065
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load default UART settings
2	apply baud rate and frame config
3	configure GPIO pins for UART
4	enable UART clocks and peripheral
5	verify UART status
6	store initialized state

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded pin mappings
2	blocking delays in init
3	unvalidated baud rates

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uart' (e.g. uartresult).`
2	`State struct: static UartInit_t uartstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_076_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartInit_t;`
	State struct for UART Peripheral Initializer

`void uart_init(void);`	Prototype: initialise UART Peripheral Initializer
`uint8_t uart_run(void *p_data);`	Prototype: run UART Peripheral Initializer process cycle
`uint8_t uart_config(void);`	Prototype: uart config
`uint8_t uart_enable(void);`	Prototype: uart enable
`uint8_t uart_set_baud(void);`	Prototype: uart set baud

`typedef enum {`
`DD_COMP_076_ERR_NONE = 0x00U,`
`DD_COMP_076_ERR_NULL = 0x01U,`
`DD_COMP_076_ERR_INVALID = 0x02U,`
`DD_COMP_076_ERR_TIMEOUT = 0x03U`
`} DD_COMP_076_ERR_t;`
	Error code enum for UART Peripheral Initializer

main.c - What to Add

Item	Purpose
`static UartInit_t uartstate = {0};`	Module-level zero-initialised state for UART Peripheral Initializer

`/* Process function logic order:`
`* Step 1: load default UART settings`
`* Step 2: apply baud rate and frame config`
`* Step 3: configure GPIO pins for UART`
`* Step 4: enable UART clocks and peripheral`
`* Step 5: verify UART status`
`* Step 6: store initialized state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call uart_init() once in main() init sequence. Call uart_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

79. DD-COMP-077 - ADC DMA Configuration Service

DD-COMP-077	**ADC DMA Configuration Service** | Service | Module: Driver | Pattern: Facade | SW-REQ-066

`@trace DD-COMP-077 SW-REQ-066`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void AdcDmaCfg_Init(void)`	void	Initialise all state for ADC DMA Configuration Service
main.c	`static uint8_t AdcDmaCfg_Run(void *adcConfig)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADC DMA Configuration Service
main.c	`static uint8_t SetupAdc(void)`	uint8_t	Helper function: SetupAdc
main.c	`static uint8_t SetupDma(void)`	uint8_t	Helper function: SetupDma
main.c	`static uint8_t LinkAdcDma(void)`	uint8_t	Helper function: LinkAdcDma

Doxygen Comments

```
/**
* @brief Initialise ADC DMA Configuration Service.
* @trace DD-COMP-077 SW-REQ-066
*/
/**
* @brief Run ADC DMA Configuration Service process cycle.
* @param adcConfig Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-077 SW-REQ-066
*/
/**
* @brief Setupadc.
* @trace DD-COMP-077 SW-REQ-066
*/
/**
* @brief Setupdma.
```

```
 * @trace DD-COMP-077 SW-REQ-066
 */
 /**
 * @brief Linkadcdma.
 * @trace DD-COMP-077 SW-REQ-066
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Validate configuration parameters
2	Initialize ADC hardware
3	Configure DMA channel
4	Link ADC to DMA trigger
5	Start calibration if needed
6	Enable ADC and DMA
7	Report configuration status

Forbidden - Must NOT Do

#	Forbidden Rule
1	reconfiguring while running
2	null handle usage
3	ignoring error flags
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'adcCfg' (e.g. adcCfgresult).
2	State struct: static AdcDmaConfigService_t adcCfgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_077_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcDmaConfigService_t;`
	State struct for ADC DMA Configuration Service

`void AdcDmaCfg_Init(void);`	Prototype: initialise ADC DMA Configuration Service
`uint8_t AdcDmaCfg_Run(void *p_data);`	Prototype: run ADC DMA Configuration Service process cycle
`uint8_t SetupAdc(void);`	Prototype: SetupAdc
`uint8_t SetupDma(void);`	Prototype: SetupDma
`uint8_t LinkAdcDma(void);`	Prototype: LinkAdcDma

`typedef enum {`
`DD_COMP_077_ERR_NONE = 0x00U,`
`DD_COMP_077_ERR_NULL = 0x01U,`
`DD_COMP_077_ERR_INVALID = 0x02U,`
`DD_COMP_077_ERR_TIMEOUT = 0x03U`
`} DD_COMP_077_ERR_t;`
	Error code enum for ADC DMA Configuration Service

main.c - What to Add

Item	Purpose
`static AdcDmaConfigService_t adcCfgstate = {0};`	Module-level zero-initialised state for ADC DMA Configuration Service

`/* Process function logic order:`
`* Step 1: Validate configuration parameters`
`* Step 2: Initialize ADC hardware`
`* Step 3: Configure DMA channel`
`* Step 4: Link ADC to DMA trigger`
`* Step 5: Start calibration if needed`
`* Step 6: Enable ADC and DMA`
`* Step 7: Report configuration status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call AdcDmaCfg_Init() once in main() init sequence. Call AdcDmaCfg_Run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

80. DD-COMP-078 - Independent Watchdog Service

DD-COMP-078	**Independent Watchdog Service** | Service | Module: Middleware | Pattern: Singleton | SW-REQ-067

`@trace DD-COMP-078 SW-REQ-067`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void wdg_init(void)`	void	Initialise all state for Independent Watchdog Service
main.c	`static uint8_t wdg_process(void *heartbeat_signal)`	uint8_t - 0 success, non-zero error code	Execute core logic for Independent Watchdog Service
main.c	`static uint8_t wdg_kick(void)`	uint8_t	Helper function: wdg kick
main.c	`static uint8_t wdg_check_timeout(void)`	uint8_t	Helper function: wdg check timeout
main.c	`static uint8_t wdg_reset_trigger(void)`	uint8_t	Helper function: wdg reset trigger

Doxygen Comments

```
/**
* @brief Initialise Independent Watchdog Service.
* @trace DD-COMP-078 SW-REQ-067
*/
/**
* @brief Run Independent Watchdog Service process cycle.
* @param heartbeat_signal Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-078 SW-REQ-067
*/
/**
* @brief Wdg kick.
* @trace DD-COMP-078 SW-REQ-067
*/
/**
```

```
 * @brief Wdg check timeout.
 * @trace DD-COMP-078 SW-REQ-067
 */
 /**
 * @brief Wdg reset trigger.
 * @trace DD-COMP-078 SW-REQ-067
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	initialize timer and state
3	monitor heartbeat inputs
4	evaluate timeout conditions
5	trigger reset action if expired
6	log status
7	update next check deadline

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	unchecked timer overflow
3	spurious resets
4	shared state without locking
5	No malloc/calloc/free anywhere in this component
6	No recursive function calls - use iterative logic only
7	No processing inside ISR - set flag and return only
8	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	All local variables prefixed 'wdg' (e.g. wdgresult).
2	State struct: static WatchdogService_t wdgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_078_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} WatchdogService_t;`
	State struct for Independent Watchdog Service

`void wdg_init(void);`	Prototype: initialise Independent Watchdog Service
`uint8_t wdg_process(void *p_data);`	Prototype: run Independent Watchdog Service process cycle
`uint8_t wdg_kick(void);`	Prototype: wdg kick
`uint8_t wdg_check_timeout(void);`	Prototype: wdg check timeout
`uint8_t wdg_reset_trigger(void);`	Prototype: wdg reset trigger

`typedef enum {`
`DD_COMP_078_ERR_NONE = 0x00U,`
`DD_COMP_078_ERR_NULL = 0x01U,`
`DD_COMP_078_ERR_INVALID = 0x02U,`
`DD_COMP_078_ERR_TIMEOUT = 0x03U`
`} DD_COMP_078_ERR_t;`
	Error code enum for Independent Watchdog Service

main.c - What to Add

Item	Purpose
`static WatchdogService_t wdgstate = {0};`	Module-level zero-initialised state for Independent Watchdog Service

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: initialize timer and state`
`* Step 3: monitor heartbeat inputs`
`* Step 4: evaluate timeout conditions`
`* Step 5: trigger reset action if expired`
`* Step 6: log status`
`* Step 7: update next check deadline`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call wdg_init() once in main() init sequence. Call wdg_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

81. DD-COMP-079 - Clock and Timer Configuration

DD-COMP-079	**Clock and Timer Configuration** | Service | Module: HAL | Pattern: Factory | SW-REQ-068

`@trace DD-COMP-079 SW-REQ-068`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void clock_cfg_init(void)`	void	Initialise all state for Clock and Timer Configuration
main.c	`static uint8_t clock_cfg_apply(void *clock_profile)`	uint8_t - 0 success, non-zero error code	Execute core logic for Clock and Timer Configuration
main.c	`static uint8_t set_pll_params(void)`	uint8_t	Helper function: set pll params
main.c	`static uint8_t enable_clock_domains(void)`	uint8_t	Helper function: enable clock domains
main.c	`static uint8_t sync_timer_sources(void)`	uint8_t	Helper function: sync timer sources

Doxygen Comments

```
/**
* @brief Initialise Clock and Timer Configuration.
* @trace DD-COMP-079 SW-REQ-068
*/
/**
* @brief Run Clock and Timer Configuration process cycle.
* @param clock_profile Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-079 SW-REQ-068
*/
/**
* @brief Set pll params.
* @trace DD-COMP-079 SW-REQ-068
*/
/**
* @brief Enable clock domains.
```

```
 * @trace DD-COMP-079 SW-REQ-068
 */
 /**
 * @brief Sync timer sources.
 * @trace DD-COMP-079 SW-REQ-068
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Load requested clock and timer settings
2	Validate frequency and divider ranges
3	Configure PLL and oscillator sources
4	Apply clock domain dividers and gates
5	Synchronize timer prescalers to sources
6	Verify stability and report status

Forbidden - Must NOT Do

#	Forbidden Rule
1	reconfiguring clocks while active peripherals run
2	using unsupported PLL multipliers
3	skipping stability checks
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'clkcfg' (e.g. clkcfgresult).`

2	State struct: static ClockTimerConfig_t clkcfgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_079_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ClockTimerConfig_t;`
	State struct for Clock and Timer Configuration

`void clock_cfg_init(void);`	Prototype: initialise Clock and Timer Configuration
`uint8_t clock_cfg_apply(void *p_data);`	Prototype: run Clock and Timer Configuration process cycle
`uint8_t set_pll_params(void);`	Prototype: set pll params
`uint8_t enable_clock_domains(void);`	Prototype: enable clock domains
`uint8_t sync_timer_sources(void);`	Prototype: sync timer sources

`typedef enum {`
`DD_COMP_079_ERR_NONE = 0x00U,`
`DD_COMP_079_ERR_NULL = 0x01U,`
`DD_COMP_079_ERR_INVALID = 0x02U,`
`DD_COMP_079_ERR_TIMEOUT = 0x03U`
`} DD_COMP_079_ERR_t;`
	Error code enum for Clock and Timer Configuration

main.c - What to Add

Item	Purpose

`static ClockTimerConfig_t clkcfgstate = {0};`	Module-level zero-initialised state for Clock and Timer Configuration

`/* Process function logic order:`
`* Step 1: Load requested clock and timer settings`
`* Step 2: Validate frequency and divider ranges`
`* Step 3: Configure PLL and oscillator sources`
`* Step 4: Apply clock domain dividers and gates`
`* Step 5: Synchronize timer prescalers to sources`
`* Step 6: Verify stability and report status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call clock_cfg_init() once in main() init sequence. Call clock_cfg_apply() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

82. DD-COMP-080 - Layered Architecture Enforcement Interface

DD-COMP-080	**Layered Architecture Enforcement Interface** | Interface | Module: Application | Pattern: Adapter | SW-REQ-069

`@trace DD-COMP-080 SW-REQ-069`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initLayerEnf(void)`	void	Initialise all state for Layered Architecture Enforcement Interface
main.c	`static uint8_t processLayerRules(void *configData)`	uint8_t - 0 success, non-zero error code	Execute core logic for Layered Architecture Enforcement Interface
main.c	`static uint8_t parseConfig(void)`	uint8_t	Helper function: parseConfig
main.c	`static uint8_t checkDependency(void)`	uint8_t	Helper function: checkDependency
main.c	`static uint8_t reportViolation(void)`	uint8_t	Helper function: reportViolation

Doxygen Comments

```
/**
* @brief Initialise Layered Architecture Enforcement Interface.
* @trace DD-COMP-080 SW-REQ-069
*/
/**
* @brief Run Layered Architecture Enforcement Interface process cycle.
* @param configData Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-080 SW-REQ-069
*/
/**
* @brief Parseconfig.
* @trace DD-COMP-080 SW-REQ-069
```

```
*/
/**
* @brief Checkdependency.
* @trace DD-COMP-080 SW-REQ-069
*/
/**
* @brief Reportviolation.
* @trace DD-COMP-080 SW-REQ-069
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load enforcement configuration
2	parse module dependency data
3	validate dependencies against layer rules
4	collect any violations
5	format enforcement report
6	return status and report

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded layer rules
2	circular dependency checks omission
3	mixing UI with enforcement logic
4	No direct register access - use HAL abstraction only
5	All input pointers must be NULL-checked before dereferencing
6	No global state mutation from API functions

Variable Naming Rules

#	Rule

1	All local variables prefixed 'lenf' (e.g. lenfresult).
2	State struct: static LayerEnforcer_t lenfstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_080_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} LayerEnforcer_t;`
	State struct for Layered Architecture Enforcement Interface

`void initLayerEnf(void);`	Prototype: initialise Layered Architecture Enforcement Interface
`uint8_t processLayerRules(void *p_data);`	Prototype: run Layered Architecture Enforcement Interface process cycle
`uint8_t parseConfig(void);`	Prototype: parseConfig
`uint8_t checkDependency(void);`	Prototype: checkDependency
`uint8_t reportViolation(void);`	Prototype: reportViolation

`typedef enum {`
`DD_COMP_080_ERR_NONE = 0x00U,`
`DD_COMP_080_ERR_NULL = 0x01U,`
`DD_COMP_080_ERR_INVALID = 0x02U,`
`DD_COMP_080_ERR_TIMEOUT = 0x03U`
`} DD_COMP_080_ERR_t;`
	Error code enum for Layered Architecture Enforcement Interface

main.c - What to Add

Item	Purpose
`static LayerEnforcer_t lenfstate = {0};`	Module-level zero-initialised state for Layered Architecture Enforcement Interface

`/* Process function logic order:`
`* Step 1: load enforcement configuration`
`* Step 2: parse module dependency data`
`* Step 3: validate dependencies against layer rules`
`* Step 4: collect any violations`
`* Step 5: format enforcement report`
`* Step 6: return status and report`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Adapter. Call initLayerEnf() once in main() init sequence. Call processLayerRules() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

83. DD-COMP-081 - System State Machine

DD-COMP-081	**System State Machine** | Class | Module: Application | Pattern: State | SW-REQ-070

`@trace DD-COMP-081 SW-REQ-070`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initSM(void)`	void	Initialise all state for System State Machine
main.c	`static uint8_t runSM(void *eventQueue)`	uint8_t - 0 success, non-zero error code	Execute core logic for System State Machine
main.c	`static uint8_t evalEvent(void)`	uint8_t	Helper function: evalEvent
main.c	`static uint8_t transition(void)`	uint8_t	Helper function: transition
main.c	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState

Doxygen Comments

```
/**
* @brief Initialise System State Machine.
* @trace DD-COMP-081 SW-REQ-070
*/
/**
* @brief Run System State Machine process cycle.
* @param eventQueue Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-081 SW-REQ-070
*/
/**
* @brief Evalevent.
* @trace DD-COMP-081 SW-REQ-070
*/
/**
* @brief Transition.
```

```
* @trace DD-COMP-081 SW-REQ-070
*/
/**
* @brief Updatestate.
* @trace DD-COMP-081 SW-REQ-070
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load current state
2	read inputs and events
3	evaluate transition rules
4	apply state change
5	update outputs
6	log or notify changes

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded state IDs
2	blocking waits
3	global mutable state

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'sm' (e.g. smresult).`
2	`State struct: static SystemStateMachine_t smstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_081_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SystemStateMachine_t;`
	State struct for System State Machine

`void initSM(void);`	Prototype: initialise System State Machine
`uint8_t runSM(void *p_data);`	Prototype: run System State Machine process cycle
`uint8_t evalEvent(void);`	Prototype: evalEvent
`uint8_t transition(void);`	Prototype: transition
`uint8_t updateState(void);`	Prototype: updateState

`typedef enum {`
`DD_COMP_081_ERR_NONE = 0x00U,`
`DD_COMP_081_ERR_NULL = 0x01U,`
`DD_COMP_081_ERR_INVALID = 0x02U,`
`DD_COMP_081_ERR_TIMEOUT = 0x03U`
`} DD_COMP_081_ERR_t;`
	Error code enum for System State Machine

main.c - What to Add

Item	Purpose
`static SystemStateMachine_t smstate = {0};`	Module-level zero-initialised state for System State Machine

`/* Process function logic order:`
`* Step 1: load current state`
`* Step 2: read inputs and events`
`* Step 3: evaluate transition rules`
`* Step 4: apply state change`
`* Step 5: update outputs`
`* Step 6: log or notify changes`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initSM() once in main() init sequence. Call runSM() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

84. DD-COMP-082 - Reset Initialization Handler

DD-COMP-082	**Reset Initialization Handler** | Service | Module: Application | Pattern: Observer | SW-REQ-071

`@trace DD-COMP-082 SW-REQ-071`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void reset_init(void)`	void	Initialise all state for Reset Initialization Handler
`main.c`	`static uint8_t reset_handle(void *reset_flag)`	uint8_t - 0 success, non-zero error code	Execute core logic for Reset Initialization Handler
`main.c`	`static uint8_t load_defaults(void)`	uint8_t	Helper function: load defaults
`main.c`	`static uint8_t clear_state(void)`	uint8_t	Helper function: clear state
`main.c`	`static uint8_t notify_ready(void)`	uint8_t	Helper function: notify ready

Doxygen Comments

```
/**
* @brief Initialise Reset Initialization Handler.
* @trace DD-COMP-082 SW-REQ-071
*/
/**
* @brief Run Reset Initialization Handler process cycle.
* @param reset_flag Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-082 SW-REQ-071
*/
/**
* @brief Load defaults.
* @trace DD-COMP-082 SW-REQ-071
*/
/**
* @brief Clear state.
```

```
 * @trace DD-COMP-082 SW-REQ-071
 */
 /**
 * @brief Notify ready.
 * @trace DD-COMP-082 SW-REQ-071
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read reset cause
2	load default settings
3	clear runtime state
4	initialize services
5	signal readiness

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	hardcoded constants
3	side effects during init
4	unused globals
5	No malloc/calloc/free anywhere in this component
6	No recursive function calls - use iterative logic only
7	No processing inside ISR - set flag and return only
8	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'rst' (e.g. rstresult).`

2	State struct: static ResetInitHandler_t rststate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_082_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ResetInitHandler_t;`
	State struct for Reset Initialization Handler

`void reset_init(void);`	Prototype: initialise Reset Initialization Handler
`uint8_t reset_handle(void *p_data);`	Prototype: run Reset Initialization Handler process cycle
`uint8_t load_defaults(void);`	Prototype: load defaults
`uint8_t clear_state(void);`	Prototype: clear state
`uint8_t notify_ready(void);`	Prototype: notify ready

`typedef enum {`
`DD_COMP_082_ERR_NONE = 0x00U,`
`DD_COMP_082_ERR_NULL = 0x01U,`
`DD_COMP_082_ERR_INVALID = 0x02U,`
`DD_COMP_082_ERR_TIMEOUT = 0x03U`
`} DD_COMP_082_ERR_t;`
	Error code enum for Reset Initialization Handler

main.c - What to Add

Item	Purpose

`static ResetInitHandler_t rststate = {0};`	Module-level zero-initialised state for Reset Initialization Handler

`/* Process function logic order:`
`* Step 1: read reset cause`
`* Step 2: load default settings`
`* Step 3: clear runtime state`
`* Step 4: initialize services`
`* Step 5: signal readiness`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call reset_init() once in main() init sequence. Call reset_handle() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

85. DD-COMP-083 - Nominal Transition Controller

DD-COMP-083	**Nominal Transition Controller** | Service | Module: Application | Pattern: Command | SW-REQ-072

`@trace DD-COMP-083 SW-REQ-072`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void ntc_init(void)`	void	Initialise all state for Nominal Transition Controller
`main.c`	`static uint8_t ntc_run(void *current_state)`	uint8_t - 0 success, non-zero error code	Execute core logic for Nominal Transition Controller
`main.c`	`static uint8_t ntc_load_config(void)`	uint8_t	Helper function: ntc load config
`main.c`	`static uint8_t ntc_eval_transition(void)`	uint8_t	Helper function: ntc eval transition
`main.c`	`static uint8_t ntc_emit_event(void)`	uint8_t	Helper function: ntc emit event

Doxygen Comments

```
/**
* @brief Initialise Nominal Transition Controller.
* @trace DD-COMP-083 SW-REQ-072
*/
/**
* @brief Run Nominal Transition Controller process cycle.
* @param current_state Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-083 SW-REQ-072
*/
/**
* @brief Ntc load config.
* @trace DD-COMP-083 SW-REQ-072
*/
/**
```

```
 * @brief Ntc eval transition.
 * @trace DD-COMP-083 SW-REQ-072
 */
 /**
 * @brief Ntc emit event.
 * @trace DD-COMP-083 SW-REQ-072
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration and current state
2	read requested transition inputs
3	evaluate guards and conditions
4	apply nominal transition rules
5	update internal state
6	emit transition outputs
7	log or report status

Forbidden - Must NOT Do

#	Forbidden Rule
1	unvalidated state changes
2	blocking operations in service loop
3	implicit state mutations
4	verbose logging in hot path
5	No malloc/calloc/free anywhere in this component
6	No recursive function calls - use iterative logic only
7	No processing inside ISR - set flag and return only
8	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	All local variables prefixed 'ntc' (e.g. ntcresult).
2	State struct: static NominalTransitionController_t ntcstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_083_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} NominalTransitionController_t;`
	State struct for Nominal Transition Controller

`void ntc_init(void);`	Prototype: initialise Nominal Transition Controller
`uint8_t ntc_run(void *p_data);`	Prototype: run Nominal Transition Controller process cycle
`uint8_t ntc_load_config(void);`	Prototype: ntc load config
`uint8_t ntc_eval_transition(void);`	Prototype: ntc eval transition
`uint8_t ntc_emit_event(void);`	Prototype: ntc emit event

`typedef enum {`
`DD_COMP_083_ERR_NONE = 0x00U,`
`DD_COMP_083_ERR_NULL = 0x01U,`
`DD_COMP_083_ERR_INVALID = 0x02U,`
`DD_COMP_083_ERR_TIMEOUT = 0x03U`
`} DD_COMP_083_ERR_t;`
	Error code enum for Nominal Transition Controller

main.c - What to Add

Item	Purpose
`static NominalTransitionController_t ntcstate = {0};`	Module-level zero-initialised state for Nominal Transition Controller

`/* Process function logic order:`
`* Step 1: load configuration and current state`
`* Step 2: read requested transition inputs`
`* Step 3: evaluate guards and conditions`
`* Step 4: apply nominal transition rules`
`* Step 5: update internal state`
`* Step 6: emit transition outputs`
`* Step 7: log or report status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Command. Call ntc_init() once in main() init sequence. Call ntc_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

86. DD-COMP-084 - FaultStateManager

DD-COMP-084 **FaultStateManager** | Class | Module: StateManagementModule | Pattern: State | SW-REQ-073

`@trace DD-COMP-084 SW-REQ-073`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initFaults(void)`	void	Initialise all state for FaultStateManager
main.c	`static uint8_t updateFaults(void *signalData)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultStateManager
main.c	`static uint8_t readSignals(void)`	uint8_t	Helper function: readSignals
main.c	`static uint8_t evaluateStates(void)`	uint8_t	Helper function: evaluateStates
main.c	`static uint8_t logTransition(void)`	uint8_t	Helper function: logTransition

Doxygen Comments

```
/**
* @brief Initialise FaultStateManager.
* @trace DD-COMP-084 SW-REQ-073
*/
/**
* @brief Run FaultStateManager process cycle.
* @param signalData Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-084 SW-REQ-073
*/
/**
* @brief Readsignals.
* @trace DD-COMP-084 SW-REQ-073
*/
/**
* @brief Evaluatestates.
```

```
 * @trace DD-COMP-084 SW-REQ-073
 */
 /**
 * @brief Logtransition.
 * @trace DD-COMP-084 SW-REQ-073
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration and thresholds
2	read current fault inputs
3	evaluate fault conditions
4	update fault state machine
5	record any state transitions
6	emit fault status outputs

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded thresholds
2	silent state changes
3	blocking I/O in update

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fault' (e.g. faultresult).`
2	`State struct: static FaultStateManager_t faultstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_084_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultStateManager_t;`
	State struct for FaultStateManager

`void initFaults(void);`	Prototype: initialise FaultStateManager
`uint8_t updateFaults(void *p_data);`	Prototype: run FaultStateManager process cycle
`uint8_t readSignals(void);`	Prototype: readSignals
`uint8_t evaluateStates(void);`	Prototype: evaluateStates
`uint8_t logTransition(void);`	Prototype: logTransition

`typedef enum {`
`DD_COMP_084_ERR_NONE = 0x00U,`
`DD_COMP_084_ERR_NULL = 0x01U,`
`DD_COMP_084_ERR_INVALID = 0x02U,`
`DD_COMP_084_ERR_TIMEOUT = 0x03U`
`} DD_COMP_084_ERR_t;`
	Error code enum for FaultStateManager

main.c - What to Add

Item	Purpose
`static FaultStateManager_t faultstate = {0};`	Module-level zero-initialised state for FaultStateManager

`/* Process function logic order:`
`* Step 1: load configuration and thresholds`
`* Step 2: read current fault inputs`
`* Step 3: evaluate fault conditions`
`* Step 4: update fault state machine`
`* Step 5: record any state transitions`
`* Step 6: emit fault status outputs`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initFaults() once in main() init sequence. Call updateFaults() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

87. DD-COMP-085 - HardwareWatchdogInitializer

DD-COMP-085	**HardwareWatchdogInitializer** | Service | Module: WatchdogModule | Pattern: Factory | SW-REQ-074

`@trace DD-COMP-085 SW-REQ-074`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void wdg_init(void)`	void	Initialise all state for HardwareWatchdogInitializer
`main.c`	`static uint8_t wdg_start(void *cfg)`	uint8_t - 0 success, non-zero error code	Execute core logic for HardwareWatchdogInitializer
`main.c`	`static uint8_t load_cfg(void)`	uint8_t	Helper function: load cfg
`main.c`	`static uint8_t enable_wdg(void)`	uint8_t	Helper function: enable wdg
`main.c`	`static uint8_t kick_wdg(void)`	uint8_t	Helper function: kick wdg

Doxygen Comments

```
/**
* @brief Initialise HardwareWatchdogInitializer.
* @trace DD-COMP-085 SW-REQ-074
*/
/**
* @brief Run HardwareWatchdogInitializer process cycle.
* @param cfg Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-085 SW-REQ-074
*/
/**
* @brief Load cfg.
* @trace DD-COMP-085 SW-REQ-074
*/
/**
```

```
 * @brief Enable wdg.
 * @trace DD-COMP-085 SW-REQ-074
 */
 /**
 * @brief Kick wdg.
 * @trace DD-COMP-085 SW-REQ-074
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load watchdog configuration
2	validate configuration values
3	configure hardware watchdog registers
4	enable watchdog timer
5	perform initial kick
6	report initialization status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	hardcoded timeouts
3	ignoring hardware faults
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'wdg' (e.g. wdgresult).`

2	State struct: static HardwareWatchdogInitializer_t wdgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_085_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} HardwareWatchdogInitializer_t;`
	State struct for HardwareWatchdogInitializer

`void wdg_init(void);`	Prototype: initialise HardwareWatchdogInitializer
`uint8_t wdg_start(void *p_data);`	Prototype: run HardwareWatchdogInitializer process cycle
`uint8_t load_cfg(void);`	Prototype: load cfg
`uint8_t enable_wdg(void);`	Prototype: enable wdg
`uint8_t kick_wdg(void);`	Prototype: kick wdg

`typedef enum {`
`DD_COMP_085_ERR_NONE = 0x00U,`
`DD_COMP_085_ERR_NULL = 0x01U,`
`DD_COMP_085_ERR_INVALID = 0x02U,`
`DD_COMP_085_ERR_TIMEOUT = 0x03U`
`} DD_COMP_085_ERR_t;`
	Error code enum for HardwareWatchdogInitializer

main.c - What to Add

Item	Purpose

`static HardwareWatchdogInitializer_t wdgstate = {0};`	Module-level zero-initialised state for HardwareWatchdogInitializer

`/* Process function logic order:`
`* Step 1: load watchdog configuration`
`* Step 2: validate configuration values`
`* Step 3: configure hardware watchdog registers`
`* Step 4: enable watchdog timer`
`* Step 5: perform initial kick`
`* Step 6: report initialization status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Factory. Call wdg_init() once in main() init sequence. Call wdg_start() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

88. DD-COMP-086 - WatchdogServicer

DD-COMP-086	**WatchdogServicer** | Service | Module: WatchdogModule | Pattern: Observer | SW-REQ-075

`@trace DD-COMP-086 SW-REQ-075`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void wdgsvc_init(void)`	void	Initialise all state for WatchdogServicer
`main.c`	`static uint8_t wdgsvc_run(void *config)`	uint8_t - 0 success, non-zero error code	Execute core logic for WatchdogServicer
`main.c`	`static uint8_t wdgsvc_check(void)`	uint8_t	Helper function: wdgsvc check
`main.c`	`static uint8_t wdgsvc_kick(void)`	uint8_t	Helper function: wdgsvc kick
`main.c`	`static uint8_t wdgsvc_log(void)`	uint8_t	Helper function: wdgsvc log

Doxygen Comments

```
/**
* @brief Initialise WatchdogServicer.
* @trace DD-COMP-086 SW-REQ-075
*/
/**
* @brief Run WatchdogServicer process cycle.
* @param config Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-086 SW-REQ-075
*/
/**
* @brief Wdgsvc check.
* @trace DD-COMP-086 SW-REQ-075
*/
/**
* @brief Wdgsvc kick.
```

```
 * @trace DD-COMP-086 SW-REQ-075
 */
 /**
 * @brief Wdgsvc log.
 * @trace DD-COMP-086 SW-REQ-075
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	initialize watchdog interface
3	monitor service heartbeats
4	detect timeout conditions
5	trigger watchdog reset
6	record events/status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking calls
2	long polling loops
3	silent failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'wdg' (e.g. wdgresult).`

2	State struct: static WatchdogServicer_t wdgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_086_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} WatchdogServicer_t;`
	State struct for WatchdogServicer

`void wdgsvc_init(void);`	Prototype: initialise WatchdogServicer
`uint8_t wdgsvc_run(void *p_data);`	Prototype: run WatchdogServicer process cycle
`uint8_t wdgsvc_check(void);`	Prototype: wdgsvc check
`uint8_t wdgsvc_kick(void);`	Prototype: wdgsvc kick
`uint8_t wdgsvc_log(void);`	Prototype: wdgsvc log

`typedef enum {`
`DD_COMP_086_ERR_NONE = 0x00U,`
`DD_COMP_086_ERR_NULL = 0x01U,`
`DD_COMP_086_ERR_INVALID = 0x02U,`
`DD_COMP_086_ERR_TIMEOUT = 0x03U`
`} DD_COMP_086_ERR_t;`
	Error code enum for WatchdogServicer

main.c - What to Add

Item	Purpose

`static WatchdogServicer_t wdgstate = {0};`	Module-level zero-initialised state for WatchdogServicer

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: initialize watchdog interface`
`* Step 3: monitor service heartbeats`
`* Step 4: detect timeout conditions`
`* Step 5: trigger watchdog reset`
`* Step 6: record events/status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call wdgsvc_init() once in main() init sequence. Call wdgsvc_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

89. DD-COMP-087 - FatalMalfunctionResetHandler

DD-COMP-087	**FatalMalfunctionResetHandler** | Service | Module: FaultHandlingModule | Pattern: Command | SW-REQ-076

`@trace DD-COMP-087 SW-REQ-076`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void initResetHandler(void)`	void	Initialise all state for FatalMalfunctionResetHandler
`main.c`	`static uint8_t handleFatalReset(void *faultStatus)`	uint8_t - 0 success, non-zero error code	Execute core logic for FatalMalfunctionResetHandler
`main.c`	`static uint8_t readFaultStatus(void)`	uint8_t	Helper function: readFaultStatus
`main.c`	`static uint8_t logResetEvent(void)`	uint8_t	Helper function: logResetEvent
`main.c`	`static uint8_t clearResetFlags(void)`	uint8_t	Helper function: clearResetFlags

Doxygen Comments

```
/**
* @brief Initialise FatalMalfunctionResetHandler.
* @trace DD-COMP-087 SW-REQ-076
*/
/**
* @brief Run FatalMalfunctionResetHandler process cycle.
* @param faultStatus Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-087 SW-REQ-076
*/
/**
* @brief Readfaultstatus.
* @trace DD-COMP-087 SW-REQ-076
*/
/**
```

```
* @brief Logresetevent.
* @trace DD-COMP-087 SW-REQ-076
*/
/**
* @brief Clearresetflags.
* @trace DD-COMP-087 SW-REQ-076
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration and dependencies
2	read current fault and reset status
3	determine if fatal malfunction occurred
4	log or report the reset event
5	perform reset flag cleanup
6	update internal state and metrics

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	clearing flags before logging
3	ignoring config limits
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fmrh' (e.g. fmrhresult).`

2	State struct: static FatalMalfunctionResetHandler_t fmrhstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_087_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FatalMalfunctionResetHandler_t;`
	State struct for FatalMalfunctionResetHandler

`void initResetHandler(void);`	Prototype: initialise FatalMalfunctionResetHandler
`uint8_t handleFatalReset(void *p_data);`	Prototype: run FatalMalfunctionResetHandler process cycle
`uint8_t readFaultStatus(void);`	Prototype: readFaultStatus
`uint8_t logResetEvent(void);`	Prototype: logResetEvent
`uint8_t clearResetFlags(void);`	Prototype: clearResetFlags

`typedef enum {`
`DD_COMP_087_ERR_NONE = 0x00U,`
`DD_COMP_087_ERR_NULL = 0x01U,`
`DD_COMP_087_ERR_INVALID = 0x02U,`
`DD_COMP_087_ERR_TIMEOUT = 0x03U`
`} DD_COMP_087_ERR_t;`
	Error code enum for FatalMalfunctionResetHandler

main.c - What to Add

Item	Purpose

`static FatalMalfunctionResetHandler_t fmrhstate = {0};`	Module-level zero-initialised state for FatalMalfunctionResetHandler

`/* Process function logic order:`
`* Step 1: load configuration and dependencies`
`* Step 2: read current fault and reset status`
`* Step 3: determine if fatal malfunction occurred`
`* Step 4: log or report the reset event`
`* Step 5: perform reset flag cleanup`
`* Step 6: update internal state and metrics`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Command. Call initResetHandler() once in main() init sequence. Call handleFatalReset() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

90. DD-COMP-088 - ADCOnlyControlAlgorithm

DD-COMP-088	**ADCOnlyControlAlgorithm** | Algorithm | Module: ControlModule | Pattern: Strategy | SW-REQ-077

`@trace DD-COMP-088 SW-REQ-077`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void adcCtlInit(void)`	void	Initialise all state for ADCOnlyControlAlgorithm
`main.c`	`static uint8_t adcCtlStep(void *adc_raw)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCOnlyControlAlgorithm
`main.c`	`static uint8_t readAdc(void)`	uint8_t	Helper function: readAdc
`main.c`	`static uint8_t filterAdc(void)`	uint8_t	Helper function: filterAdc
`main.c`	`static uint8_t calcControl(void)`	uint8_t	Helper function: calcControl

Doxygen Comments

```
/**
* @brief Initialise ADCOnlyControlAlgorithm.
* @trace DD-COMP-088 SW-REQ-077
*/
/**
* @brief Run ADCOnlyControlAlgorithm process cycle.
* @param adc_raw Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-088 SW-REQ-077
*/
/**
* @brief Readadc.
* @trace DD-COMP-088 SW-REQ-077
*/
/**
```

```
 * @brief Filteradc.
 * @trace DD-COMP-088 SW-REQ-077
 */
 /**
 * @brief Calccontrol.
 * @trace DD-COMP-088 SW-REQ-077
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	read ADC samples
3	filter and scale readings
4	compute control output
5	apply limits
6	store output/state

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	magic numbers
3	unbounded output
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adc' (e.g. adcresult).`

2	State struct: static AdcOnlyControl_t adcstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_088_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcOnlyControl_t;`
	State struct for ADCOnlyControlAlgorithm

`void adcCtlInit(void);`	Prototype: initialise ADCOnlyControlAlgorithm
`uint8_t adcCtlStep(void *p_data);`	Prototype: run ADCOnlyControlAlgorithm process cycle
`uint8_t readAdc(void);`	Prototype: readAdc
`uint8_t filterAdc(void);`	Prototype: filterAdc
`uint8_t calcControl(void);`	Prototype: calcControl

`typedef enum {`
`DD_COMP_088_ERR_NONE = 0x00U,`
`DD_COMP_088_ERR_NULL = 0x01U,`
`DD_COMP_088_ERR_INVALID = 0x02U,`
`DD_COMP_088_ERR_TIMEOUT = 0x03U`
`} DD_COMP_088_ERR_t;`
	Error code enum for ADCOnlyControlAlgorithm

main.c - What to Add

Item	Purpose

`static AdcOnlyControl_t adcstate = {0};`	Module-level zero-initialised state for ADCOnlyControlAlgorithm

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: read ADC samples`
`* Step 3: filter and scale readings`
`* Step 4: compute control output`
`* Step 5: apply limits`
`* Step 6: store output/state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Strategy. Call adcCtlInit() once in main() init sequence. Call adcCtlStep() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

91. DD-COMP-089 - ControlLoopScheduler

DD-COMP-089	**ControlLoopScheduler** | Service | Module: ControlModule | Pattern: Singleton | SW-REQ-078

`@trace DD-COMP-089 SW-REQ-078`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initCtrlLoop(void)`	void	Initialise all state for ControlLoopScheduler
main.c	`static uint8_t runCtrlLoop(void *sensor_data)`	uint8_t - 0 success, non-zero error code	Execute core logic for ControlLoopScheduler
main.c	`static uint8_t readInputs(void)`	uint8_t	Helper function: readInputs
main.c	`static uint8_t computeControl(void)`	uint8_t	Helper function: computeControl
main.c	`static uint8_t writeOutputs(void)`	uint8_t	Helper function: writeOutputs

Doxygen Comments

```
/**
* @brief Initialise ControlLoopScheduler.
* @trace DD-COMP-089 SW-REQ-078
*/
/**
* @brief Run ControlLoopScheduler process cycle.
* @param sensor_data Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-089 SW-REQ-078
*/
/**
* @brief Readinputs.
* @trace DD-COMP-089 SW-REQ-078
*/
/**
* @brief Computecontrol.
```

```
 * @trace DD-COMP-089 SW-REQ-078
 */
 /**
 * @brief Writeoutputs.
 * @trace DD-COMP-089 SW-REQ-078
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	read sensor inputs
3	compute control actions
4	schedule actuators
5	handle timing and errors
6	log cycle results

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking calls in loop
2	unsafe pointer use
3	hardcoded timing
4	silent failures
5	No malloc/calloc/free anywhere in this component
6	No recursive function calls - use iterative logic only
7	No processing inside ISR - set flag and return only
8	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'cls' (e.g. clsresult).
2	State struct: static ControlLoopScheduler_t clsstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_089_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ControlLoopScheduler_t;`
	State struct for ControlLoopScheduler

`void initCtrlLoop(void);`	Prototype: initialise ControlLoopScheduler
`uint8_t runCtrlLoop(void *p_data);`	Prototype: run ControlLoopScheduler process cycle
`uint8_t readInputs(void);`	Prototype: readInputs
`uint8_t computeControl(void);`	Prototype: computeControl
`uint8_t writeOutputs(void);`	Prototype: writeOutputs

`typedef enum {`
`DD_COMP_089_ERR_NONE = 0x00U,`
`DD_COMP_089_ERR_NULL = 0x01U,`
`DD_COMP_089_ERR_INVALID = 0x02U,`
`DD_COMP_089_ERR_TIMEOUT = 0x03U`
`} DD_COMP_089_ERR_t;`
	Error code enum for ControlLoopScheduler

main.c - What to Add

Item	Purpose
`static ControlLoopScheduler_t clsstate = {0};`	Module-level zero-initialised state for ControlLoopScheduler

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: read sensor inputs`
`* Step 3: compute control actions`
`* Step 4: schedule actuators`
`* Step 5: handle timing and errors`
`* Step 6: log cycle results`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call initCtrlLoop() once in main() init sequence. Call runCtrlLoop() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

92. DD-COMP-090 - ThresholdStopEvaluator

DD-COMP-090	**ThresholdStopEvaluator** | Class | Module: SafetyControlModule | Pattern: N/A | SW-REQ-079

`@trace DD-COMP-090 SW-REQ-079`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initEvaluator(void)`	void	Initialise all state for ThresholdStopEvaluator
main.c	`static uint8_t processThreshold(void *currentValue)`	uint8_t - 0 success, non-zero error code	Execute core logic for ThresholdStopEvaluator
main.c	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState
main.c	`static uint8_t checkThreshold(void)`	uint8_t	Helper function: checkThreshold
main.c	`static uint8_t emitStop(void)`	uint8_t	Helper function: emitStop

Doxygen Comments

```
/**
* @brief Initialise ThresholdStopEvaluator.
* @trace DD-COMP-090 SW-REQ-079
*/
/**
* @brief Run ThresholdStopEvaluator process cycle.
* @param currentValue Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-090 SW-REQ-079
*/
/**
* @brief Updatestate.
* @trace DD-COMP-090 SW-REQ-079
*/
/**
* @brief Checkthreshold.
```

```
 * @trace DD-COMP-090 SW-REQ-079
 */
 /**
 * @brief Emitstop.
 * @trace DD-COMP-090 SW-REQ-079
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	read current value
3	compare against threshold
4	update internal state
5	decide stop condition
6	output stop flag

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded thresholds
2	state mutation without validation
3	silent overflow
4	blocking I/O

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'thres' (e.g. thresresult).`
2	`State struct: static ThresholdStopEvaluator_t thresstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`

5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_090_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ThresholdStopEvaluator_t;`
	State struct for ThresholdStopEvaluator

`void initEvaluator(void);`	Prototype: initialise ThresholdStopEvaluator
`uint8_t processThreshold(void *p_data);`	Prototype: run ThresholdStopEvaluator process cycle
`uint8_t updateState(void);`	Prototype: updateState
`uint8_t checkThreshold(void);`	Prototype: checkThreshold
`uint8_t emitStop(void);`	Prototype: emitStop

`typedef enum {`
`DD_COMP_090_ERR_NONE = 0x00U,`
`DD_COMP_090_ERR_NULL = 0x01U,`
`DD_COMP_090_ERR_INVALID = 0x02U,`
`DD_COMP_090_ERR_TIMEOUT = 0x03U`
`} DD_COMP_090_ERR_t;`
	Error code enum for ThresholdStopEvaluator

main.c - What to Add

Item	Purpose
`static ThresholdStopEvaluator_t thresstate = {0};`	Module-level zero-initialised state for ThresholdStopEvaluator

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: read current value`
`* Step 3: compare against threshold`
`* Step 4: update internal state`
`* Step 5: decide stop condition`
`* Step 6: output stop flag`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call initEvaluator() once in main() init sequence. Call processThreshold() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

93. DD-COMP-091 - HysteresisBandManager

DD-COMP-091	**HysteresisBandManager** | Class | Module: SafetyControlModule | Pattern: State | SW-REQ-080

`@trace DD-COMP-091 SW-REQ-080`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void InitBandMgr(void)`	void	Initialise all state for HysteresisBandManager
main.c	`static uint8_t UpdateBand(void *signal_in)`	uint8_t - 0 success, non-zero error code	Execute core logic for HysteresisBandManager
main.c	`static uint8_t ClampInput(void)`	uint8_t	Helper function: ClampInput
main.c	`static uint8_t ApplyHysteresis(void)`	uint8_t	Helper function: ApplyHysteresis
main.c	`static uint8_t UpdateState(void)`	uint8_t	Helper function: UpdateState

Doxygen Comments

```
/**
* @brief Initialise HysteresisBandManager.
* @trace DD-COMP-091 SW-REQ-080
*/
/**
* @brief Run HysteresisBandManager process cycle.
* @param signal_in Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-091 SW-REQ-080
*/
/**
* @brief Clampinput.
* @trace DD-COMP-091 SW-REQ-080
*/
/**
* @brief Applyhysteresis.
```

```
 * @trace DD-COMP-091 SW-REQ-080
 */
 /**
 * @brief Updatestate.
 * @trace DD-COMP-091 SW-REQ-080
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load current input and thresholds
2	clamp input within valid range
3	compare input to upper and lower bands
4	update hysteresis state if thresholds crossed
5	derive output based on state
6	store updated state for next cycle

Forbidden - Must NOT Do

#	Forbidden Rule
1	magic numbers
2	state desync
3	unchecked limits

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'hb' (e.g. hbresult).`
2	`State struct: static HysteresisBandManager_t hbstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_091_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} HysteresisBandManager_t;`
	State struct for HysteresisBandManager

`void InitBandMgr(void);`	Prototype: initialise HysteresisBandManager
`uint8_t UpdateBand(void *p_data);`	Prototype: run HysteresisBandManager process cycle
`uint8_t ClampInput(void);`	Prototype: ClampInput
`uint8_t ApplyHysteresis(void);`	Prototype: ApplyHysteresis
`uint8_t UpdateState(void);`	Prototype: UpdateState

`typedef enum {`
`DD_COMP_091_ERR_NONE = 0x00U,`
`DD_COMP_091_ERR_NULL = 0x01U,`
`DD_COMP_091_ERR_INVALID = 0x02U,`
`DD_COMP_091_ERR_TIMEOUT = 0x03U`
`} DD_COMP_091_ERR_t;`
	Error code enum for HysteresisBandManager

main.c - What to Add

Item	Purpose
`static HysteresisBandManager_t hbstate = {0};`	Module-level zero-initialised state for HysteresisBandManager

`/* Process function logic order:`
`* Step 1: load current input and thresholds`
`* Step 2: clamp input within valid range`
`* Step 3: compare input to upper and lower bands`
`* Step 4: update hysteresis state if thresholds crossed`
`* Step 5: derive output based on state`
`* Step 6: store updated state for next cycle`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call InitBandMgr() once in main() init sequence. Call UpdateBand() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

94. DD-COMP-092 - RawAdcDataPolicy

DD-COMP-092	**RawAdcDataPolicy** | Class | Module: ControlModule | Pattern: Facade | SW-REQ-081

`@trace DD-COMP-092 SW-REQ-081`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_policy(void)`	void	Initialise all state for RawAdcDataPolicy
main.c	`static uint8_t process_adc(void *adc_buffer)`	uint8_t - 0 success, non-zero error code	Execute core logic for RawAdcDataPolicy
main.c	`static uint8_t parse_raw(void)`	uint8_t	Helper function: parse raw
main.c	`static uint8_t apply_policy(void)`	uint8_t	Helper function: apply policy
main.c	`static uint8_t validate_sample(void)`	uint8_t	Helper function: validate sample

Doxygen Comments

```
/**
* @brief Initialise RawAdcDataPolicy.
* @trace DD-COMP-092 SW-REQ-081
*/
/**
* @brief Run RawAdcDataPolicy process cycle.
* @param adc_buffer Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-092 SW-REQ-081
*/
/**
* @brief Parse raw.
* @trace DD-COMP-092 SW-REQ-081
*/
/**
* @brief Apply policy.
```

```
 * @trace DD-COMP-092 SW-REQ-081
 */
 /**
 * @brief Validate sample.
 * @trace DD-COMP-092 SW-REQ-081
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	ingest raw ADC buffer
3	validate sample range
4	apply policy rules
5	transform to internal format
6	emit processed data

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded constants
2	blocking I/O
3	silent failures

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adc' (e.g. adcresult).`
2	`State struct: static RawAdcDataPolicy_t adcstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_092_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} RawAdcDataPolicy_t;`
	State struct for RawAdcDataPolicy

`void init_policy(void);`	Prototype: initialise RawAdcDataPolicy
`uint8_t process_adc(void *p_data);`	Prototype: run RawAdcDataPolicy process cycle
`uint8_t parse_raw(void);`	Prototype: parse raw
`uint8_t apply_policy(void);`	Prototype: apply policy
`uint8_t validate_sample(void);`	Prototype: validate sample

`typedef enum {`
`DD_COMP_092_ERR_NONE = 0x00U,`
`DD_COMP_092_ERR_NULL = 0x01U,`
`DD_COMP_092_ERR_INVALID = 0x02U,`
`DD_COMP_092_ERR_TIMEOUT = 0x03U`
`} DD_COMP_092_ERR_t;`
	Error code enum for RawAdcDataPolicy

main.c - What to Add

Item	Purpose
`static RawAdcDataPolicy_t adcstate = {0};`	Module-level zero-initialised state for RawAdcDataPolicy

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: ingest raw ADC buffer`
`* Step 3: validate sample range`
`* Step 4: apply policy rules`
`* Step 5: transform to internal format`
`* Step 6: emit processed data`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call init_policy() once in main() init sequence. Call process_adc() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

95. DD-COMP-093 - UartStatusScheduler

DD-COMP-093	**UartStatusScheduler** | Service | Module: CommsModule | Pattern: Observer | SW-REQ-082

`@trace DD-COMP-093 SW-REQ-082`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void uartStatInit(void)`	void	Initialise all state for UartStatusScheduler
`main.c`	`static uint8_t uartStatRun(void *uart_handle)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartStatusScheduler
`main.c`	`static uint8_t pollUart(void)`	uint8_t	Helper function: pollUart
`main.c`	`static uint8_t updateStatus(void)`	uint8_t	Helper function: updateStatus
`main.c`	`static uint8_t scheduleNext(void)`	uint8_t	Helper function: scheduleNext

Doxygen Comments

```
/**
* @brief Initialise UartStatusScheduler.
* @trace DD-COMP-093 SW-REQ-082
*/
/**
* @brief Run UartStatusScheduler process cycle.
* @param uart_handle Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-093 SW-REQ-082
*/
/**
* @brief Polluart.
* @trace DD-COMP-093 SW-REQ-082
*/
/**
* @brief Updatestatus.
```

```
 * @trace DD-COMP-093 SW-REQ-082
 */
 /**
 * @brief Schedulenext.
 * @trace DD-COMP-093 SW-REQ-082
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize scheduler state
2	poll UART status registers
3	update internal status flags
4	enqueue or schedule status event
5	publish status output
6	wait for next tick

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UART reads
2	tight polling loops without delays
3	modifying shared state without locks
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'usched' (e.g. uschedresult).`

2	State struct: static UartStatusScheduler_t uschedstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_093_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartStatusScheduler_t;`
	State struct for UartStatusScheduler

`void uartStatInit(void);`	Prototype: initialise UartStatusScheduler
`uint8_t uartStatRun(void *p_data);`	Prototype: run UartStatusScheduler process cycle
`uint8_t pollUart(void);`	Prototype: pollUart
`uint8_t updateStatus(void);`	Prototype: updateStatus
`uint8_t scheduleNext(void);`	Prototype: scheduleNext

`typedef enum {`
`DD_COMP_093_ERR_NONE = 0x00U,`
`DD_COMP_093_ERR_NULL = 0x01U,`
`DD_COMP_093_ERR_INVALID = 0x02U,`
`DD_COMP_093_ERR_TIMEOUT = 0x03U`
`} DD_COMP_093_ERR_t;`
	Error code enum for UartStatusScheduler

main.c - What to Add

Item	Purpose

`static UartStatusScheduler_t uschedstate = {0};`	Module-level zero-initialised state for UartStatusScheduler

`/* Process function logic order:`
`* Step 1: initialize scheduler state`
`* Step 2: poll UART status registers`
`* Step 3: update internal status flags`
`* Step 4: enqueue or schedule status event`
`* Step 5: publish status output`
`* Step 6: wait for next tick`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call uartStatInit() once in main() init sequence. Call uartStatRun() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

96. DD-COMP-094 - UartStatusMessageBuilder

DD-COMP-094	**UartStatusMessageBuilder** | Class | Module: CommsModule | Pattern: Builder | SW-REQ-082

`@trace DD-COMP-094 SW-REQ-082`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init(void)`	void	Initialise all state for UartStatusMessageBuilder
main.c	`static uint8_t build(void *status_flags)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartStatusMessageBuilder
main.c	`static uint8_t appendHeader(void)`	uint8_t	Helper function: appendHeader
main.c	`static uint8_t encodeStatus(void)`	uint8_t	Helper function: encodeStatus
main.c	`static uint8_t calcChecksum(void)`	uint8_t	Helper function: calcChecksum

Doxygen Comments

```
/**
* @brief Initialise UartStatusMessageBuilder.
* @trace DD-COMP-094 SW-REQ-082
*/
/**
* @brief Run UartStatusMessageBuilder process cycle.
* @param status_flags Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-094 SW-REQ-082
*/
/**
* @brief Appendheader.
* @trace DD-COMP-094 SW-REQ-082
*/
/**
* @brief Encodestatus.
```

```
 * @trace DD-COMP-094 SW-REQ-082
 */
 /**
 * @brief Calcchecksum.
 * @trace DD-COMP-094 SW-REQ-082
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	reset internal buffer
2	append message header
3	encode status fields
4	append payload data
5	compute checksum
6	finalize message length

Forbidden - Must NOT Do

#	Forbidden Rule
1	dynamic allocation
2	blocking I/O
3	unchecked buffer writes

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'usb' (e.g. usbresult).`
2	`State struct: static UartStatusMessageBuilder_t usbstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_094_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartStatusMessageBuilder_t;`
	State struct for UartStatusMessageBuilder

`void init(void);`	Prototype: initialise UartStatusMessageBuilder
`uint8_t build(void *p_data);`	Prototype: run UartStatusMessageBuilder process cycle
`uint8_t appendHeader(void);`	Prototype: appendHeader
`uint8_t encodeStatus(void);`	Prototype: encodeStatus
`uint8_t calcChecksum(void);`	Prototype: calcChecksum

`typedef enum {`
`DD_COMP_094_ERR_NONE = 0x00U,`
`DD_COMP_094_ERR_NULL = 0x01U,`
`DD_COMP_094_ERR_INVALID = 0x02U,`
`DD_COMP_094_ERR_TIMEOUT = 0x03U`
`} DD_COMP_094_ERR_t;`
	Error code enum for UartStatusMessageBuilder

main.c - What to Add

Item	Purpose
`static UartStatusMessageBuilder_t usbstate = {0};`	Module-level zero-initialised state for UartStatusMessageBuilder

`/* Process function logic order:`
`* Step 1: reset internal buffer`
`* Step 2: append message header`
`* Step 3: encode status fields`
`* Step 4: append payload data`
`* Step 5: compute checksum`
`* Step 6: finalize message length`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Builder. Call init() once in main() init sequence. Call build() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

97. DD-COMP-095 - UartConfigService

DD-COMP-095	**UartConfigService** | Service | Module: CommsModule | Pattern: Singleton | SW-REQ-083

`@trace DD-COMP-095 SW-REQ-083`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void InitUartCfg(void)`	void	Initialise all state for UartConfigService
main.c	`static uint8_t ProcUartCfg(void *cfg_request)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartConfigService
main.c	`static uint8_t LoadDefaults(void)`	uint8_t	Helper function: LoadDefaults
main.c	`static uint8_t ValidateParams(void)`	uint8_t	Helper function: ValidateParams
main.c	`static uint8_t ApplyConfig(void)`	uint8_t	Helper function: ApplyConfig

Doxygen Comments

```
/**
* @brief Initialise UartConfigService.
* @trace DD-COMP-095 SW-REQ-083
*/
/**
* @brief Run UartConfigService process cycle.
* @param cfg_request Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-095 SW-REQ-083
*/
/**
* @brief Loaddefaults.
* @trace DD-COMP-095 SW-REQ-083
*/
/**
* @brief Validateparams.
```

```
 * @trace DD-COMP-095 SW-REQ-083
 */
 /**
 * @brief Applyconfig.
 * @trace DD-COMP-095 SW-REQ-083
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read requested UART settings
2	load default configuration
3	validate settings against limits
4	apply baud and frame settings
5	configure hardware registers
6	report configuration status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded magic values
2	blocking waits
3	invalid baud rates
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uartCfg' (e.g. uartCfgresult).`

2	State struct: static UartConfigService_t uartCfgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_095_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartConfigService_t;`
	State struct for UartConfigService

`void InitUartCfg(void);`	Prototype: initialise UartConfigService
`uint8_t ProcUartCfg(void *p_data);`	Prototype: run UartConfigService process cycle
`uint8_t LoadDefaults(void);`	Prototype: LoadDefaults
`uint8_t ValidateParams(void);`	Prototype: ValidateParams
`uint8_t ApplyConfig(void);`	Prototype: ApplyConfig

`typedef enum {`
`DD_COMP_095_ERR_NONE = 0x00U,`
`DD_COMP_095_ERR_NULL = 0x01U,`
`DD_COMP_095_ERR_INVALID = 0x02U,`
`DD_COMP_095_ERR_TIMEOUT = 0x03U`
`} DD_COMP_095_ERR_t;`
	Error code enum for UartConfigService

main.c - What to Add

Item	Purpose

`static UartConfigService_t uartCfgstate = {0};`	Module-level zero-initialised state for UartConfigService

`/* Process function logic order:`
`* Step 1: read requested UART settings`
`* Step 2: load default configuration`
`* Step 3: validate settings against limits`
`* Step 4: apply baud and frame settings`
`* Step 5: configure hardware registers`
`* Step 6: report configuration status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call InitUartCfg() once in main() init sequence. Call ProcUartCfg() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

98. DD-COMP-096 - UartBaudRateConfig

DD-COMP-096 **UartBaudRateConfig** | Algorithm | Module: CommsModule | Pattern: N/A | SW-REQ-083

`@trace DD-COMP-096 SW-REQ-083`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void uartBaudInit(void)`	void	Initialise all state for UartBaudRateConfig
main.c	`static uint8_t uartBaudCalc(void *clkHz)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartBaudRateConfig
main.c	`static uint8_t findDivisors(void)`	uint8_t	Helper function: findDivisors
main.c	`static uint8_t calcBaudError(void)`	uint8_t	Helper function: calcBaudError
main.c	`static uint8_t selectBestBaud(void)`	uint8_t	Helper function: selectBestBaud

Doxygen Comments

```
/**
* @brief Initialise UartBaudRateConfig.
* @trace DD-COMP-096 SW-REQ-083
*/
/**
* @brief Run UartBaudRateConfig process cycle.
* @param clkHz Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-096 SW-REQ-083
*/
/**
* @brief Finddivisors.
* @trace DD-COMP-096 SW-REQ-083
*/
/**
* @brief Calcbauderror.
```

```
 * @trace DD-COMP-096 SW-REQ-083
 */
 /**
 * @brief Selectbestbaud.
 * @trace DD-COMP-096 SW-REQ-083
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read clock and target baud inputs
2	compute candidate divisors
3	calculate achievable baud rates
4	evaluate error for each candidate
5	select best match
6	store configuration outputs

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded clock values
2	ignoring oversampling mode
3	selecting out-of-range divisors
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'baud' (e.g. baudresult).`

2	State struct: static UartBaudCfg_t baudstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_096_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartBaudCfg_t;`
	State struct for UartBaudRateConfig

`void uartBaudInit(void);`	Prototype: initialise UartBaudRateConfig
`uint8_t uartBaudCalc(void *p_data);`	Prototype: run UartBaudRateConfig process cycle
`uint8_t findDivisors(void);`	Prototype: findDivisors
`uint8_t calcBaudError(void);`	Prototype: calcBaudError
`uint8_t selectBestBaud(void);`	Prototype: selectBestBaud

`typedef enum {`
`DD_COMP_096_ERR_NONE = 0x00U,`
`DD_COMP_096_ERR_NULL = 0x01U,`
`DD_COMP_096_ERR_INVALID = 0x02U,`
`DD_COMP_096_ERR_TIMEOUT = 0x03U`
`} DD_COMP_096_ERR_t;`
	Error code enum for UartBaudRateConfig

main.c - What to Add

Item	Purpose

`static UartBaudCfg_t baudstate = {0};`	Module-level zero-initialised state for UartBaudRateConfig

`/* Process function logic order:`
`* Step 1: read clock and target baud inputs`
`* Step 2: compute candidate divisors`
`* Step 3: calculate achievable baud rates`
`* Step 4: evaluate error for each candidate`
`* Step 5: select best match`
`* Step 6: store configuration outputs`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call uartBaudInit() once in main() init sequence. Call uartBaudCalc() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

99. DD-COMP-097 - UartFrameFormatConfig

DD-COMP-097 **UartFrameFormatConfig** | Algorithm | Module: CommsModule | Pattern: N/A | SW-REQ-084

`@trace DD-COMP-097 SW-REQ-084`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void uart_fmt_init(void)`	void	Initialise all state for UartFrameFormatConfig
main.c	`static uint8_t uart_fmt_apply(void *data_bits)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartFrameFormatConfig
main.c	`static uint8_t calc_parity(void)`	uint8_t	Helper function: calc parity
main.c	`static uint8_t build_lcr(void)`	uint8_t	Helper function: build lcr
main.c	`static uint8_t validate_cfg(void)`	uint8_t	Helper function: validate cfg

Doxygen Comments

```
/**
* @brief Initialise UartFrameFormatConfig.
* @trace DD-COMP-097 SW-REQ-084
*/
/**
* @brief Run UartFrameFormatConfig process cycle.
* @param data_bits Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-097 SW-REQ-084
*/
/**
* @brief Calc parity.
* @trace DD-COMP-097 SW-REQ-084
*/
/**
* @brief Build lcr.
```

```
 * @trace DD-COMP-097 SW-REQ-084
 */
 /**
 * @brief Validate cfg.
 * @trace DD-COMP-097 SW-REQ-084
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load user frame settings
2	validate data bits, stop bits, and parity
3	compute parity control fields
4	compose line control register value
5	apply configuration to target
6	report status

Forbidden - Must NOT Do

#	Forbidden Rule
1	invalid bit ranges
2	mismatched parity settings
3	hardware register side effects
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uart' (e.g. uartresult).`

2	State struct: static UartFrameFormatConfig_t uartstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_097_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartFrameFormatConfig_t;`
	State struct for UartFrameFormatConfig

`void uart_fmt_init(void);`	Prototype: initialise UartFrameFormatConfig
`uint8_t uart_fmt_apply(void *p_data);`	Prototype: run UartFrameFormatConfig process cycle
`uint8_t calc_parity(void);`	Prototype: calc parity
`uint8_t build_lcr(void);`	Prototype: build lcr
`uint8_t validate_cfg(void);`	Prototype: validate cfg

`typedef enum {`
`DD_COMP_097_ERR_NONE = 0x00U,`
`DD_COMP_097_ERR_NULL = 0x01U,`
`DD_COMP_097_ERR_INVALID = 0x02U,`
`DD_COMP_097_ERR_TIMEOUT = 0x03U`
`} DD_COMP_097_ERR_t;`
	Error code enum for UartFrameFormatConfig

main.c - What to Add

Item	Purpose

`static UartFrameFormatConfig_t uartstate = {0};`	Module-level zero-initialised state for UartFrameFormatConfig

`/* Process function logic order:`
`* Step 1: load user frame settings`
`* Step 2: validate data bits, stop bits, and parity`
`* Step 3: compute parity control fields`
`* Step 4: compose line control register value`
`* Step 5: apply configuration to target`
`* Step 6: report status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call uart_fmt_init() once in main() init sequence. Call uart_fmt_apply() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

100. DD-COMP-098 - UartTransmitIntervalValidator

DD-COMP-098	**UartTransmitIntervalValidator** | Class | Module: CommsModule | Pattern: N/A | SW-REQ-085

`@trace DD-COMP-098 SW-REQ-085`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init(void)`	void	Initialise all state for UartTransmitIntervalValidator
main.c	`static uint8_t validate(void *currentTime)`	uint8_t - 0 success, non-zero error code	Execute core logic for UartTransmitIntervalValidator
main.c	`static uint8_t calcInterval(void)`	uint8_t	Helper function: calcInterval
main.c	`static uint8_t checkBounds(void)`	uint8_t	Helper function: checkBounds
main.c	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState

Doxygen Comments

```
/**
* @brief Initialise UartTransmitIntervalValidator.
* @trace DD-COMP-098 SW-REQ-085
*/
/**
* @brief Run UartTransmitIntervalValidator process cycle.
* @param currentTime Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-098 SW-REQ-085
*/
/**
* @brief Calcinterval.
* @trace DD-COMP-098 SW-REQ-085
*/
/**
* @brief Checkbounds.
```

```
 * @trace DD-COMP-098 SW-REQ-085
 */
 /**
 * @brief Updatestate.
 * @trace DD-COMP-098 SW-REQ-085
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load current timestamp
2	compute interval since last transmit
3	compare interval against min/max limits
4	flag valid or invalid result
5	store timestamp for next check

Forbidden - Must NOT Do

#	Forbidden Rule
1	global state
2	blocking delays
3	floating point math
4	implicit units

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uartInt' (e.g. uartIntresult).`
2	`State struct: static UartTransmitIntervalValidator_t uartIntstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_098_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UartTransmitIntervalValidator_t;`
	State struct for UartTransmitIntervalValidator

`void init(void);`	Prototype: initialise UartTransmitIntervalValidator
`uint8_t validate(void *p_data);`	Prototype: run UartTransmitIntervalValidator process cycle
`uint8_t calcInterval(void);`	Prototype: calcInterval
`uint8_t checkBounds(void);`	Prototype: checkBounds
`uint8_t updateState(void);`	Prototype: updateState

`typedef enum {`
`DD_COMP_098_ERR_NONE = 0x00U,`
`DD_COMP_098_ERR_NULL = 0x01U,`
`DD_COMP_098_ERR_INVALID = 0x02U,`
`DD_COMP_098_ERR_TIMEOUT = 0x03U`
`} DD_COMP_098_ERR_t;`
	Error code enum for UartTransmitIntervalValidator

main.c - What to Add

Item	Purpose
`static UartTransmitIntervalValidator_t uartIntstate = {0};`	Module-level zero-initialised state for UartTransmitIntervalValidator

`/* Process function logic order:`
`* Step 1: load current timestamp`
`* Step 2: compute interval since last transmit`
`* Step 3: compare interval against min/max limits`
`* Step 4: flag valid or invalid result`
`* Step 5: store timestamp for next check`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call init() once in main() init sequence. Call validate() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

101. DD-COMP-099 - OvercurrentMonitor

DD-COMP-099	**OvercurrentMonitor** | Service | Module: SafetyModule | Pattern: Observer | SW-REQ-086

`@trace DD-COMP-099 SW-REQ-086`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void ocmInit(void)`	void	Initialise all state for OvercurrentMonitor
main.c	`static uint8_t ocmProcess(void *currentReading)`	uint8_t - 0 success, non-zero error code	Execute core logic for OvercurrentMonitor
main.c	`static uint8_t ocmCheckLevel(void)`	uint8_t	Helper function: ocmCheckLevel
main.c	`static uint8_t ocmLatchFault(void)`	uint8_t	Helper function: ocmLatchFault
main.c	`static uint8_t ocmClearFault(void)`	uint8_t	Helper function: ocmClearFault

Doxygen Comments

```
/**
* @brief Initialise OvercurrentMonitor.
* @trace DD-COMP-099 SW-REQ-086
*/
/**
* @brief Run OvercurrentMonitor process cycle.
* @param currentReading Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-099 SW-REQ-086
*/
/**
* @brief Ocmchecklevel.
* @trace DD-COMP-099 SW-REQ-086
*/
/**
* @brief Ocmlatchfault.
```

```
 * @trace DD-COMP-099 SW-REQ-086
 */
 /**
 * @brief Ocmclearfault.
 * @trace DD-COMP-099 SW-REQ-086
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	Read current sensor value
2	Filter or validate the sample
3	Compare against overcurrent threshold
4	Latch fault if limit exceeded
5	Update status outputs
6	Handle reset or clear condition

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	magic numbers
3	uncleared latches
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ocm' (e.g. ocmresult).`

2	State struct: static OvercurrentMonitor_t ocmstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_099_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} OvercurrentMonitor_t;`
	State struct for OvercurrentMonitor

`void ocmInit(void);`	Prototype: initialise OvercurrentMonitor
`uint8_t ocmProcess(void *p_data);`	Prototype: run OvercurrentMonitor process cycle
`uint8_t ocmCheckLevel(void);`	Prototype: ocmCheckLevel
`uint8_t ocmLatchFault(void);`	Prototype: ocmLatchFault
`uint8_t ocmClearFault(void);`	Prototype: ocmClearFault

`typedef enum {`
`DD_COMP_099_ERR_NONE = 0x00U,`
`DD_COMP_099_ERR_NULL = 0x01U,`
`DD_COMP_099_ERR_INVALID = 0x02U,`
`DD_COMP_099_ERR_TIMEOUT = 0x03U`
`} DD_COMP_099_ERR_t;`
	Error code enum for OvercurrentMonitor

main.c - What to Add

Item	Purpose

`static OvercurrentMonitor_t ocmstate = {0};`	Module-level zero-initialised state for OvercurrentMonitor

`/* Process function logic order:`
`* Step 1: Read current sensor value`
`* Step 2: Filter or validate the sample`
`* Step 3: Compare against overcurrent threshold`
`* Step 4: Latch fault if limit exceeded`
`* Step 5: Update status outputs`
`* Step 6: Handle reset or clear condition`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call ocmInit() once in main() init sequence. Call ocmProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

102. DD-COMP-100 - ThermalMonitor

DD-COMP-100	**ThermalMonitor** | Service | Module: SafetyModule | Pattern: Observer | SW-REQ-087

`@trace DD-COMP-100 SW-REQ-087`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void thermInit(void)`	void	Initialise all state for ThermalMonitor
`main.c`	`static uint8_t thermProcess(void *tempSensors)`	uint8_t - 0 success, non-zero error code	Execute core logic for ThermalMonitor
`main.c`	`static uint8_t readSensors(void)`	uint8_t	Helper function: readSensors
`main.c`	`static uint8_t calcTrend(void)`	uint8_t	Helper function: calcTrend
`main.c`	`static uint8_t checkLimits(void)`	uint8_t	Helper function: checkLimits

Doxygen Comments

```
/**
* @brief Initialise ThermalMonitor.
* @trace DD-COMP-100 SW-REQ-087
*/
/**
* @brief Run ThermalMonitor process cycle.
* @param tempSensors Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-100 SW-REQ-087
*/
/**
* @brief Readsensors.
* @trace DD-COMP-100 SW-REQ-087
*/
/**
* @brief Calctrend.
```

```
 * @trace DD-COMP-100 SW-REQ-087
 */
 /**
 * @brief Checklimits.
 * @trace DD-COMP-100 SW-REQ-087
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	read temperature inputs
3	filter and average readings
4	evaluate limits and trends
5	update status and alarms
6	publish outputs

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	magic constants
3	global mutable state
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'therm' (e.g. thermresult).`

2	State struct: static ThermalMonitor_t thermstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_100_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ThermalMonitor_t;`
	State struct for ThermalMonitor

`void thermInit(void);`	Prototype: initialise ThermalMonitor
`uint8_t thermProcess(void *p_data);`	Prototype: run ThermalMonitor process cycle
`uint8_t readSensors(void);`	Prototype: readSensors
`uint8_t calcTrend(void);`	Prototype: calcTrend
`uint8_t checkLimits(void);`	Prototype: checkLimits

`typedef enum {`
`DD_COMP_100_ERR_NONE = 0x00U,`
`DD_COMP_100_ERR_NULL = 0x01U,`
`DD_COMP_100_ERR_INVALID = 0x02U,`
`DD_COMP_100_ERR_TIMEOUT = 0x03U`
`} DD_COMP_100_ERR_t;`
	Error code enum for ThermalMonitor

main.c - What to Add

Item	Purpose

`static ThermalMonitor_t thermstate = {0};`	Module-level zero-initialised state for ThermalMonitor

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: read temperature inputs`
`* Step 3: filter and average readings`
`* Step 4: evaluate limits and trends`
`* Step 5: update status and alarms`
`* Step 6: publish outputs`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call thermInit() once in main() init sequence. Call thermProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

103. DD-COMP-101 - StallDetectionService

DD-COMP-101	**StallDetectionService** | Service | Module: SafetyModule | Pattern: State | SW-REQ-088

`@trace DD-COMP-101 SW-REQ-088`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initStallDetect(void)`	void	Initialise all state for StallDetectionService
main.c	`static uint8_t runStallDetect(void *speed)`	uint8_t - 0 success, non-zero error code	Execute core logic for StallDetectionService
main.c	`static uint8_t readSensors(void)`	uint8_t	Helper function: readSensors
main.c	`static uint8_t checkStall(void)`	uint8_t	Helper function: checkStall
main.c	`static uint8_t resetStall(void)`	uint8_t	Helper function: resetStall

Doxygen Comments

```
/**
* @brief Initialise StallDetectionService.
* @trace DD-COMP-101 SW-REQ-088
*/
/**
* @brief Run StallDetectionService process cycle.
* @param speed Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-101 SW-REQ-088
*/
/**
* @brief Readsensors.
* @trace DD-COMP-101 SW-REQ-088
*/
/**
* @brief Checkstall.
```

```
 * @trace DD-COMP-101 SW-REQ-088
 */
 /**
 * @brief Resetstall.
 * @trace DD-COMP-101 SW-REQ-088
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration thresholds
2	poll motor speed and current
3	compute stall indicators
4	compare against thresholds
5	update stall state
6	emit stall status
7	log events

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	magic thresholds
3	unbounded logs
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'stall' (e.g. stallresult).
2	State struct: static StallDetectionService_t stallstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_101_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} StallDetectionService_t;`
	State struct for StallDetectionService

`void initStallDetect(void);`	Prototype: initialise StallDetectionService
`uint8_t runStallDetect(void *p_data);`	Prototype: run StallDetectionService process cycle
`uint8_t readSensors(void);`	Prototype: readSensors
`uint8_t checkStall(void);`	Prototype: checkStall
`uint8_t resetStall(void);`	Prototype: resetStall

`typedef enum {`
`DD_COMP_101_ERR_NONE = 0x00U,`
`DD_COMP_101_ERR_NULL = 0x01U,`
`DD_COMP_101_ERR_INVALID = 0x02U,`
`DD_COMP_101_ERR_TIMEOUT = 0x03U`
`} DD_COMP_101_ERR_t;`
	Error code enum for StallDetectionService

main.c - What to Add

Item	Purpose
`static StallDetectionService_t stallstate = {0};`	Module-level zero-initialised state for StallDetectionService

`/* Process function logic order:`
`* Step 1: load configuration thresholds`
`* Step 2: poll motor speed and current`
`* Step 3: compute stall indicators`
`* Step 4: compare against thresholds`
`* Step 5: update stall state`
`* Step 6: emit stall status`
`* Step 7: log events`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initStallDetect() once in main() init sequence. Call runStallDetect() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

104. DD-COMP-102 - ShortCircuitDetector

DD-COMP-102	**ShortCircuitDetector** | Class | Module: FaultMonitoringModule | Pattern: Observer | SW-REQ-089

`@trace DD-COMP-102 SW-REQ-089`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init_sc(void)`	void	Initialise all state for ShortCircuitDetector
`main.c`	`static uint8_t run_scan(void *current_samples)`	uint8_t - 0 success, non-zero error code	Execute core logic for ShortCircuitDetector
`main.c`	`static uint8_t read_samples(void)`	uint8_t	Helper function: read samples
`main.c`	`static uint8_t check_thresholds(void)`	uint8_t	Helper function: check thresholds
`main.c`	`static uint8_t flag_fault(void)`	uint8_t	Helper function: flag fault

Doxygen Comments

```
/**
* @brief Initialise ShortCircuitDetector.
* @trace DD-COMP-102 SW-REQ-089
*/
/**
* @brief Run ShortCircuitDetector process cycle.
* @param current_samples Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-102 SW-REQ-089
*/
/**
* @brief Read samples.
* @trace DD-COMP-102 SW-REQ-089
*/
/**
```

```
* @brief Check thresholds.
* @trace DD-COMP-102 SW-REQ-089
*/
/**
* @brief Flag fault.
* @trace DD-COMP-102 SW-REQ-089
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration parameters
2	acquire current and voltage samples
3	compute instantaneous power and deltas
4	compare against short-circuit thresholds
5	apply debounce or time window
6	set fault status and record event

Forbidden - Must NOT Do

#	Forbidden Rule
1	hard-coded thresholds
2	blocking I/O in main loop
3	silent failures

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'sc' (e.g. scresult).`
2	`State struct: static ShortCircuitDetector_t scstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_102_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ShortCircuitDetector_t;`
	State struct for ShortCircuitDetector

`void init_sc(void);`	Prototype: initialise ShortCircuitDetector
`uint8_t run_scan(void *p_data);`	Prototype: run ShortCircuitDetector process cycle
`uint8_t read_samples(void);`	Prototype: read samples
`uint8_t check_thresholds(void);`	Prototype: check thresholds
`uint8_t flag_fault(void);`	Prototype: flag fault

`typedef enum {`
`DD_COMP_102_ERR_NONE = 0x00U,`
`DD_COMP_102_ERR_NULL = 0x01U,`
`DD_COMP_102_ERR_INVALID = 0x02U,`
`DD_COMP_102_ERR_TIMEOUT = 0x03U`
`} DD_COMP_102_ERR_t;`
	Error code enum for ShortCircuitDetector

main.c - What to Add

Item	Purpose
`static ShortCircuitDetector_t scstate = {0};`	Module-level zero-initialised state for ShortCircuitDetector

`/* Process function logic order:`
`* Step 1: load configuration parameters`
`* Step 2: acquire current and voltage samples`
`* Step 3: compute instantaneous power and deltas`
`* Step 4: compare against short-circuit thresholds`
`* Step 5: apply debounce or time window`
`* Step 6: set fault status and record event`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call init_sc() once in main() init sequence. Call run_scan() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

105. DD-COMP-103 - UndervoltageMonitor

DD-COMP-103	**UndervoltageMonitor** | Class | Module: PowerMonitoringModule | Pattern: Observer | SW-REQ-090

`@trace DD-COMP-103 SW-REQ-090`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void uvm_init(void)`	void	Initialise all state for UndervoltageMonitor
main.c	`static uint8_t uvm_run(void *voltage_sample)`	uint8_t - 0 success, non-zero error code	Execute core logic for UndervoltageMonitor
main.c	`static uint8_t uvm_read(void)`	uint8_t	Helper function: uvm read
main.c	`static uint8_t uvm_check(void)`	uint8_t	Helper function: uvm check
main.c	`static uint8_t uvm_latch(void)`	uint8_t	Helper function: uvm latch

Doxygen Comments

```
/**
* @brief Initialise UndervoltageMonitor.
* @trace DD-COMP-103 SW-REQ-090
*/
/**
* @brief Run UndervoltageMonitor process cycle.
* @param voltage_sample Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-103 SW-REQ-090
*/
/**
* @brief Uvm read.
* @trace DD-COMP-103 SW-REQ-090
*/
/**
```

```
 * @brief Uvm check.
 * @trace DD-COMP-103 SW-REQ-090
 */
 /**
 * @brief Uvm latch.
 * @trace DD-COMP-103 SW-REQ-090
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration thresholds
2	sample input voltage
3	compare against limits
4	update undervoltage state
5	apply debounce or delay
6	report status change

Forbidden - Must NOT Do

#	Forbidden Rule
1	magic numbers
2	blocking delays
3	unbounded state changes

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'uvm' (e.g. uvmresult).`
2	`State struct: static UndervoltageMonitor_t uvmstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_103_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UndervoltageMonitor_t;`
	State struct for UndervoltageMonitor

`void uvm_init(void);`	Prototype: initialise UndervoltageMonitor
`uint8_t uvm_run(void *p_data);`	Prototype: run UndervoltageMonitor process cycle
`uint8_t uvm_read(void);`	Prototype: uvm read
`uint8_t uvm_check(void);`	Prototype: uvm check
`uint8_t uvm_latch(void);`	Prototype: uvm latch

`typedef enum {`
`DD_COMP_103_ERR_NONE = 0x00U,`
`DD_COMP_103_ERR_NULL = 0x01U,`
`DD_COMP_103_ERR_INVALID = 0x02U,`
`DD_COMP_103_ERR_TIMEOUT = 0x03U`
`} DD_COMP_103_ERR_t;`
	Error code enum for UndervoltageMonitor

main.c - What to Add

Item	Purpose
`static UndervoltageMonitor_t uvmstate = {0};`	Module-level zero-initialised state for UndervoltageMonitor

`/* Process function logic order:`
`* Step 1: load configuration thresholds`
`* Step 2: sample input voltage`
`* Step 3: compare against limits`
`* Step 4: update undervoltage state`
`* Step 5: apply debounce or delay`
`* Step 6: report status change`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call uvm_init() once in main() init sequence. Call uvm_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

106. DD-COMP-104 - WatchdogService

DD-COMP-104	**WatchdogService** | Service | Module: ReliabilityModule | Pattern: Singleton | SW-REQ-091

`@trace DD-COMP-104 SW-REQ-091`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void wdog_init(void)`	void	Initialise all state for WatchdogService
`main.c`	`static uint8_t wdog_run(void *config)`	uint8_t - 0 success, non-zero error code	Execute core logic for WatchdogService
`main.c`	`static uint8_t wdog_check(void)`	uint8_t	Helper function: wdog check
`main.c`	`static uint8_t wdog_kick(void)`	uint8_t	Helper function: wdog kick
`main.c`	`static uint8_t wdog_log(void)`	uint8_t	Helper function: wdog log

Doxygen Comments

```
/**
* @brief Initialise WatchdogService.
* @trace DD-COMP-104 SW-REQ-091
*/
/**
* @brief Run WatchdogService process cycle.
* @param config Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-104 SW-REQ-091
*/
/**
* @brief Wdog check.
* @trace DD-COMP-104 SW-REQ-091
*/
/**
* @brief Wdog kick.
```

```
 * @trace DD-COMP-104 SW-REQ-091
 */
 /**
 * @brief Wdog log.
 * @trace DD-COMP-104 SW-REQ-091
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	monitor heartbeat
3	detect timeout
4	trigger recovery
5	log status
6	update counters

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking waits
2	global state
3	silent failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'wd' (e.g. wdresult).`

2	State struct: static WatchdogService_t wdstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_104_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} WatchdogService_t;`
	State struct for WatchdogService

`void wdog_init(void);`	Prototype: initialise WatchdogService
`uint8_t wdog_run(void *p_data);`	Prototype: run WatchdogService process cycle
`uint8_t wdog_check(void);`	Prototype: wdog check
`uint8_t wdog_kick(void);`	Prototype: wdog kick
`uint8_t wdog_log(void);`	Prototype: wdog log

`typedef enum {`
`DD_COMP_104_ERR_NONE = 0x00U,`
`DD_COMP_104_ERR_NULL = 0x01U,`
`DD_COMP_104_ERR_INVALID = 0x02U,`
`DD_COMP_104_ERR_TIMEOUT = 0x03U`
`} DD_COMP_104_ERR_t;`
	Error code enum for WatchdogService

main.c - What to Add

Item	Purpose

`static WatchdogService_t wdstate = {0};`	Module-level zero-initialised state for WatchdogService

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: monitor heartbeat`
`* Step 3: detect timeout`
`* Step 4: trigger recovery`
`* Step 5: log status`
`* Step 6: update counters`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call wdog_init() once in main() init sequence. Call wdog_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

107. DD-COMP-105 - FaultStateTransitionHandler

DD-COMP-105	**FaultStateTransitionHandler** | Class | Module: StateMachineModule | Pattern: State | SW-REQ-092

`@trace DD-COMP-105 SW-REQ-092`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initHandler(void)`	void	Initialise all state for FaultStateTransitionHandler
main.c	`static uint8_t processTransitions(void *fault_event)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultStateTransitionHandler
main.c	`static uint8_t validateState(void)`	uint8_t	Helper function: validateState
main.c	`static uint8_t applyTransition(void)`	uint8_t	Helper function: applyTransition
main.c	`static uint8_t logFault(void)`	uint8_t	Helper function: logFault

Doxygen Comments

```
/**
* @brief Initialise FaultStateTransitionHandler.
* @trace DD-COMP-105 SW-REQ-092
*/
/**
* @brief Run FaultStateTransitionHandler process cycle.
* @param fault_event Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-105 SW-REQ-092
*/
/**
* @brief Validatestate.
* @trace DD-COMP-105 SW-REQ-092
*/
/**
```

```
 * @brief Applytransition.
 * @trace DD-COMP-105 SW-REQ-092
 */
 /**
 * @brief Logfault.
 * @trace DD-COMP-105 SW-REQ-092
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load current fault state
2	check incoming fault event
3	validate allowed transition
4	update state and timestamps
5	emit transition output
6	record diagnostics

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded state IDs
2	silent failures
3	blocking operations

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fst' (e.g. fstresult).`
2	`State struct: static FaultStateTransitionHandler_t fststate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_105_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultStateTransitionHandler_t;`
	State struct for FaultStateTransitionHandler

`void initHandler(void);`	Prototype: initialise FaultStateTransitionHandler
`uint8_t processTransitions(void *p_data);`	Prototype: run FaultStateTransitionHandler process cycle
`uint8_t validateState(void);`	Prototype: validateState
`uint8_t applyTransition(void);`	Prototype: applyTransition
`uint8_t logFault(void);`	Prototype: logFault

`typedef enum {`
`DD_COMP_105_ERR_NONE = 0x00U,`
`DD_COMP_105_ERR_NULL = 0x01U,`
`DD_COMP_105_ERR_INVALID = 0x02U,`
`DD_COMP_105_ERR_TIMEOUT = 0x03U`
`} DD_COMP_105_ERR_t;`
	Error code enum for FaultStateTransitionHandler

main.c - What to Add

Item	Purpose
`static FaultStateTransitionHandler_t fststate = {0};`	Module-level zero-initialised state for FaultStateTransitionHandler

`/* Process function logic order:`
`* Step 1: load current fault state`
`* Step 2: check incoming fault event`
`* Step 3: validate allowed transition`
`* Step 4: update state and timestamps`
`* Step 5: emit transition output`
`* Step 6: record diagnostics`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initHandler() once in main() init sequence. Call processTransitions() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

108. DD-COMP-106 - TemperatureProcessing

DD-COMP-106	**TemperatureProcessing** | Algorithm | Module: SensorProcessingModule | Pattern: N/A | SW-REQ-093

`@trace DD-COMP-106 SW-REQ-093`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void tempProcInit(void)`	void	Initialise all state for TemperatureProcessing
`main.c`	`static uint8_t tempProcRun(void *tempSamples)`	uint8_t - 0 success, non-zero error code	Execute core logic for TemperatureProcessing
`main.c`	`static uint8_t loadTemps(void)`	uint8_t	Helper function: loadTemps
`main.c`	`static uint8_t filterTemps(void)`	uint8_t	Helper function: filterTemps
`main.c`	`static uint8_t calcStats(void)`	uint8_t	Helper function: calcStats

Doxygen Comments

```
/**
* @brief Initialise TemperatureProcessing.
* @trace DD-COMP-106 SW-REQ-093
*/
/**
* @brief Run TemperatureProcessing process cycle.
* @param tempSamples Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-106 SW-REQ-093
*/
/**
* @brief Loadtemps.
* @trace DD-COMP-106 SW-REQ-093
*/
/**
```

```
 * @brief Filtertemps.
 * @trace DD-COMP-106 SW-REQ-093
 */
 /**
 * @brief Calcstats.
 * @trace DD-COMP-106 SW-REQ-093
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize state and buffers
2	ingest raw temperature samples
3	filter or smooth readings
4	detect outliers or invalid values
5	compute summary metrics
6	update internal state
7	prepare outputs

Forbidden - Must NOT Do

#	Forbidden Rule
1	avoid hard-coded thresholds
2	avoid blocking I/O
3	avoid mutable global state
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'temp' (e.g. tempresult).
2	State struct: static TemperatureProcessing_t tempstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_106_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} TemperatureProcessing_t;`
	State struct for TemperatureProcessing

`void tempProcInit(void);`	Prototype: initialise TemperatureProcessing
`uint8_t tempProcRun(void *p_data);`	Prototype: run TemperatureProcessing process cycle
`uint8_t loadTemps(void);`	Prototype: loadTemps
`uint8_t filterTemps(void);`	Prototype: filterTemps
`uint8_t calcStats(void);`	Prototype: calcStats

`typedef enum {`
`DD_COMP_106_ERR_NONE = 0x00U,`
`DD_COMP_106_ERR_NULL = 0x01U,`
`DD_COMP_106_ERR_INVALID = 0x02U,`
`DD_COMP_106_ERR_TIMEOUT = 0x03U`
`} DD_COMP_106_ERR_t;`
	Error code enum for TemperatureProcessing

main.c - What to Add

Item	Purpose
`static TemperatureProcessing_t tempstate = {0};`	Module-level zero-initialised state for TemperatureProcessing

`/* Process function logic order:`
`* Step 1: initialize state and buffers`
`* Step 2: ingest raw temperature samples`
`* Step 3: filter or smooth readings`
`* Step 4: detect outliers or invalid values`
`* Step 5: compute summary metrics`
`* Step 6: update internal state`
`* Step 7: prepare outputs`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call tempProcInit() once in main() init sequence. Call tempProcRun() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

109. DD-COMP-107 - NvmConfigStorage

DD-COMP-107	**NvmConfigStorage** | Service | Module: DataPersistenceModule | Pattern: Repository | SW-REQ-094

`@trace DD-COMP-107 SW-REQ-094`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void cfgStoreInit(void)`	void	Initialise all state for NvmConfigStorage
`main.c`	`static uint8_t cfgStoreProcess(void *nvmHandle)`	uint8_t - 0 success, non-zero error code	Execute core logic for NvmConfigStorage
`main.c`	`static uint8_t loadFromNvm(void)`	uint8_t	Helper function: loadFromNvm
`main.c`	`static uint8_t validateConfig(void)`	uint8_t	Helper function: validateConfig
`main.c`	`static uint8_t saveToNvm(void)`	uint8_t	Helper function: saveToNvm

Doxygen Comments

```
/**
* @brief Initialise NvmConfigStorage.
* @trace DD-COMP-107 SW-REQ-094
*/
/**
* @brief Run NvmConfigStorage process cycle.
* @param nvmHandle Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-107 SW-REQ-094
*/
/**
* @brief Loadfromnvm.
* @trace DD-COMP-107 SW-REQ-094
*/
/**
```

```
 * @brief Validateconfig.
 * @trace DD-COMP-107 SW-REQ-094
 */
 /**
 * @brief Savetonvm.
 * @trace DD-COMP-107 SW-REQ-094
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize storage context
2	read config blob from NVM
3	verify integrity and version
4	load defaults if invalid
5	apply runtime updates
6	write back changes
7	report status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking long NVM writes
2	silent data corruption
3	unbounded retries
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'cfg' (e.g. cfgresult).
2	State struct: static NvmConfigStorage_t cfgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_107_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} NvmConfigStorage_t;`
	State struct for NvmConfigStorage

`void cfgStoreInit(void);`	Prototype: initialise NvmConfigStorage
`uint8_t cfgStoreProcess(void *p_data);`	Prototype: run NvmConfigStorage process cycle
`uint8_t loadFromNvm(void);`	Prototype: loadFromNvm
`uint8_t validateConfig(void);`	Prototype: validateConfig
`uint8_t saveToNvm(void);`	Prototype: saveToNvm

`typedef enum {`
`DD_COMP_107_ERR_NONE = 0x00U,`
`DD_COMP_107_ERR_NULL = 0x01U,`
`DD_COMP_107_ERR_INVALID = 0x02U,`
`DD_COMP_107_ERR_TIMEOUT = 0x03U`
`} DD_COMP_107_ERR_t;`
	Error code enum for NvmConfigStorage

main.c - What to Add

Item	Purpose

`static NvmConfigStorage_t cfgstate = {0};`	Module-level zero-initialised state for NvmConfigStorage

`/* Process function logic order:`
`* Step 1: initialize storage context`
`* Step 2: read config blob from NVM`
`* Step 3: verify integrity and version`
`* Step 4: load defaults if invalid`
`* Step 5: apply runtime updates`
`* Step 6: write back changes`
`* Step 7: report status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call cfgStoreInit() once in main() init sequence. Call cfgStoreProcess() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

110. DD-COMP-108 - HumidityIndependencePolicy

DD-COMP-108	**HumidityIndependencePolicy** | Class | Module: SystemConstraintsModule | Pattern: N/A | SW-REQ-095

`@trace DD-COMP-108 SW-REQ-095`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initPolicy(void)`	void	Initialise all state for HumidityIndependencePolicy
main.c	`static uint8_t applyPolicy(void *humidity)`	uint8_t - 0 success, non-zero error code	Execute core logic for HumidityIndependencePolicy
main.c	`static uint8_t loadThresholds(void)`	uint8_t	Helper function: loadThresholds
main.c	`static uint8_t checkHumidity(void)`	uint8_t	Helper function: checkHumidity
main.c	`static uint8_t computeAdjust(void)`	uint8_t	Helper function: computeAdjust

Doxygen Comments

```
/**
* @brief Initialise HumidityIndependencePolicy.
* @trace DD-COMP-108 SW-REQ-095
*/
/**
* @brief Run HumidityIndependencePolicy process cycle.
* @param humidity Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-108 SW-REQ-095
*/
/**
* @brief Loadthresholds.
* @trace DD-COMP-108 SW-REQ-095
*/
/**
```

```
* @brief Checkhumidity.
* @trace DD-COMP-108 SW-REQ-095
*/
/**
* @brief Computeadjust.
* @trace DD-COMP-108 SW-REQ-095
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration thresholds
2	read current humidity
3	evaluate independence criteria
4	compute adjustment factors
5	apply policy decision
6	store/update output state

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded magic numbers
2	side effects on input state
3	unused configuration fields

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'hip' (e.g. hipresult).`
2	`State struct: static HumidityIndependencePolicy_t hipstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_108_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} HumidityIndependencePolicy_t;`
	State struct for HumidityIndependencePolicy

`void initPolicy(void);`	Prototype: initialise HumidityIndependencePolicy
`uint8_t applyPolicy(void *p_data);`	Prototype: run HumidityIndependencePolicy process cycle
`uint8_t loadThresholds(void);`	Prototype: loadThresholds
`uint8_t checkHumidity(void);`	Prototype: checkHumidity
`uint8_t computeAdjust(void);`	Prototype: computeAdjust

`typedef enum {`
`DD_COMP_108_ERR_NONE = 0x00U,`
`DD_COMP_108_ERR_NULL = 0x01U,`
`DD_COMP_108_ERR_INVALID = 0x02U,`
`DD_COMP_108_ERR_TIMEOUT = 0x03U`
`} DD_COMP_108_ERR_t;`
	Error code enum for HumidityIndependencePolicy

main.c - What to Add

Item	Purpose
`static HumidityIndependencePolicy_t hipstate = {0};`	Module-level zero-initialised state for HumidityIndependencePolicy

`/* Process function logic order:`
`* Step 1: load configuration thresholds`
`* Step 2: read current humidity`
`* Step 3: evaluate independence criteria`
`* Step 4: compute adjustment factors`
`* Step 5: apply policy decision`
`* Step 6: store/update output state`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: N/A. Call initPolicy() once in main() init sequence. Call applyPolicy() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

111. DD-COMP-109 - AdcAcquisitionService

DD-COMP-109	**AdcAcquisitionService** | Service | Module: AcquisitionModule | Pattern: Producer-Consumer | SW-REQ-096

`@trace DD-COMP-109 SW-REQ-096`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initAdcAcq(void)`	void	Initialise all state for AdcAcquisitionService
main.c	`static uint8_t runAdcAcq(void *adcConfig)`	uint8_t - 0 success, non-zero error code	Execute core logic for AdcAcquisitionService
main.c	`static uint8_t readAdc(void)`	uint8_t	Helper function: readAdc
main.c	`static uint8_t filterSample(void)`	uint8_t	Helper function: filterSample
main.c	`static uint8_t storeSample(void)`	uint8_t	Helper function: storeSample

Doxygen Comments

```
/**
* @brief Initialise AdcAcquisitionService.
* @trace DD-COMP-109 SW-REQ-096
*/
/**
* @brief Run AdcAcquisitionService process cycle.
* @param adcConfig Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-109 SW-REQ-096
*/
/**
* @brief Readadc.
* @trace DD-COMP-109 SW-REQ-096
*/
/**
```

```
* @brief Filtersample.
* @trace DD-COMP-109 SW-REQ-096
*/
/**
* @brief Storesample.
* @trace DD-COMP-109 SW-REQ-096
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize ADC channels and buffers
2	trigger ADC conversion
3	read raw samples
4	apply basic filtering or scaling
5	store samples to buffer
6	notify data availability

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking in ISR
2	dynamic memory allocation
3	hard-coded channel counts
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adc' (e.g. adcresult).`

2	State struct: static AdcAcquisitionService_t adcstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_109_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcAcquisitionService_t;`
	State struct for AdcAcquisitionService

`void initAdcAcq(void);`	Prototype: initialise AdcAcquisitionService
`uint8_t runAdcAcq(void *p_data);`	Prototype: run AdcAcquisitionService process cycle
`uint8_t readAdc(void);`	Prototype: readAdc
`uint8_t filterSample(void);`	Prototype: filterSample
`uint8_t storeSample(void);`	Prototype: storeSample

`typedef enum {`
`DD_COMP_109_ERR_NONE = 0x00U,`
`DD_COMP_109_ERR_NULL = 0x01U,`
`DD_COMP_109_ERR_INVALID = 0x02U,`
`DD_COMP_109_ERR_TIMEOUT = 0x03U`
`} DD_COMP_109_ERR_t;`
	Error code enum for AdcAcquisitionService

main.c - What to Add

Item	Purpose

`static AdcAcquisitionService_t adcstate = {0};`	Module-level zero-initialised state for AdcAcquisitionService

`/* Process function logic order:`
`* Step 1: initialize ADC channels and buffers`
`* Step 2: trigger ADC conversion`
`* Step 3: read raw samples`
`* Step 4: apply basic filtering or scaling`
`* Step 5: store samples to buffer`
`* Step 6: notify data availability`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Producer-Consumer. Call initAdcAcq() once in main() init sequence. Call runAdcAcq() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

112. DD-COMP-110 - ControlLoopScheduler

DD-COMP-110	**ControlLoopScheduler** | Service | Module: ControlModule | Pattern: Observer | SW-REQ-097

`@trace DD-COMP-110 SW-REQ-097`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initLoop(void)`	void	Initialise all state for ControlLoopScheduler
main.c	`static uint8_t runLoop(void *taskQueue)`	uint8_t - 0 success, non-zero error code	Execute core logic for ControlLoopScheduler
main.c	`static uint8_t calcNextTick(void)`	uint8_t	Helper function: calcNextTick
main.c	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState
main.c	`static uint8_t dispatchTasks(void)`	uint8_t	Helper function: dispatchTasks

Doxygen Comments

```
/**
* @brief Initialise ControlLoopScheduler.
* @trace DD-COMP-110 SW-REQ-097
*/
/**
* @brief Run ControlLoopScheduler process cycle.
* @param taskQueue Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-110 SW-REQ-097
*/
/**
* @brief Calcnexttick.
* @trace DD-COMP-110 SW-REQ-097
*/
/**
* @brief Updatestate.
```

```
 * @trace DD-COMP-110 SW-REQ-097
 */
 /**
 * @brief Dispatchtasks.
 * @trace DD-COMP-110 SW-REQ-097
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	initialize timing state
3	compute next tick deadline
4	check ready tasks
5	dispatch due tasks
6	update loop state
7	sleep until next tick

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking long sleeps
2	unbounded task growth
3	timer drift
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'loop' (e.g. loopresult).
2	State struct: static ControlLoopScheduler_t loopstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_110_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ControlLoopScheduler_t;`
	State struct for ControlLoopScheduler

`void initLoop(void);`	Prototype: initialise ControlLoopScheduler
`uint8_t runLoop(void *p_data);`	Prototype: run ControlLoopScheduler process cycle
`uint8_t calcNextTick(void);`	Prototype: calcNextTick
`uint8_t updateState(void);`	Prototype: updateState
`uint8_t dispatchTasks(void);`	Prototype: dispatchTasks

`typedef enum {`
`DD_COMP_110_ERR_NONE = 0x00U,`
`DD_COMP_110_ERR_NULL = 0x01U,`
`DD_COMP_110_ERR_INVALID = 0x02U,`
`DD_COMP_110_ERR_TIMEOUT = 0x03U`
`} DD_COMP_110_ERR_t;`
	Error code enum for ControlLoopScheduler

main.c - What to Add

Item	Purpose
`static ControlLoopScheduler_t loopstate = {0};`	Module-level zero-initialised state for ControlLoopScheduler

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: initialize timing state`
`* Step 3: compute next tick deadline`
`* Step 4: check ready tasks`
`* Step 5: dispatch due tasks`
`* Step 6: update loop state`
`* Step 7: sleep until next tick`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initLoop() once in main() init sequence. Call runLoop() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

113. DD-COMP-111 - ADCReader

DD-COMP-111	**ADCReader** | Class | Module: SensorModule | Pattern: Singleton | SW-REQ-097

`@trace DD-COMP-111 SW-REQ-097`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void adc_init(void)`	void	Initialise all state for ADCReader
main.c	`static uint8_t adc_read(void *adc_channel)`	uint8_t - 0 success, non-zero error code	Execute core logic for ADCReader
main.c	`static uint8_t adc_config(void)`	uint8_t	Helper function: adc config
main.c	`static uint8_t adc_start(void)`	uint8_t	Helper function: adc start
main.c	`static uint8_t adc_convert(void)`	uint8_t	Helper function: adc convert

Doxygen Comments

```
/**
* @brief Initialise ADCReader.
* @trace DD-COMP-111 SW-REQ-097
*/
/**
* @brief Run ADCReader process cycle.
* @param adc_channel Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-111 SW-REQ-097
*/
/**
* @brief Adc config.
* @trace DD-COMP-111 SW-REQ-097
*/
/**
* @brief Adc start.
* @trace DD-COMP-111 SW-REQ-097
```

```
*/
/**
* @brief Adc convert.
* @trace DD-COMP-111 SW-REQ-097
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	validate configuration
2	configure ADC hardware
3	start conversion
4	wait for completion
5	read raw value
6	apply scaling
7	store result
8	signal availability

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	hardcoded constants
3	ignoring error codes

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'adc' (e.g. adcresult).`
2	`State struct: static AdcReader_t adcstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`

5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_111_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} AdcReader_t;`
	State struct for ADCReader

`void adc_init(void);`	Prototype: initialise ADCReader
`uint8_t adc_read(void *p_data);`	Prototype: run ADCReader process cycle
`uint8_t adc_config(void);`	Prototype: adc config
`uint8_t adc_start(void);`	Prototype: adc start
`uint8_t adc_convert(void);`	Prototype: adc convert

`typedef enum {`
`DD_COMP_111_ERR_NONE = 0x00U,`
`DD_COMP_111_ERR_NULL = 0x01U,`
`DD_COMP_111_ERR_INVALID = 0x02U,`
`DD_COMP_111_ERR_TIMEOUT = 0x03U`
`} DD_COMP_111_ERR_t;`
	Error code enum for ADCReader

main.c - What to Add

Item	Purpose
`static AdcReader_t adcstate = {0};`	Module-level zero-initialised state for ADCReader

`/* Process function logic order:`
`* Step 1: validate configuration`
`* Step 2: configure ADC hardware`
`* Step 3: start conversion`
`* Step 4: wait for completion`
`* Step 5: read raw value`
`* Step 6: apply scaling`
`* Step 7: store result`
`* Step 8: signal availability`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Singleton. Call adc_init() once in main() init sequence. Call adc_read() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

114. DD-COMP-112 - CommandCalculator

DD-COMP-112 **CommandCalculator** | Algorithm | Module: ControlModule | Pattern: Strategy | SW-REQ-097

`@trace DD-COMP-112 SW-REQ-097`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void calc_init(void)`	void	Initialise all state for CommandCalculator
main.c	`static uint8_t calc_run(void *command_str)`	uint8_t - 0 success, non-zero error code	Execute core logic for CommandCalculator
main.c	`static uint8_t parse_cmd(void)`	uint8_t	Helper function: parse cmd
main.c	`static uint8_t eval_expr(void)`	uint8_t	Helper function: eval expr
main.c	`static uint8_t fmt_result(void)`	uint8_t	Helper function: fmt result

Doxygen Comments

```
/**
* @brief Initialise CommandCalculator.
* @trace DD-COMP-112 SW-REQ-097
*/
/**
* @brief Run CommandCalculator process cycle.
* @param command_str Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-112 SW-REQ-097
*/
/**
* @brief Parse cmd.
* @trace DD-COMP-112 SW-REQ-097
*/
/**
* @brief Eval expr.
```

```
 * @trace DD-COMP-112 SW-REQ-097
 */
 /**
 * @brief Fmt result.
 * @trace DD-COMP-112 SW-REQ-097
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize state
2	parse input command
3	validate operands and operator
4	compute result
5	format output message
6	return or store result

Forbidden - Must NOT Do

#	Forbidden Rule
1	unsafe eval on raw input
2	division by zero
3	overflow without checks
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'cc' (e.g. ccresult).`

2	State struct: static CommandCalculator_t ccstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_112_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} CommandCalculator_t;`
	State struct for CommandCalculator

`void calc_init(void);`	Prototype: initialise CommandCalculator
`uint8_t calc_run(void *p_data);`	Prototype: run CommandCalculator process cycle
`uint8_t parse_cmd(void);`	Prototype: parse cmd
`uint8_t eval_expr(void);`	Prototype: eval expr
`uint8_t fmt_result(void);`	Prototype: fmt result

`typedef enum {`
`DD_COMP_112_ERR_NONE = 0x00U,`
`DD_COMP_112_ERR_NULL = 0x01U,`
`DD_COMP_112_ERR_INVALID = 0x02U,`
`DD_COMP_112_ERR_TIMEOUT = 0x03U`
`} DD_COMP_112_ERR_t;`
	Error code enum for CommandCalculator

main.c - What to Add

Item	Purpose

`static CommandCalculator_t ccstate = {0};`	Module-level zero-initialised state for CommandCalculator

`/* Process function logic order:`
`* Step 1: initialize state`
`* Step 2: parse input command`
`* Step 3: validate operands and operator`
`* Step 4: compute result`
`* Step 5: format output message`
`* Step 6: return or store result`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Strategy. Call calc_init() once in main() init sequence. Call calc_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

115. DD-COMP-113 - ShippingLimitGuard

DD-COMP-113	**ShippingLimitGuard** | Class | Module: SafetyModule | Pattern: Decorator | SW-REQ-098

`@trace DD-COMP-113 SW-REQ-098`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void setup_guard(void)`	void	Initialise all state for ShippingLimitGuard
main.c	`static uint8_t check_limit(void *order_data)`	uint8_t - 0 success, non-zero error code	Execute core logic for ShippingLimitGuard
main.c	`static uint8_t load_rules(void)`	uint8_t	Helper function: load rules
main.c	`static uint8_t calc_total(void)`	uint8_t	Helper function: calc total
main.c	`static uint8_t flag_violation(void)`	uint8_t	Helper function: flag violation

Doxygen Comments

```
/**
* @brief Initialise ShippingLimitGuard.
* @trace DD-COMP-113 SW-REQ-098
*/
/**
* @brief Run ShippingLimitGuard process cycle.
* @param order_data Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-113 SW-REQ-098
*/
/**
* @brief Load rules.
* @trace DD-COMP-113 SW-REQ-098
*/
/**
* @brief Calc total.
```

```
 * @trace DD-COMP-113 SW-REQ-098
 */
 /**
 * @brief Flag violation.
 * @trace DD-COMP-113 SW-REQ-098
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load shipping limits
2	gather order metrics
3	compute totals
4	compare against limits
5	set status flag
6	report result

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded limits
2	silent failures
3	unused helpers

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ship' (e.g. shipresult).`
2	`State struct: static ShippingLimitGuard_t shipstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_113_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ShippingLimitGuard_t;`
	State struct for ShippingLimitGuard

`void setup_guard(void);`	Prototype: initialise ShippingLimitGuard
`uint8_t check_limit(void *p_data);`	Prototype: run ShippingLimitGuard process cycle
`uint8_t load_rules(void);`	Prototype: load rules
`uint8_t calc_total(void);`	Prototype: calc total
`uint8_t flag_violation(void);`	Prototype: flag violation

`typedef enum {`
`DD_COMP_113_ERR_NONE = 0x00U,`
`DD_COMP_113_ERR_NULL = 0x01U,`
`DD_COMP_113_ERR_INVALID = 0x02U,`
`DD_COMP_113_ERR_TIMEOUT = 0x03U`
`} DD_COMP_113_ERR_t;`
	Error code enum for ShippingLimitGuard

main.c - What to Add

Item	Purpose
`static ShippingLimitGuard_t shipstate = {0};`	Module-level zero-initialised state for ShippingLimitGuard

`/* Process function logic order:`
`* Step 1: load shipping limits`
`* Step 2: gather order metrics`
`* Step 3: compute totals`
`* Step 4: compare against limits`
`* Step 5: set status flag`
`* Step 6: report result`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Decorator. Call setup_guard() once in main() init sequence. Call check_limit() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

116. DD-COMP-114 - UARTCommandHandler

DD-COMP-114	**UARTCommandHandler** | Service | Module: CommunicationModule | Pattern: Command | SW-REQ-099

`@trace DD-COMP-114 SW-REQ-099`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void uartcmd_init(void)`	void	Initialise all state for UARTCommandHandler
`main.c`	`static uint8_t uartcmd_process(void *uart_rx_buffer)`	uint8_t - 0 success, non-zero error code	Execute core logic for UARTCommandHandler
`main.c`	`static uint8_t uartcmd_parse(void)`	uint8_t	Helper function: uartcmd parse
`main.c`	`static uint8_t uartcmd_execute(void)`	uint8_t	Helper function: uartcmd execute
`main.c`	`static uint8_t uartcmd_reply(void)`	uint8_t	Helper function: uartcmd reply

Doxygen Comments

```
/**
* @brief Initialise UARTCommandHandler.
* @trace DD-COMP-114 SW-REQ-099
*/
/**
* @brief Run UARTCommandHandler process cycle.
* @param uart_rx_buffer Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-114 SW-REQ-099
*/
/**
* @brief Uartcmd parse.
* @trace DD-COMP-114 SW-REQ-099
*/
/**
```

```
 * @brief Uartcmd execute.
 * @trace DD-COMP-114 SW-REQ-099
 */
 /**
 * @brief Uartcmd reply.
 * @trace DD-COMP-114 SW-REQ-099
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read incoming UART bytes
2	detect complete command frame
3	parse command and arguments
4	validate against allowed commands
5	execute command action
6	format response message
7	send response over UART

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UART reads
2	dynamic memory allocation
3	long processing in ISR
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'uartCmd' (e.g. uartCmdresult).
2	State struct: static UARTCommandHandler_t uartCmdstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_114_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} UARTCommandHandler_t;`
	State struct for UARTCommandHandler

`void uartcmd_init(void);`	Prototype: initialise UARTCommandHandler
`uint8_t uartcmd_process(void *p_data);`	Prototype: run UARTCommandHandler process cycle
`uint8_t uartcmd_parse(void);`	Prototype: uartcmd parse
`uint8_t uartcmd_execute(void);`	Prototype: uartcmd execute
`uint8_t uartcmd_reply(void);`	Prototype: uartcmd reply

`typedef enum {`
`DD_COMP_114_ERR_NONE = 0x00U,`
`DD_COMP_114_ERR_NULL = 0x01U,`
`DD_COMP_114_ERR_INVALID = 0x02U,`
`DD_COMP_114_ERR_TIMEOUT = 0x03U`
`} DD_COMP_114_ERR_t;`
	Error code enum for UARTCommandHandler

main.c - What to Add

Item	Purpose
`static UARTCommandHandler_t uartCmdstate = {0};`	Module-level zero-initialised state for UARTCommandHandler

`/* Process function logic order:`
`* Step 1: read incoming UART bytes`
`* Step 2: detect complete command frame`
`* Step 3: parse command and arguments`
`* Step 4: validate against allowed commands`
`* Step 5: execute command action`
`* Step 6: format response message`
`* Step 7: send response over UART`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Command. Call uartcmd_init() once in main() init sequence. Call uartcmd_process() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

117. DD-COMP-115 - ActuatorEnableController

DD-COMP-115	**ActuatorEnableController** | Class | Module: ControlModule | Pattern: State | SW-REQ-099

`@trace DD-COMP-115 SW-REQ-099`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initEnableCtrl(void)`	void	Initialise all state for ActuatorEnableController
main.c	`static uint8_t processEnableCtrl(void *enable_cmd)`	uint8_t - 0 success, non-zero error code	Execute core logic for ActuatorEnableController
main.c	`static uint8_t readEnableCmd(void)`	uint8_t	Helper function: readEnableCmd
main.c	`static uint8_t updateEnableState(void)`	uint8_t	Helper function: updateEnableState
main.c	`static uint8_t applyEnableOutput(void)`	uint8_t	Helper function: applyEnableOutput

Doxygen Comments

```
/**
* @brief Initialise ActuatorEnableController.
* @trace DD-COMP-115 SW-REQ-099
*/
/**
* @brief Run ActuatorEnableController process cycle.
* @param enable_cmd Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-115 SW-REQ-099
*/
/**
* @brief Readenablecmd.
* @trace DD-COMP-115 SW-REQ-099
*/
/**
* @brief Updateenablestate.
```

```
 * @trace DD-COMP-115 SW-REQ-099
 */
 /**
 * @brief Applyenableoutput.
 * @trace DD-COMP-115 SW-REQ-099
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	read enable command input
2	validate command and safety conditions
3	update internal enable state
4	drive actuator enable output
5	report status or faults

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking delays
2	global state mutation
3	unchecked fault conditions

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'enCtrl' (e.g. enCtrlresult).`
2	`State struct: static ActuatorEnableController_t enCtrlstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`
6	`Error returns: use DD_COMP_115_ERR_t enum values, never raw integers.`

7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ActuatorEnableController_t;`
	State struct for ActuatorEnableController

`void initEnableCtrl(void);`	Prototype: initialise ActuatorEnableController
`uint8_t processEnableCtrl(void *p_data);`	Prototype: run ActuatorEnableController process cycle
`uint8_t readEnableCmd(void);`	Prototype: readEnableCmd
`uint8_t updateEnableState(void);`	Prototype: updateEnableState
`uint8_t applyEnableOutput(void);`	Prototype: applyEnableOutput

`typedef enum {`
`DD_COMP_115_ERR_NONE = 0x00U,`
`DD_COMP_115_ERR_NULL = 0x01U,`
`DD_COMP_115_ERR_INVALID = 0x02U,`
`DD_COMP_115_ERR_TIMEOUT = 0x03U`
`} DD_COMP_115_ERR_t;`
	Error code enum for ActuatorEnableController

main.c - What to Add

Item	Purpose
`static ActuatorEnableController_t enCtrlstate = {0};`	Module-level zero-initialised state for ActuatorEnableController

`/* Process function logic order:`
`* Step 1: read enable command input`
`* Step 2: validate command and safety conditions`
`* Step 3: update internal enable state`
`* Step 4: drive actuator enable output`
`* Step 5: report status or faults`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initEnableCtrl() once in main() init sequence. Call processEnableCtrl() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

118. DD-COMP-116 - FaultMonitor

DD-COMP-116	**FaultMonitor** | Service | Module: FaultModule | Pattern: Observer | SW-REQ-100

`@trace DD-COMP-116 SW-REQ-100`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void fm_setup(void)`	void	Initialise all state for FaultMonitor
main.c	`static uint8_t fm_run(void *sensor_status)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultMonitor
main.c	`static uint8_t fm_poll(void)`	uint8_t	Helper function: fm poll
main.c	`static uint8_t fm_eval(void)`	uint8_t	Helper function: fm eval
main.c	`static uint8_t fm_report(void)`	uint8_t	Helper function: fm report

Doxygen Comments

```
/**
* @brief Initialise FaultMonitor.
* @trace DD-COMP-116 SW-REQ-100
*/
/**
* @brief Run FaultMonitor process cycle.
* @param sensor_status Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-116 SW-REQ-100
*/
/**
* @brief Fm poll.
* @trace DD-COMP-116 SW-REQ-100
*/
/**
* @brief Fm eval.
```

```
 * @trace DD-COMP-116 SW-REQ-100
 */
 /**
 * @brief Fm report.
 * @trace DD-COMP-116 SW-REQ-100
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	poll fault sources
3	evaluate thresholds
4	update fault state
5	log events
6	emit alerts

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking I/O
2	silent failures
3	unbounded retries
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fm' (e.g. fmresult).`

2	State struct: static FaultMonitor_t fmstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_116_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultMonitor_t;`
	State struct for FaultMonitor

`void fm_setup(void);`	Prototype: initialise FaultMonitor
`uint8_t fm_run(void *p_data);`	Prototype: run FaultMonitor process cycle
`uint8_t fm_poll(void);`	Prototype: fm poll
`uint8_t fm_eval(void);`	Prototype: fm eval
`uint8_t fm_report(void);`	Prototype: fm report

`typedef enum {`
`DD_COMP_116_ERR_NONE = 0x00U,`
`DD_COMP_116_ERR_NULL = 0x01U,`
`DD_COMP_116_ERR_INVALID = 0x02U,`
`DD_COMP_116_ERR_TIMEOUT = 0x03U`
`} DD_COMP_116_ERR_t;`
	Error code enum for FaultMonitor

main.c - What to Add

Item	Purpose

`static FaultMonitor_t fmstate = {0};`	Module-level zero-initialised state for FaultMonitor

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: poll fault sources`
`* Step 3: evaluate thresholds`
`* Step 4: update fault state`
`* Step 5: log events`
`* Step 6: emit alerts`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call fm_setup() once in main() init sequence. Call fm_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

119. DD-COMP-117 - FaultInputProvider

DD-COMP-117	**FaultInputProvider** | Interface | Module: FaultModule | Pattern: Adapter | SW-REQ-100

`@trace DD-COMP-117 SW-REQ-100`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initFaultInput(void)`	void	Initialise all state for FaultInputProvider
main.c	`static uint8_t provideFaultInput(void *fault_source)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultInputProvider
main.c	`static uint8_t readSource(void)`	uint8_t	Helper function: readSource
main.c	`static uint8_t validateFault(void)`	uint8_t	Helper function: validateFault
main.c	`static uint8_t formatPacket(void)`	uint8_t	Helper function: formatPacket

Doxygen Comments

```
/**
* @brief Initialise FaultInputProvider.
* @trace DD-COMP-117 SW-REQ-100
*/
/**
* @brief Run FaultInputProvider process cycle.
* @param fault_source Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-117 SW-REQ-100
*/
/**
* @brief Readsource.
* @trace DD-COMP-117 SW-REQ-100
*/
/**
* @brief Validatefault.
```

```
 * @trace DD-COMP-117 SW-REQ-100
 */
 /**
 * @brief Formatpacket.
 * @trace DD-COMP-117 SW-REQ-100
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize input sources
2	poll for new fault data
3	validate incoming fault fields
4	normalize data format
5	queue or dispatch fault input
6	report status to caller

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking reads
2	silent data drops
3	ambiguous error codes
4	No direct register access - use HAL abstraction only
5	All input pointers must be NULL-checked before dereferencing
6	No global state mutation from API functions

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fault' (e.g. faultresult).`
2	`State struct: static FaultInputProvider_t faultstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_117_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultInputProvider_t;`
	State struct for FaultInputProvider

`void initFaultInput(void);`	Prototype: initialise FaultInputProvider
`uint8_t provideFaultInput(void *p_data);`	Prototype: run FaultInputProvider process cycle
`uint8_t readSource(void);`	Prototype: readSource
`uint8_t validateFault(void);`	Prototype: validateFault
`uint8_t formatPacket(void);`	Prototype: formatPacket

`typedef enum {`
`DD_COMP_117_ERR_NONE = 0x00U,`
`DD_COMP_117_ERR_NULL = 0x01U,`
`DD_COMP_117_ERR_INVALID = 0x02U,`
`DD_COMP_117_ERR_TIMEOUT = 0x03U`
`} DD_COMP_117_ERR_t;`
	Error code enum for FaultInputProvider

main.c - What to Add

Item	Purpose
`static FaultInputProvider_t faultstate = {0};`	Module-level zero-initialised state for FaultInputProvider

`/* Process function logic order:`
`* Step 1: initialize input sources`
`* Step 2: poll for new fault data`
`* Step 3: validate incoming fault fields`
`* Step 4: normalize data format`
`* Step 5: queue or dispatch fault input`
`* Step 6: report status to caller`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Adapter. Call initFaultInput() once in main() init sequence. Call provideFaultInput() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

120. DD-COMP-118 - FaultLogger

DD-COMP-118	**FaultLogger** | Service | Module: FaultModule | Pattern: Observer | SW-REQ-101

`@trace DD-COMP-118 SW-REQ-101`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void initLog(void)`	void	Initialise all state for FaultLogger
`main.c`	`static uint8_t runLogger(void *faultEvent)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultLogger
`main.c`	`static uint8_t formatEntry(void)`	uint8_t	Helper function: formatEntry
`main.c`	`static uint8_t writeStore(void)`	uint8_t	Helper function: writeStore
`main.c`	`static uint8_t rotateLog(void)`	uint8_t	Helper function: rotateLog

Doxygen Comments

```
/**
* @brief Initialise FaultLogger.
* @trace DD-COMP-118 SW-REQ-101
*/
/**
* @brief Run FaultLogger process cycle.
* @param faultEvent Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-118 SW-REQ-101
*/
/**
* @brief Formatentry.
* @trace DD-COMP-118 SW-REQ-101
*/
/**
* @brief Writestore.
* @trace DD-COMP-118 SW-REQ-101
```

```
*/
/**
* @brief Rotatelog.
* @trace DD-COMP-118 SW-REQ-101
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	open or create log target
3	receive fault event
4	format log entry
5	write entry to storage
6	handle rotation or limits
7	report status

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking on slow storage
2	dropping critical faults
3	unbounded log growth
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fl' (e.g. flresult).`

2	State struct: static FaultLogger_t flstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_118_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultLogger_t;`
	State struct for FaultLogger

`void initLog(void);`	Prototype: initialise FaultLogger
`uint8_t runLogger(void *p_data);`	Prototype: run FaultLogger process cycle
`uint8_t formatEntry(void);`	Prototype: formatEntry
`uint8_t writeStore(void);`	Prototype: writeStore
`uint8_t rotateLog(void);`	Prototype: rotateLog

`typedef enum {`
`DD_COMP_118_ERR_NONE = 0x00U,`
`DD_COMP_118_ERR_NULL = 0x01U,`
`DD_COMP_118_ERR_INVALID = 0x02U,`
`DD_COMP_118_ERR_TIMEOUT = 0x03U`
`} DD_COMP_118_ERR_t;`
	Error code enum for FaultLogger

main.c - What to Add

Item	Purpose

`static FaultLogger_t flstate = {0};`	Module-level zero-initialised state for FaultLogger

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: open or create log target`
`* Step 3: receive fault event`
`* Step 4: format log entry`
`* Step 5: write entry to storage`
`* Step 6: handle rotation or limits`
`* Step 7: report status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initLog() once in main() init sequence. Call runLogger() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

121. DD-COMP-119 - FaultLogStore

DD-COMP-119	**FaultLogStore** | Database | Module: DataLayer | Pattern: Repository | SW-REQ-101

`@trace DD-COMP-119 SW-REQ-101`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_store(void)`	void	Initialise all state for FaultLogStore
main.c	`static uint8_t save_fault(void *fault_record)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultLogStore
main.c	`static uint8_t open_db(void)`	uint8_t	Helper function: open db
main.c	`static uint8_t write_entry(void)`	uint8_t	Helper function: write entry
main.c	`static uint8_t purge_old(void)`	uint8_t	Helper function: purge old

Doxygen Comments

```
/**
* @brief Initialise FaultLogStore.
* @trace DD-COMP-119 SW-REQ-101
*/
/**
* @brief Run FaultLogStore process cycle.
* @param fault_record Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-119 SW-REQ-101
*/
/**
* @brief Open db.
* @trace DD-COMP-119 SW-REQ-101
*/
/**
* @brief Write entry.
```

```
 * @trace DD-COMP-119 SW-REQ-101
 */
 /**
 * @brief Purge old.
 * @trace DD-COMP-119 SW-REQ-101
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	open database connection
3	validate fault record
4	write record to storage
5	update indexes
6	handle errors and retries

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking main thread
2	silent write failures
3	unbounded log growth
4	No dynamic struct allocation
5	Array sizes must be compile-time #define constants
6	No pointer arithmetic on struct members

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fls' (e.g. flsresult).`
2	`State struct: static FaultLogStore_t flsstate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_119_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultLogStore_t;`
	State struct for FaultLogStore

`void init_store(void);`	Prototype: initialise FaultLogStore
`uint8_t save_fault(void *p_data);`	Prototype: run FaultLogStore process cycle
`uint8_t open_db(void);`	Prototype: open db
`uint8_t write_entry(void);`	Prototype: write entry
`uint8_t purge_old(void);`	Prototype: purge old

`typedef enum {`
`DD_COMP_119_ERR_NONE = 0x00U,`
`DD_COMP_119_ERR_NULL = 0x01U,`
`DD_COMP_119_ERR_INVALID = 0x02U,`
`DD_COMP_119_ERR_TIMEOUT = 0x03U`
`} DD_COMP_119_ERR_t;`
	Error code enum for FaultLogStore

main.c - What to Add

Item	Purpose
`static FaultLogStore_t flsstate = {0};`	Module-level zero-initialised state for FaultLogStore

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: open database connection`
`* Step 3: validate fault record`
`* Step 4: write record to storage`
`* Step 5: update indexes`
`* Step 6: handle errors and retries`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call init_store() once in main() init sequence. Call save_fault() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

122. DD-COMP-120 - SystemStateMachine

DD-COMP-120	**SystemStateMachine** | Class | Module: StateModule | Pattern: State | SW-REQ-102

`@trace DD-COMP-120 SW-REQ-102`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initMachine(void)`	void	Initialise all state for SystemStateMachine
main.c	`static uint8_t runStep(void *event)`	uint8_t - 0 success, non-zero error code	Execute core logic for SystemStateMachine
main.c	`static uint8_t loadState(void)`	uint8_t	Helper function: loadState
main.c	`static uint8_t evalTransition(void)`	uint8_t	Helper function: evalTransition
main.c	`static uint8_t saveState(void)`	uint8_t	Helper function: saveState

Doxygen Comments

```
/**
* @brief Initialise SystemStateMachine.
* @trace DD-COMP-120 SW-REQ-102
*/
/**
* @brief Run SystemStateMachine process cycle.
* @param event Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-120 SW-REQ-102
*/
/**
* @brief Loadstate.
* @trace DD-COMP-120 SW-REQ-102
*/
/**
* @brief Evaltransition.
```

```
* @trace DD-COMP-120 SW-REQ-102
*/
/**
* @brief Savestate.
* @trace DD-COMP-120 SW-REQ-102
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	Initialize current state
2	Read inputs/events
3	Evaluate transition rules
4	Update state if needed
5	Trigger entry/exit actions
6	Record state/output

Forbidden - Must NOT Do

#	Forbidden Rule
1	avoid global state
2	avoid blocking waits
3	avoid deep nesting

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ssm' (e.g. ssmresult).`
2	`State struct: static SystemStateMachine_t ssmstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_120_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SystemStateMachine_t;`
	State struct for SystemStateMachine

`void initMachine(void);`	Prototype: initialise SystemStateMachine
`uint8_t runStep(void *p_data);`	Prototype: run SystemStateMachine process cycle
`uint8_t loadState(void);`	Prototype: loadState
`uint8_t evalTransition(void);`	Prototype: evalTransition
`uint8_t saveState(void);`	Prototype: saveState

`typedef enum {`
`DD_COMP_120_ERR_NONE = 0x00U,`
`DD_COMP_120_ERR_NULL = 0x01U,`
`DD_COMP_120_ERR_INVALID = 0x02U,`
`DD_COMP_120_ERR_TIMEOUT = 0x03U`
`} DD_COMP_120_ERR_t;`
	Error code enum for SystemStateMachine

main.c - What to Add

Item	Purpose
`static SystemStateMachine_t ssmstate = {0};`	Module-level zero-initialised state for SystemStateMachine

`/* Process function logic order:`
`* Step 1: Initialize current state`
`* Step 2: Read inputs/events`
`* Step 3: Evaluate transition rules`
`* Step 4: Update state if needed`
`* Step 5: Trigger entry/exit actions`
`* Step 6: Record state/output`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: State. Call initMachine() once in main() init sequence. Call runStep() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

123. DD-COMP-121 - FaultToStateTransitionService

DD-COMP-121	**FaultToStateTransitionService** | Service | Module: StateModule | Pattern: Observer | SW-REQ-102

`@trace DD-COMP-121 SW-REQ-102`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initFaultTrans(void)`	void	Initialise all state for FaultToStateTransitionService
main.c	`static uint8_t processFaultState(void *faultCode)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultToStateTransitionService
main.c	`static uint8_t mapFaultToState(void)`	uint8_t	Helper function: mapFaultToState
main.c	`static uint8_t validateFaultData(void)`	uint8_t	Helper function: validateFaultData
main.c	`static uint8_t updateStateCache(void)`	uint8_t	Helper function: updateStateCache

Doxygen Comments

```
/**
* @brief Initialise FaultToStateTransitionService.
* @trace DD-COMP-121 SW-REQ-102
*/
/**
* @brief Run FaultToStateTransitionService process cycle.
* @param faultCode Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-121 SW-REQ-102
*/
/**
* @brief Mapfaulttostate.
* @trace DD-COMP-121 SW-REQ-102
*/
```

```
/**
* @brief Validatefaultdata.
* @trace DD-COMP-121 SW-REQ-102
*/
/**
* @brief Updatestatecache.
* @trace DD-COMP-121 SW-REQ-102
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load current system state
2	validate incoming fault data
3	map fault to target state
4	check transition rules
5	update state and log change
6	emit state transition result

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoding fault mappings
2	silent transition failures
3	mutating inputs directly
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule

1	All local variables prefixed 'fts' (e.g. ftsresult).
2	State struct: static FaultToStateTransitionService_t ftsstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_121_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultToStateTransitionService_t;`
	State struct for FaultToStateTransitionService

`void initFaultTrans(void);`	Prototype: initialise FaultToStateTransitionService
`uint8_t processFaultState(void *p_data);`	Prototype: run FaultToStateTransitionService process cycle
`uint8_t mapFaultToState(void);`	Prototype: mapFaultToState
`uint8_t validateFaultData(void);`	Prototype: validateFaultData
`uint8_t updateStateCache(void);`	Prototype: updateStateCache

`typedef enum {`
`DD_COMP_121_ERR_NONE = 0x00U,`
`DD_COMP_121_ERR_NULL = 0x01U,`
`DD_COMP_121_ERR_INVALID = 0x02U,`
`DD_COMP_121_ERR_TIMEOUT = 0x03U`
`} DD_COMP_121_ERR_t;`
	Error code enum for FaultToStateTransitionService

main.c - What to Add

Item	Purpose
`static FaultToStateTransitionService_t ftsstate = {0};`	Module-level zero-initialised state for FaultToStateTransitionService

`/* Process function logic order:`
`* Step 1: load current system state`
`* Step 2: validate incoming fault data`
`* Step 3: map fault to target state`
`* Step 4: check transition rules`
`* Step 5: update state and log change`
`* Step 6: emit state transition result`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initFaultTrans() once in main() init sequence. Call processFaultState() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

124. DD-COMP-122 - RecoveryCommandHandler

DD-COMP-122	**RecoveryCommandHandler** | Service | Module: StateModule | Pattern: Command | SW-REQ-103

`@trace DD-COMP-122 SW-REQ-103`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init_recovery(void)`	void	Initialise all state for RecoveryCommandHandler
`main.c`	`static uint8_t handle_recovery(void *cmd_msg)`	uint8_t - 0 success, non-zero error code	Execute core logic for RecoveryCommandHandler
`main.c`	`static uint8_t parse_cmd(void)`	uint8_t	Helper function: parse cmd
`main.c`	`static uint8_t validate_req(void)`	uint8_t	Helper function: validate req
`main.c`	`static uint8_t send_reply(void)`	uint8_t	Helper function: send reply

Doxygen Comments

```
/**
* @brief Initialise RecoveryCommandHandler.
* @trace DD-COMP-122 SW-REQ-103
*/
/**
* @brief Run RecoveryCommandHandler process cycle.
* @param cmd_msg Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-122 SW-REQ-103
*/
/**
* @brief Parse cmd.
* @trace DD-COMP-122 SW-REQ-103
*/
/**
```

```
 * @brief Validate req.
 * @trace DD-COMP-122 SW-REQ-103
 */
 /**
 * @brief Send reply.
 * @trace DD-COMP-122 SW-REQ-103
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load recovery configuration
2	parse incoming command
3	validate command parameters
4	execute recovery action
5	collect result status
6	send response message

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking operations
2	silent failures
3	undefined states
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'rec' (e.g. recresult).`

2	State struct: static RecoveryCommandHandler_t recstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_122_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} RecoveryCommandHandler_t;`
	State struct for RecoveryCommandHandler

`void init_recovery(void);`	Prototype: initialise RecoveryCommandHandler
`uint8_t handle_recovery(void *p_data);`	Prototype: run RecoveryCommandHandler process cycle
`uint8_t parse_cmd(void);`	Prototype: parse cmd
`uint8_t validate_req(void);`	Prototype: validate req
`uint8_t send_reply(void);`	Prototype: send reply

`typedef enum {`
`DD_COMP_122_ERR_NONE = 0x00U,`
`DD_COMP_122_ERR_NULL = 0x01U,`
`DD_COMP_122_ERR_INVALID = 0x02U,`
`DD_COMP_122_ERR_TIMEOUT = 0x03U`
`} DD_COMP_122_ERR_t;`
	Error code enum for RecoveryCommandHandler

main.c - What to Add

Item	Purpose

`static RecoveryCommandHandler_t recstate = {0};`	Module-level zero-initialised state for RecoveryCommandHandler

`/* Process function logic order:`
`* Step 1: load recovery configuration`
`* Step 2: parse incoming command`
`* Step 3: validate command parameters`
`* Step 4: execute recovery action`
`* Step 5: collect result status`
`* Step 6: send response message`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Command. Call init_recovery() once in main() init sequence. Call handle_recovery() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

125. DD-COMP-123 - FaultStatusTracker

DD-COMP-123	**FaultStatusTracker** | Class | Module: FaultModule | Pattern: Observer | SW-REQ-103

`@trace DD-COMP-123 SW-REQ-103`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initTracker(void)`	void	Initialise all state for FaultStatusTracker
main.c	`static uint8_t processStatus(void *faultSignals)`	uint8_t - 0 success, non-zero error code	Execute core logic for FaultStatusTracker
main.c	`static uint8_t updateState(void)`	uint8_t	Helper function: updateState
main.c	`static uint8_t detectFaults(void)`	uint8_t	Helper function: detectFaults
main.c	`static uint8_t emitReport(void)`	uint8_t	Helper function: emitReport

Doxygen Comments

```
/**
* @brief Initialise FaultStatusTracker.
* @trace DD-COMP-123 SW-REQ-103
*/
/**
* @brief Run FaultStatusTracker process cycle.
* @param faultSignals Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-123 SW-REQ-103
*/
/**
* @brief Updatestate.
* @trace DD-COMP-123 SW-REQ-103
*/
/**
* @brief Detectfaults.
```

```
 * @trace DD-COMP-123 SW-REQ-103
 */
 /**
 * @brief Emitreport.
 * @trace DD-COMP-123 SW-REQ-103
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration and thresholds
2	ingest current fault signals
3	update internal fault state
4	detect new or cleared faults
5	record status changes
6	generate status output

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking operations
2	global mutable state
3	ambiguous status codes

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'fst' (e.g. fstresult).`
2	`State struct: static FaultStatusTracker_t fststate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`
4	`Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).`
5	`Buffers: uint8_t array, suffix _buf, size via #define constant.`

6	Error returns: use DD_COMP_123_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FaultStatusTracker_t;`
	State struct for FaultStatusTracker

`void initTracker(void);`	Prototype: initialise FaultStatusTracker
`uint8_t processStatus(void *p_data);`	Prototype: run FaultStatusTracker process cycle
`uint8_t updateState(void);`	Prototype: updateState
`uint8_t detectFaults(void);`	Prototype: detectFaults
`uint8_t emitReport(void);`	Prototype: emitReport

`typedef enum {`
`DD_COMP_123_ERR_NONE = 0x00U,`
`DD_COMP_123_ERR_NULL = 0x01U,`
`DD_COMP_123_ERR_INVALID = 0x02U,`
`DD_COMP_123_ERR_TIMEOUT = 0x03U`
`} DD_COMP_123_ERR_t;`
	Error code enum for FaultStatusTracker

main.c - What to Add

Item	Purpose
`static FaultStatusTracker_t fststate = {0};`	Module-level zero-initialised state for FaultStatusTracker

`/* Process function logic order:`
`* Step 1: load configuration and thresholds`
`* Step 2: ingest current fault signals`
`* Step 3: update internal fault state`
`* Step 4: detect new or cleared faults`
`* Step 5: record status changes`
`* Step 6: generate status output`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initTracker() once in main() init sequence. Call processStatus() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

126. DD-COMP-124 - StateStatusProvider

DD-COMP-124	**StateStatusProvider** | Service | Module: StateModule | Pattern: Observer | SW-REQ-104

`@trace DD-COMP-124 SW-REQ-104`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void initState(void)`	void	Initialise all state for StateStatusProvider
`main.c`	`static uint8_t processStatus(void *config)`	uint8_t - 0 success, non-zero error code	Execute core logic for StateStatusProvider
`main.c`	`static uint8_t fetchState(void)`	uint8_t	Helper function: fetchState
`main.c`	`static uint8_t mapStatus(void)`	uint8_t	Helper function: mapStatus
`main.c`	`static uint8_t emitStatus(void)`	uint8_t	Helper function: emitStatus

Doxygen Comments

```
/**
* @brief Initialise StateStatusProvider.
* @trace DD-COMP-124 SW-REQ-104
*/
/**
* @brief Run StateStatusProvider process cycle.
* @param config Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-124 SW-REQ-104
*/
/**
* @brief Fetchstate.
* @trace DD-COMP-124 SW-REQ-104
*/
/**
* @brief Mapstatus.
```

```
 * @trace DD-COMP-124 SW-REQ-104
 */
 /**
 * @brief Emitstatus.
 * @trace DD-COMP-124 SW-REQ-104
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration
2	query current state
3	translate state to status
4	build status payload
5	emit status update
6	log processing result

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking calls
2	global state
3	tight coupling
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'ssp' (e.g. sspresult).`

2	State struct: static StateStatusProvider_t sspstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_124_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} StateStatusProvider_t;`
	State struct for StateStatusProvider

`void initState(void);`	Prototype: initialise StateStatusProvider
`uint8_t processStatus(void *p_data);`	Prototype: run StateStatusProvider process cycle
`uint8_t fetchState(void);`	Prototype: fetchState
`uint8_t mapStatus(void);`	Prototype: mapStatus
`uint8_t emitStatus(void);`	Prototype: emitStatus

`typedef enum {`
`DD_COMP_124_ERR_NONE = 0x00U,`
`DD_COMP_124_ERR_NULL = 0x01U,`
`DD_COMP_124_ERR_INVALID = 0x02U,`
`DD_COMP_124_ERR_TIMEOUT = 0x03U`
`} DD_COMP_124_ERR_t;`
	Error code enum for StateStatusProvider

main.c - What to Add

Item	Purpose

`static StateStatusProvider_t sspstate = {0};`	Module-level zero-initialised state for StateStatusProvider

`/* Process function logic order:`
`* Step 1: load configuration`
`* Step 2: query current state`
`* Step 3: translate state to status`
`* Step 4: build status payload`
`* Step 5: emit status update`
`* Step 6: log processing result`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Observer. Call initState() once in main() init sequence. Call processStatus() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

127. DD-COMP-125 - StateStatusAPI

DD-COMP-125	**StateStatusAPI** | API Endpoint | Module: InterfaceModule | Pattern: Facade | SW-REQ-104

`@trace DD-COMP-125 SW-REQ-104`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_state_status(void)`	void	Initialise all state for StateStatusAPI
main.c	`static uint8_t process_state_status(void *request)`	uint8_t - 0 success, non-zero error code	Execute core logic for StateStatusAPI
main.c	`static uint8_t parse_state_query(void)`	uint8_t	Helper function: parse state query
main.c	`static uint8_t fetch_state_data(void)`	uint8_t	Helper function: fetch state data
main.c	`static uint8_t format_status_response(void)`	uint8_t	Helper function: format status response

Doxygen Comments

```
/**
* @brief Initialise StateStatusAPI.
* @trace DD-COMP-125 SW-REQ-104
*/
/**
* @brief Run StateStatusAPI process cycle.
* @param request Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-125 SW-REQ-104
*/
/**
* @brief Parse state query.
* @trace DD-COMP-125 SW-REQ-104
*/
/**
* @brief Fetch state data.
```

```
 * @trace DD-COMP-125 SW-REQ-104
 */
 /**
 * @brief Format status response.
 * @trace DD-COMP-125 SW-REQ-104
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize request context
2	validate query parameters
3	retrieve state status data
4	handle missing or invalid state
5	assemble response payload
6	return status response

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking I/O
2	leaking internal errors
3	overly verbose responses
4	No direct register access - use HAL abstraction only
5	All input pointers must be NULL-checked before dereferencing
6	No global state mutation from API functions

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'state' (e.g. stateresult).`
2	`State struct: static StateStatusApi_t statestate = {0};`

3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_125_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} StateStatusApi_t;`
	State struct for StateStatusAPI

`void init_state_status(void);`	Prototype: initialise StateStatusAPI
`uint8_t process_state_status(void *p_data);`	Prototype: run StateStatusAPI process cycle
`uint8_t parse_state_query(void);`	Prototype: parse state query
`uint8_t fetch_state_data(void);`	Prototype: fetch state data
`uint8_t format_status_response(void);`	Prototype: format status response

`typedef enum {`
`DD_COMP_125_ERR_NONE = 0x00U,`
`DD_COMP_125_ERR_NULL = 0x01U,`
`DD_COMP_125_ERR_INVALID = 0x02U,`
`DD_COMP_125_ERR_TIMEOUT = 0x03U`
`} DD_COMP_125_ERR_t;`
	Error code enum for StateStatusAPI

main.c - What to Add

Item	Purpose
`static StateStatusApi_t statestate = {0};`	Module-level zero-initialised state for StateStatusAPI

`/* Process function logic order:`
`* Step 1: initialize request context`
`* Step 2: validate query parameters`
`* Step 3: retrieve state status data`
`* Step 4: handle missing or invalid state`
`* Step 5: assemble response payload`
`* Step 6: return status response`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call init_state_status() once in main() init sequence. Call process_state_status() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

128. DD-COMP-126 - ArchitectureDiagramExporter

DD-COMP-126	**ArchitectureDiagramExporter** | Service | Module: ArtifactGenerationModule | Pattern: Facade | SW-REQ-105

`@trace DD-COMP-126 SW-REQ-105`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void setupExport(void)`	void	Initialise all state for ArchitectureDiagramExporter
`main.c`	`static uint8_t runExport(void *model_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for ArchitectureDiagramExporter
`main.c`	`static uint8_t loadModel(void)`	uint8_t	Helper function: loadModel
`main.c`	`static uint8_t renderDiagram(void)`	uint8_t	Helper function: renderDiagram
`main.c`	`static uint8_t writeOutput(void)`	uint8_t	Helper function: writeOutput

Doxygen Comments

```
/**
* @brief Initialise ArchitectureDiagramExporter.
* @trace DD-COMP-126 SW-REQ-105
*/
/**
* @brief Run ArchitectureDiagramExporter process cycle.
* @param model_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-126 SW-REQ-105
*/
/**
* @brief Loadmodel.
* @trace DD-COMP-126 SW-REQ-105
*/
/**
```

```
 * @brief Renderdiagram.
 * @trace DD-COMP-126 SW-REQ-105
 */
 /**
 * @brief Writeoutput.
 * @trace DD-COMP-126 SW-REQ-105
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	validate inputs
2	load architecture model
3	generate diagram layout
4	render diagram to target format
5	write output file
6	report status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	blocking UI
3	silent failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'exp' (e.g. expresult).`

2	State struct: static ArchitectureDiagramExporter_t expstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_126_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ArchitectureDiagramExporter_t;`
	State struct for ArchitectureDiagramExporter

`void setupExport(void);`	Prototype: initialise ArchitectureDiagramExporter
`uint8_t runExport(void *p_data);`	Prototype: run ArchitectureDiagramExporter process cycle
`uint8_t loadModel(void);`	Prototype: loadModel
`uint8_t renderDiagram(void);`	Prototype: renderDiagram
`uint8_t writeOutput(void);`	Prototype: writeOutput

`typedef enum {`
`DD_COMP_126_ERR_NONE = 0x00U,`
`DD_COMP_126_ERR_NULL = 0x01U,`
`DD_COMP_126_ERR_INVALID = 0x02U,`
`DD_COMP_126_ERR_TIMEOUT = 0x03U`
`} DD_COMP_126_ERR_t;`
	Error code enum for ArchitectureDiagramExporter

main.c - What to Add

Item	Purpose

`static ArchitectureDiagramExporter_t expstate = {0};`	Module-level zero-initialised state for ArchitectureDiagramExporter

`/* Process function logic order:`
`* Step 1: validate inputs`
`* Step 2: load architecture model`
`* Step 3: generate diagram layout`
`* Step 4: render diagram to target format`
`* Step 5: write output file`
`* Step 6: report status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call setupExport() once in main() init sequence. Call runExport() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

129. DD-COMP-127 - DocumentationRepositoryAdapter

DD-COMP-127	**DocumentationRepositoryAdapter** | Class | Module: DocRepositoryModule | Pattern: Repository | SW-REQ-105

`@trace DD-COMP-127 SW-REQ-105`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initAdapter(void)`	void	Initialise all state for DocumentationRepositoryAdapter
main.c	`static uint8_t processDocs(void *config)`	uint8_t - 0 success, non-zero error code	Execute core logic for DocumentationRepositoryAdapter
main.c	`static uint8_t loadConfig(void)`	uint8_t	Helper function: loadConfig
main.c	`static uint8_t mapDocs(void)`	uint8_t	Helper function: mapDocs
main.c	`static uint8_t saveResults(void)`	uint8_t	Helper function: saveResults

Doxygen Comments

```
/**
* @brief Initialise DocumentationRepositoryAdapter.
* @trace DD-COMP-127 SW-REQ-105
*/
/**
* @brief Run DocumentationRepositoryAdapter process cycle.
* @param config Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-127 SW-REQ-105
*/
/**
* @brief Loadconfig.
* @trace DD-COMP-127 SW-REQ-105
*/
/**
```

```
 * @brief Mapdocs.
 * @trace DD-COMP-127 SW-REQ-105
 */
 /**
 * @brief Saveresults.
 * @trace DD-COMP-127 SW-REQ-105
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load repository settings
2	connect to documentation store
3	fetch or update documents
4	transform records to internal format
5	validate required metadata
6	persist changes
7	return summary status

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	blocking long operations
3	silent failure
4	mutating inputs

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'doc' (e.g. docresult).`
2	`State struct: static DocumentationRepositoryAdapter_t docstate = {0};`
3	`Counters: uint32_t, suffix _cnt (e.g. retry_cnt).`

4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_127_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} DocumentationRepositoryAdapter_t;`
	State struct for DocumentationRepositoryAdapter

`void initAdapter(void);`	Prototype: initialise DocumentationRepositoryAdapter
`uint8_t processDocs(void *p_data);`	Prototype: run DocumentationRepositoryAdapter process cycle
`uint8_t loadConfig(void);`	Prototype: loadConfig
`uint8_t mapDocs(void);`	Prototype: mapDocs
`uint8_t saveResults(void);`	Prototype: saveResults

`typedef enum {`
`DD_COMP_127_ERR_NONE = 0x00U,`
`DD_COMP_127_ERR_NULL = 0x01U,`
`DD_COMP_127_ERR_INVALID = 0x02U,`
`DD_COMP_127_ERR_TIMEOUT = 0x03U`
`} DD_COMP_127_ERR_t;`
	Error code enum for DocumentationRepositoryAdapter

main.c - What to Add

Item	Purpose
`static DocumentationRepositoryAdapter_t docstate = {0};`	Module-level zero-initialised state for DocumentationRepositoryAdapter

`/* Process function logic order:`
`* Step 1: load repository settings`
`* Step 2: connect to documentation store`
`* Step 3: fetch or update documents`
`* Step 4: transform records to internal format`
`* Step 5: validate required metadata`
`* Step 6: persist changes`
`* Step 7: return summary status`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Repository. Call initAdapter() once in main() init sequence. Call processDocs() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

130. DD-COMP-128 - SchematicPdfExporter

DD-COMP-128	**SchematicPdfExporter** | Service | Module: ArtifactGenerationModule | Pattern: Facade | SW-REQ-106

`@trace DD-COMP-128 SW-REQ-106`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_export(void)`	void	Initialise all state for SchematicPdfExporter
main.c	`static uint8_t run_export(void *schematic_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for SchematicPdfExporter
main.c	`static uint8_t load_schematic(void)`	uint8_t	Helper function: load schematic
main.c	`static uint8_t render_pages(void)`	uint8_t	Helper function: render pages
main.c	`static uint8_t write_pdf(void)`	uint8_t	Helper function: write pdf

Doxygen Comments

```
/**
* @brief Initialise SchematicPdfExporter.
* @trace DD-COMP-128 SW-REQ-106
*/
/**
* @brief Run SchematicPdfExporter process cycle.
* @param schematic_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-128 SW-REQ-106
*/
/**
* @brief Load schematic.
* @trace DD-COMP-128 SW-REQ-106
*/
/**
```

```
 * @brief Render pages.
 * @trace DD-COMP-128 SW-REQ-106
 */
 /**
 * @brief Write pdf.
 * @trace DD-COMP-128 SW-REQ-106
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	validate input files and options
2	load and parse schematic data
3	render schematic into PDF pages
4	apply metadata and layout settings
5	write PDF to output path
6	report success or errors

Forbidden - Must NOT Do

#	Forbidden Rule
1	blocking UI during export
2	hardcoded paths
3	silently ignoring errors
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'exp' (e.g. expresult).`

2	State struct: static SchematicPdfExporter_t expstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_128_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} SchematicPdfExporter_t;`
	State struct for SchematicPdfExporter

`void init_export(void);`	Prototype: initialise SchematicPdfExporter
`uint8_t run_export(void *p_data);`	Prototype: run SchematicPdfExporter process cycle
`uint8_t load_schematic(void);`	Prototype: load schematic
`uint8_t render_pages(void);`	Prototype: render pages
`uint8_t write_pdf(void);`	Prototype: write pdf

`typedef enum {`
`DD_COMP_128_ERR_NONE = 0x00U,`
`DD_COMP_128_ERR_NULL = 0x01U,`
`DD_COMP_128_ERR_INVALID = 0x02U,`
`DD_COMP_128_ERR_TIMEOUT = 0x03U`
`} DD_COMP_128_ERR_t;`
	Error code enum for SchematicPdfExporter

main.c - What to Add

Item	Purpose

`static SchematicPdfExporter_t expstate = {0};`	Module-level zero-initialised state for SchematicPdfExporter

`/* Process function logic order:`
`* Step 1: validate input files and options`
`* Step 2: load and parse schematic data`
`* Step 3: render schematic into PDF pages`
`* Step 4: apply metadata and layout settings`
`* Step 5: write PDF to output path`
`* Step 6: report success or errors`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call init_export() once in main() init sequence. Call run_export() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

131. DD-COMP-129 - PcbGerberExporter

DD-COMP-129	**PcbGerberExporter** | Service | Module: ArtifactGenerationModule | Pattern: Facade | SW-REQ-107

`@trace DD-COMP-129 SW-REQ-107`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void init_exporter(void)`	void	Initialise all state for PcbGerberExporter
main.c	`static uint8_t export_gerber(void *pcb_data)`	uint8_t - 0 success, non-zero error code	Execute core logic for PcbGerberExporter
main.c	`static uint8_t load_board(void)`	uint8_t	Helper function: load board
main.c	`static uint8_t render_layers(void)`	uint8_t	Helper function: render layers
main.c	`static uint8_t write_outputs(void)`	uint8_t	Helper function: write outputs

Doxygen Comments

```
/**
* @brief Initialise PcbGerberExporter.
* @trace DD-COMP-129 SW-REQ-107
*/
/**
* @brief Run PcbGerberExporter process cycle.
* @param pcb_data Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-129 SW-REQ-107
*/
/**
* @brief Load board.
* @trace DD-COMP-129 SW-REQ-107
*/
/**
```

```
* @brief Render layers.
* @trace DD-COMP-129 SW-REQ-107
*/
/**
* @brief Write outputs.
* @trace DD-COMP-129 SW-REQ-107
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	validate input board data
2	load export settings
3	convert PCB layers to Gerber format
4	generate drill files
5	write output files
6	report export summary

Forbidden - Must NOT Do

#	Forbidden Rule
1	skipping layer validation
2	overwriting existing files without prompt
3	using hardcoded paths
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'gerb' (e.g. gerbresult).`

2	State struct: static PcbGerberExporter_t gerbstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_129_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} PcbGerberExporter_t;`
	State struct for PcbGerberExporter

`void init_exporter(void);`	Prototype: initialise PcbGerberExporter
`uint8_t export_gerber(void *p_data);`	Prototype: run PcbGerberExporter process cycle
`uint8_t load_board(void);`	Prototype: load board
`uint8_t render_layers(void);`	Prototype: render layers
`uint8_t write_outputs(void);`	Prototype: write outputs

`typedef enum {`
`DD_COMP_129_ERR_NONE = 0x00U,`
`DD_COMP_129_ERR_NULL = 0x01U,`
`DD_COMP_129_ERR_INVALID = 0x02U,`
`DD_COMP_129_ERR_TIMEOUT = 0x03U`
`} DD_COMP_129_ERR_t;`
	Error code enum for PcbGerberExporter

main.c - What to Add

Item	Purpose

`static PcbGerberExporter_t gerbstate = {0};`	Module-level zero-initialised state for PcbGerberExporter

`/* Process function logic order:`
`* Step 1: validate input board data`
`* Step 2: load export settings`
`* Step 3: convert PCB layers to Gerber format`
`* Step 4: generate drill files`
`* Step 5: write output files`
`* Step 6: report export summary`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call init_exporter() once in main() init sequence. Call export_gerber() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

132. DD-COMP-130 - FirmwareReleaseTagger

DD-COMP-130 **FirmwareReleaseTagger** | Service | Module: VCSModule | Pattern: Command | SW-REQ-108

`@trace DD-COMP-130 SW-REQ-108`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void tagger_init(void)`	void	Initialise all state for FirmwareReleaseTagger
`main.c`	`static uint8_t tagger_run(void *firmware_manifest)`	uint8_t - 0 success, non-zero error code	Execute core logic for FirmwareReleaseTagger
`main.c`	`static uint8_t load_manifest(void)`	uint8_t	Helper function: load manifest
`main.c`	`static uint8_t compute_tag(void)`	uint8_t	Helper function: compute tag
`main.c`	`static uint8_t write_tag_file(void)`	uint8_t	Helper function: write tag file

Doxygen Comments

```
/**
* @brief Initialise FirmwareReleaseTagger.
* @trace DD-COMP-130 SW-REQ-108
*/
/**
* @brief Run FirmwareReleaseTagger process cycle.
* @param firmware_manifest Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-130 SW-REQ-108
*/
/**
* @brief Load manifest.
* @trace DD-COMP-130 SW-REQ-108
*/
/**
* @brief Compute tag.
```

```
 * @trace DD-COMP-130 SW-REQ-108
 */
 /**
 * @brief Write tag file.
 * @trace DD-COMP-130 SW-REQ-108
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load firmware metadata
2	validate version and build info
3	compute release tag
4	update release record
5	emit tag artifact
6	log completion

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoding version strings
2	mutating input files
3	silencing validation errors
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'tag' (e.g. tagresult).`

2	State struct: static FirmwareReleaseTagger_t tagstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_130_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} FirmwareReleaseTagger_t;`
	State struct for FirmwareReleaseTagger

`void tagger_init(void);`	Prototype: initialise FirmwareReleaseTagger
`uint8_t tagger_run(void *p_data);`	Prototype: run FirmwareReleaseTagger process cycle
`uint8_t load_manifest(void);`	Prototype: load manifest
`uint8_t compute_tag(void);`	Prototype: compute tag
`uint8_t write_tag_file(void);`	Prototype: write tag file

`typedef enum {`
`DD_COMP_130_ERR_NONE = 0x00U,`
`DD_COMP_130_ERR_NULL = 0x01U,`
`DD_COMP_130_ERR_INVALID = 0x02U,`
`DD_COMP_130_ERR_TIMEOUT = 0x03U`
`} DD_COMP_130_ERR_t;`
	Error code enum for FirmwareReleaseTagger

main.c - What to Add

Item	Purpose

`static FirmwareReleaseTagger_t tagstate = {0};`	Module-level zero-initialised state for FirmwareReleaseTagger

`/* Process function logic order:`
`* Step 1: load firmware metadata`
`* Step 2: validate version and build info`
`* Step 3: compute release tag`
`* Step 4: update release record`
`* Step 5: emit tag artifact`
`* Step 6: log completion`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Command. Call tagger_init() once in main() init sequence. Call tagger_run() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

133. DD-COMP-131 - CalibrationDocIncluder

DD-COMP-131	**CalibrationDocIncluder** | Service | Module: ReleasePackagingModule | Pattern: Facade | SW-REQ-109

`@trace DD-COMP-131 SW-REQ-109`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void initInclude(void)`	void	Initialise all state for CalibrationDocIncluder
main.c	`static uint8_t includeDocs(void *configPath)`	uint8_t - 0 success, non-zero error code	Execute core logic for CalibrationDocIncluder
main.c	`static uint8_t loadManifest(void)`	uint8_t	Helper function: loadManifest
main.c	`static uint8_t filterCalDocs(void)`	uint8_t	Helper function: filterCalDocs
main.c	`static uint8_t attachDocs(void)`	uint8_t	Helper function: attachDocs

Doxygen Comments

```
/**
* @brief Initialise CalibrationDocIncluder.
* @trace DD-COMP-131 SW-REQ-109
*/
/**
* @brief Run CalibrationDocIncluder process cycle.
* @param configPath Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-131 SW-REQ-109
*/
/**
* @brief Loadmanifest.
* @trace DD-COMP-131 SW-REQ-109
*/
/**
```

```
 * @brief Filtercaldocs.
 * @trace DD-COMP-131 SW-REQ-109
 */
 /**
 * @brief Attachdocs.
 * @trace DD-COMP-131 SW-REQ-109
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	load inclusion configuration
2	scan calibration document sources
3	filter documents by criteria
4	merge selected documents into package
5	update metadata references
6	report inclusion results

Forbidden - Must NOT Do

#	Forbidden Rule
1	including duplicate documents
2	missing metadata updates
3	hardcoded paths
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'cal' (e.g. calresult).`

2	State struct: static CalibrationDocIncluder_t calstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_131_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} CalibrationDocIncluder_t;`
	State struct for CalibrationDocIncluder

`void initInclude(void);`	Prototype: initialise CalibrationDocIncluder
`uint8_t includeDocs(void *p_data);`	Prototype: run CalibrationDocIncluder process cycle
`uint8_t loadManifest(void);`	Prototype: loadManifest
`uint8_t filterCalDocs(void);`	Prototype: filterCalDocs
`uint8_t attachDocs(void);`	Prototype: attachDocs

`typedef enum {`
`DD_COMP_131_ERR_NONE = 0x00U,`
`DD_COMP_131_ERR_NULL = 0x01U,`
`DD_COMP_131_ERR_INVALID = 0x02U,`
`DD_COMP_131_ERR_TIMEOUT = 0x03U`
`} DD_COMP_131_ERR_t;`
	Error code enum for CalibrationDocIncluder

main.c - What to Add

Item	Purpose

`static CalibrationDocIncluder_t calstate = {0};`	Module-level zero-initialised state for CalibrationDocIncluder

`/* Process function logic order:`
`* Step 1: load inclusion configuration`
`* Step 2: scan calibration document sources`
`* Step 3: filter documents by criteria`
`* Step 4: merge selected documents into package`
`* Step 5: update metadata references`
`* Step 6: report inclusion results`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call initInclude() once in main() init sequence. Call includeDocs() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

134. DD-COMP-132 - TestReportGenerator

DD-COMP-132	**TestReportGenerator** | Service | Module: TestReportingModule | Pattern: Builder | SW-REQ-110

`@trace DD-COMP-132 SW-REQ-110`

Function Signatures

File	Signature	Returns	Purpose
main.c	`static void setup(void)`	void	Initialise all state for TestReportGenerator
main.c	`static uint8_t generate(void *test_results)`	uint8_t - 0 success, non-zero error code	Execute core logic for TestReportGenerator
main.c	`static uint8_t load_tests(void)`	uint8_t	Helper function: load tests
main.c	`static uint8_t calc_stats(void)`	uint8_t	Helper function: calc stats
main.c	`static uint8_t render_report(void)`	uint8_t	Helper function: render report

Doxygen Comments

```
/**
* @brief Initialise TestReportGenerator.
* @trace DD-COMP-132 SW-REQ-110
*/
/**
* @brief Run TestReportGenerator process cycle.
* @param test_results Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-132 SW-REQ-110
*/
/**
* @brief Load tests.
* @trace DD-COMP-132 SW-REQ-110
*/
/**
* @brief Calc stats.
```

```
 * @trace DD-COMP-132 SW-REQ-110
 */
 /**
 * @brief Render report.
 * @trace DD-COMP-132 SW-REQ-110
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	initialize configuration and templates
2	load test results from source
3	compute summary metrics
4	format report sections
5	assemble final report output
6	store or emit report

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	blocking I/O in main thread
3	mixing data parsing with rendering
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'trg' (e.g. trgresult).`

2	State struct: static TestReportGenerator_t trgstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_132_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} TestReportGenerator_t;`
	State struct for TestReportGenerator

`void setup(void);`	Prototype: initialise TestReportGenerator
`uint8_t generate(void *p_data);`	Prototype: run TestReportGenerator process cycle
`uint8_t load_tests(void);`	Prototype: load tests
`uint8_t calc_stats(void);`	Prototype: calc stats
`uint8_t render_report(void);`	Prototype: render report

`typedef enum {`
`DD_COMP_132_ERR_NONE = 0x00U,`
`DD_COMP_132_ERR_NULL = 0x01U,`
`DD_COMP_132_ERR_INVALID = 0x02U,`
`DD_COMP_132_ERR_TIMEOUT = 0x03U`
`} DD_COMP_132_ERR_t;`
	Error code enum for TestReportGenerator

main.c - What to Add

Item	Purpose

`static TestReportGenerator_t trgstate = {0};`	Module-level zero-initialised state for TestReportGenerator

`/* Process function logic order:`
`* Step 1: initialize configuration and templates`
`* Step 2: load test results from source`
`* Step 3: compute summary metrics`
`* Step 4: format report sections`
`* Step 5: assemble final report output`
`* Step 6: store or emit report`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Builder. Call setup() once in main() init sequence. Call generate() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

135. DD-COMP-133 - DemonstrationVideoIncluder

DD-COMP-133	**DemonstrationVideoIncluder** | Service | Module: ReleasePackagingModule | Pattern: Facade | SW-REQ-111

`@trace DD-COMP-133 SW-REQ-111`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init_demo_video(void)`	void	Initialise all state for DemonstrationVideoIncluder
`main.c`	`static uint8_t include_demo_video(void *video_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for DemonstrationVideoIncluder
`main.c`	`static uint8_t load_video_meta(void)`	uint8_t	Helper function: load video meta
`main.c`	`static uint8_t validate_video(void)`	uint8_t	Helper function: validate video
`main.c`	`static uint8_t embed_video_tag(void)`	uint8_t	Helper function: embed video tag

Doxygen Comments

```
/**
* @brief Initialise DemonstrationVideoIncluder.
* @trace DD-COMP-133 SW-REQ-111
*/
/**
* @brief Run DemonstrationVideoIncluder process cycle.
* @param video_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-133 SW-REQ-111
*/
/**
* @brief Load video meta.
* @trace DD-COMP-133 SW-REQ-111
*/
/**
```

```
* @brief Validate video.
* @trace DD-COMP-133 SW-REQ-111
*/
/**
* @brief Embed video tag.
* @trace DD-COMP-133 SW-REQ-111
*/
```

Step-by-Step Logic (process function order)

Step	Action
1	load configuration and templates
2	receive video reference input
3	validate format and accessibility
4	generate embed markup
5	inject markup into target output
6	return updated content

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded URLs
2	blocking network calls
3	silent failures
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'demo' (e.g. demoresult).`

2	State struct: static DemonstrationVideoIncluder_t demostate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_133_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} DemonstrationVideoIncluder_t;`
	State struct for DemonstrationVideoIncluder

`void init_demo_video(void);`	Prototype: initialise DemonstrationVideoIncluder
`uint8_t include_demo_video(void *p_data);`	Prototype: run DemonstrationVideoIncluder process cycle
`uint8_t load_video_meta(void);`	Prototype: load video meta
`uint8_t validate_video(void);`	Prototype: validate video
`uint8_t embed_video_tag(void);`	Prototype: embed video tag

`typedef enum {`
`DD_COMP_133_ERR_NONE = 0x00U,`
`DD_COMP_133_ERR_NULL = 0x01U,`
`DD_COMP_133_ERR_INVALID = 0x02U,`
`DD_COMP_133_ERR_TIMEOUT = 0x03U`
`} DD_COMP_133_ERR_t;`
	Error code enum for DemonstrationVideoIncluder

main.c - What to Add

Item	Purpose

`static DemonstrationVideoIncluder_t demostate = {0};`	Module-level zero-initialised state for DemonstrationVideoIncluder

`/* Process function logic order:`
`* Step 1: load configuration and templates`
`* Step 2: receive video reference input`
`* Step 3: validate format and accessibility`
`* Step 4: generate embed markup`
`* Step 5: inject markup into target output`
`* Step 6: return updated content`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Facade. Call init_demo_video() once in main() init sequence. Call include_demo_video() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).

136. DD-COMP-134 - ReleasePackageBuilder

DD-COMP-134	**ReleasePackageBuilder** | Service | Module: ReleasePackagingModule | Pattern: Builder | SW-REQ-106

`@trace DD-COMP-134 SW-REQ-106`

Function Signatures

File	Signature	Returns	Purpose
`main.c`	`static void init_builder(void)`	void	Initialise all state for ReleasePackageBuilder
`main.c`	`static uint8_t build_release(void *manifest_path)`	uint8_t - 0 success, non-zero error code	Execute core logic for ReleasePackageBuilder
`main.c`	`static uint8_t load_manifest(void)`	uint8_t	Helper function: load manifest
`main.c`	`static uint8_t collect_artifacts(void)`	uint8_t	Helper function: collect artifacts
`main.c`	`static uint8_t write_bundle(void)`	uint8_t	Helper function: write bundle

Doxygen Comments

```
/**
* @brief Initialise ReleasePackageBuilder.
* @trace DD-COMP-134 SW-REQ-106
*/
/**
* @brief Run ReleasePackageBuilder process cycle.
* @param manifest_path Pointer to input data (must not be NULL).
* @return 0 on success, error code on failure.
* @trace DD-COMP-134 SW-REQ-106
*/
/**
* @brief Load manifest.
* @trace DD-COMP-134 SW-REQ-106
*/
/**
```

```
 * @brief Collect artifacts.
 * @trace DD-COMP-134 SW-REQ-106
 */
 /**
 * @brief Write bundle.
 * @trace DD-COMP-134 SW-REQ-106
 */
```

Step-by-Step Logic (process function order)

Step	Action
1	validate inputs and configuration
2	load release manifest and metadata
3	collect and verify artifacts
4	assemble package structure
5	generate checksums and version info
6	write output package

Forbidden - Must NOT Do

#	Forbidden Rule
1	hardcoded paths
2	silent failures
3	skipping validation
4	No malloc/calloc/free anywhere in this component
5	No recursive function calls - use iterative logic only
6	No processing inside ISR - set flag and return only
7	No blocking while() loops - use state machine pattern

Variable Naming Rules

#	Rule
1	`All local variables prefixed 'rpb' (e.g. rpbresult).`

2	State struct: static ReleasePackageBuilder_t rpbstate = {0};
3	Counters: uint32_t, suffix _cnt (e.g. retry_cnt).
4	Flags: volatile uint8_t, suffix _flag (e.g. rx_ready_flag).
5	Buffers: uint8_t array, suffix _buf, size via #define constant.
6	Error returns: use DD_COMP_134_ERR_t enum values, never raw integers.
7	No variables declared at non-function scope except static module state.

main.h - What to Add

Item	Purpose

`typedef struct {`
`uint8_t state;`
`uint8_t status;`
`uint32_t tick_cnt;`
`} ReleasePackageBuilder_t;`
	State struct for ReleasePackageBuilder

`void init_builder(void);`	Prototype: initialise ReleasePackageBuilder
`uint8_t build_release(void *p_data);`	Prototype: run ReleasePackageBuilder process cycle
`uint8_t load_manifest(void);`	Prototype: load manifest
`uint8_t collect_artifacts(void);`	Prototype: collect artifacts
`uint8_t write_bundle(void);`	Prototype: write bundle

`typedef enum {`
`DD_COMP_134_ERR_NONE = 0x00U,`
`DD_COMP_134_ERR_NULL = 0x01U,`
`DD_COMP_134_ERR_INVALID = 0x02U,`
`DD_COMP_134_ERR_TIMEOUT = 0x03U`
`} DD_COMP_134_ERR_t;`
	Error code enum for ReleasePackageBuilder

main.c - What to Add

Item	Purpose

`static ReleasePackageBuilder_t rpbstate = {0};`	Module-level zero-initialised state for ReleasePackageBuilder

`/* Process function logic order:`
`* Step 1: validate inputs and configuration`
`* Step 2: load release manifest and metadata`
`* Step 3: collect and verify artifacts`
`* Step 4: assemble package structure`
`* Step 5: generate checksums and version info`
`* Step 6: write output package`
`*/`
	Ordered implementation checklist for process function

Notes: Design pattern: Builder. Call init_builder() once in main() init sequence. Call build_release() from main loop or RTOS task tick. All code in main.c (static) and main.h (declarations only).